PUC Minas



Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Sumário

- Documentação
  - Diagrama da arquitetura
  - Fluxos de solicitação, endpoints e dependências
  - Documentando as rotas

Documentação

- A arquitetura de microsserviços é uma abordagem de design de software em que um aplicativo é composto por vários serviços independentes, cada um executando uma função específica e se comunicando entre si por meio de APIs.

- Embora os microsserviços ofereçam benefícios como escalabilidade, flexibilidade e facilidade de manutenção, **também podem apresentar desafios** que, se não forem gerenciados adequadamente, podem resultar em **defasagem tecnológica**.

Documentação

- Alguns motivos pelos quais isso pode acontecer incluem:
  - **1) Diversidade tecnológica**: Em um ambiente de microsserviços, diferentes equipes podem ser responsáveis por diferentes serviços, o que pode levar à escolha de tecnologias diversas para cada serviço.

  - Isso pode resultar em uma variedade de tecnologias e versões em uso, tornando desafiador manter todas as partes atualizadas e compatíveis.

Documentação

- Alguns motivos pelos quais isso pode acontecer incluem:
  - **2) Crescimento orgânico**: À medida que um sistema baseado em microsserviços cresce, novos serviços podem ser adicionados ao longo do tempo para atender a novos requisitos.

  - Essa expansão orgânica pode levar a uma arquitetura mais complexa, com partes do sistema usando tecnologias mais antigas e outras adotando tecnologias mais recentes.

Documentação

- Alguns motivos pelos quais isso pode acontecer incluem:
  - **3) Comunicação e consistência**: A comunicação entre microsserviços muitas vezes ocorre por meio de APIs.

  - Diferenças nas versões dessas APIs podem resultar em incompatibilidades, o que pode ser um desafio quando se tenta manter a consistência nas interações entre os serviços.

Documentação

- Alguns motivos pelos quais isso pode acontecer incluem:
  - **4) Gerenciamento de dados**: O gerenciamento de dados distribuídos em um ambiente de microsserviços pode ser complexo.

  - Diferentes serviços podem utilizar diferentes tecnologias de armazenamento de dados, o que pode resultar em desafios de integração e consistência de dados.

Documentação

- Alguns motivos pelos quais isso pode acontecer incluem:
  - **5) Desafios de monitoramento e depuração**: Monitorar e depurar sistemas distribuídos pode ser mais desafiador do que em sistemas monolíticos.

  - A complexidade aumenta à medida que mais serviços são introduzidos, tornando essencial o uso de ferramentas adequadas para monitoramento e rastreamento.

Documentação

- Alguns motivos pelos quais isso pode acontecer incluem:
  - **6) Atualizações assíncronas**: Em um ambiente de microsserviços, as atualizações e melhorias podem ocorrer de forma assíncrona em diferentes serviços.

  - Isso pode resultar em defasagens tecnológicas quando alguns serviços são atualizados com tecnologias mais recentes antes de outros.

Documentação

- Para mitigar esses desafios, é crucial implementar práticas robustas de gerenciamento de configuração, automação, testes contínuos, manutenção regular, documentação abrangente, monitoramento proativo, logging eficiente e dashboards informativos.

- Esses elementos garantem não apenas a estabilidade e confiabilidade do sistema, mas também facilitam a identificação e resolução rápida de problemas, contribuindo para a eficiência operacional e o sucesso contínuo da arquitetura de microsserviços.

Documentação

- A arquitetura de microsserviços tem o potencial de causar maior defasagem técnica – uma das principais contrapartidas de se adotar os microsserviços.

- Como regra, a defasagem técnica tende a crescer com a **velocidade do desenvolvedor**: quanto mais rapidamente um serviço pode gerar iterações, ser alterado e implantado, mais frequentemente atalhos e patches são ativados.

Documentação

- Clareza e estrutura organizacionais relativas à documentação e compreensão de um microsserviço contornam esta defasagem técnica e evitam muito a confusão, a falta de percepção e compreensão da arquitetura que tendem a acompanhar essa desafagem.

- Reduzir a defasagem técnica não é a única razão para tornar uma boa **documentação** um dos princípios de disponibilidade de produção: fazê-lo seria como uma consideração a *posteriori*.

Documentação

- Assim como os demais padrões de disponibilidade de produção, a documentação e sua compreensão influenciam de forma direta e mensurável na disponibilidade de um microsserviço.

- Um exercício simples para provar isto é colocar a equipe de desenvolvimento em uma sala em frente a um quadro branco e pedir que eles esbocem a arquitetura e os detalhes importantes do serviço. **Veremos que o conhecimento e a compreensão do serviço não são coesos ou coerentes em todo o grupo**.

Documentação

- Um desenvolvedor conhecerá um aspecto sobre a aplicação que ninguém mais conhece, enquanto um segundo desenvolvedor terá uma compreensão tão diferente do microsserviço que você se perguntará se eles estão contribuindo para o mesmo código-base.



Documentação

- Quando chegar o momento de as alterações de código serem revisadas, tecnologias trocadas ou recursos acrescentados, a falta de alinhamento do conhecimento e da compreensão resultará em um projeto e/ou uma evolução de **microsserviços que não são prontos para a produção**, contendo sérios defeitos que prejudicam a capacidade do serviço de atender com confiabilidade o tráfego de produção.



Documentação

- Esta confusão e os problemas causados por ela podem ser evitados facilmente e com sucesso se exigirmos que cada microsserviço siga um conjunto rigorosamente padronizado de requisitos de **documentação**.

- A documentação precisa conter todo o conhecimento essencial (fatos) sobre um microsserviço, incluindo um **diagrama de arquitetura do serviço**, um guia de integração e de desenvolvimento, detalhes sobre o **fluxo de solicitações** e quaisquer **endpoints de API**, e um roteiro de prontidão para cada alerta do serviço.

Documentação

- A compreensão de um microsserviço pode ser alcançada de várias formas: A primeira, como vimos, é colocando a equipe de desenvolvimento em uma sala de conferência e pedindo que seus integrantes esbocem o **diagrama de arquitetura do serviço**.

- Graças à velocidade sempre maior de desenvolvimento, os microsserviços mudam radicalmente em diferentes momentos de seu ciclo de vida.



Documentação

- Ao tornar essas revisões de arquitetura parte do processo de cada equipe e programá-las regularmente, você pode garantir que o conhecimento e a compreensão sobre quaisquer alterações no microsserviço sejam disseminados para toda a equipe.

- Para abordar o segundo aspecto da compreensão de um microsserviço, precisamos subir um nível de abstração e considerar os próprios padrões de disponibilidade de produção.

Documentação

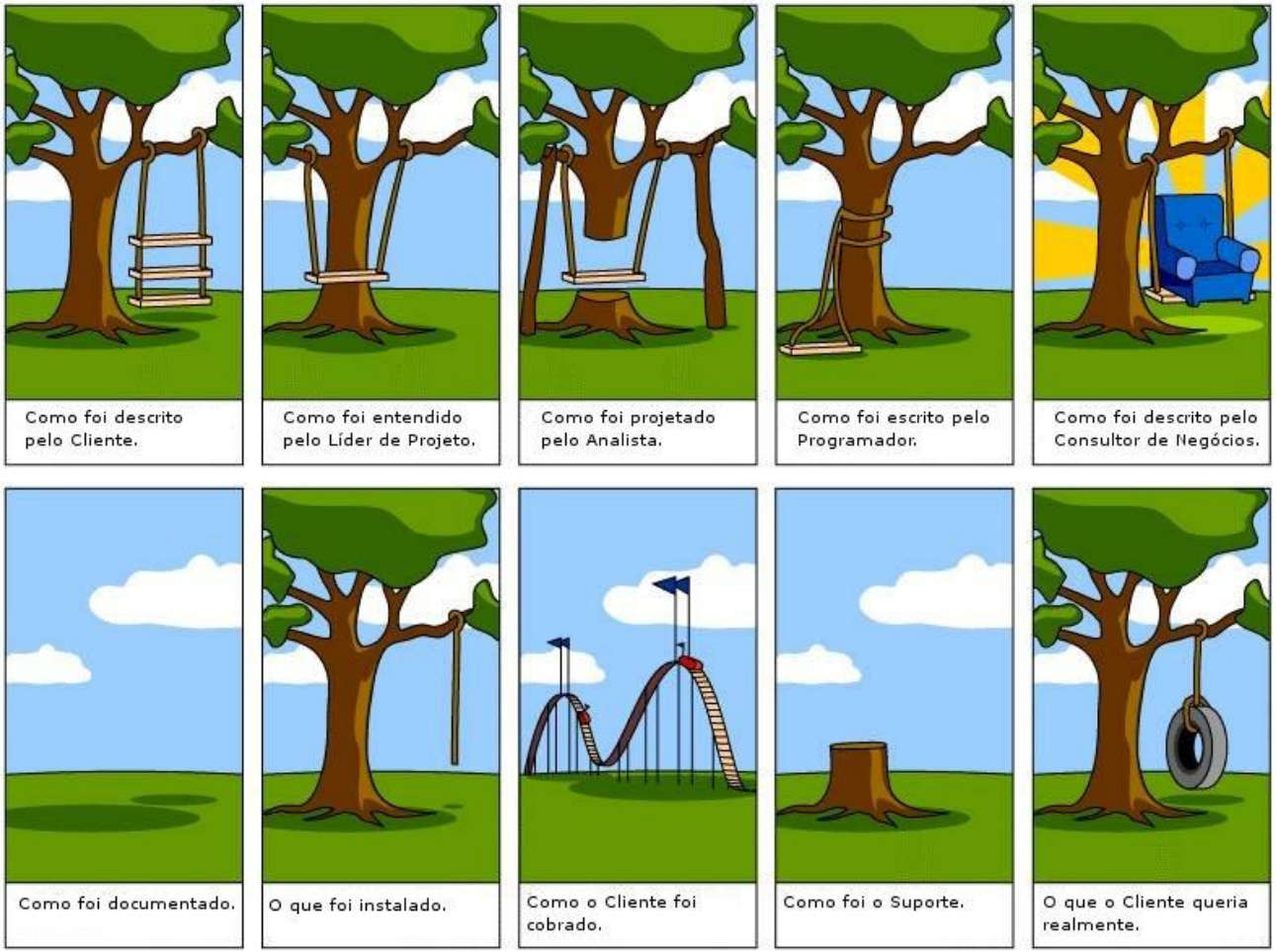
- Uma grande parte da compreensão de um microsserviço é capturada determinando **se um microsserviço está pronto para produção ou não** e qual seu status em relação aos padrões de disponibilidade de produção e seus requisitos individuais.

- Isso pode ser feito de muitas formas; uma das quais é executar auditorias para determinar se um microsserviço atende aos requisitos e então criar um roteiro para o serviço, detalhando como levá-lo até um estado de disponibilidade para produção. A verificação dos requisitos também pode ser **automatizada** para toda a organização.

Documentação

- **Requisitos de documentação**:
  - Os requisitos para construir um microsserviço bem documentado são:
    - Documentação completa, atualizada e centralizada contendo todas as informações relevantes e essenciais sobre o microsserviço;
    - Compreensão organizacional nos níveis de desenvolvedor, equipe e ecossistema.

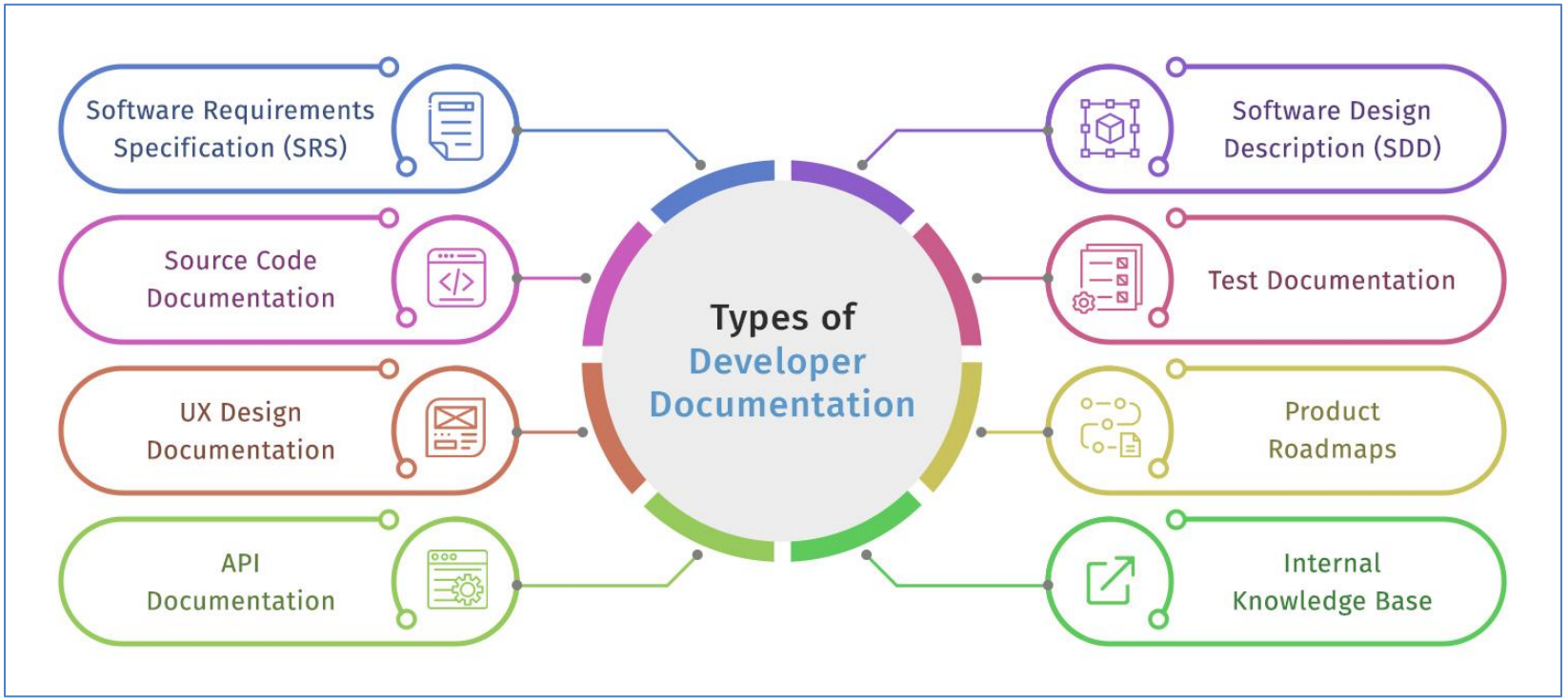




Quando falamos da importância de uma boa documentação de software, a imagem ao lado é talvez a melhor ilustração.

É uma imagem “batida”, mas é sempre bom revisitá-la.

- Leitura sugerida:
  - https://technicalwriterhq.com/documentation/software-documentation/



Documentação

- Pergunte aos desenvolvedores que trabalham com ecossistema de microsserviços **quais são suas principais preocupações**, e eles recitarão uma lista de recursos que ainda serão implementados, de erros a serem corrigidos, de dependências que estão causando problemas e de coisas que eles não entendem sobre seu próprio serviço e as dependências das quais ele depende.

Documentação

- Quando pedimos para dar mais detalhes sobre os dois últimos itens, eles tendem a dar respostas parecidas: eles não entendem como funciona, é uma caixa misteriosa, e a documentação é completamente inútil.

- 2 reais ou um microsserviço misterioso?



Documentação

- Uma documentação de baixa qualidade das dependências e ferramentas internas retarda o trabalho dos desenvolvedores e afeta sua capacidade de tornar seus próprios serviços prontos para produção.

- Ela impede que eles usem dependências e ferramentas internas corretamente e desperdiça inúmeras horas de engenharia, pois às vezes a única maneira de descobrir o que um serviço ou uma ferramenta faz (sem uma documentação adequada) é fazer **engenharia reversa** até entender seu funcionamento.

Documentação

- Uma documentação de baixa qualidade de um serviço também prejudica a produtividade dos desenvolvedores que estão contribuindo com ele. Por exemplo, a falta de roteiros para um turno de plantão significa que quem estiver de plantão terá de solucionar cada problema desde o início toda vez.

- Sem um guia de bordo, cada novo desenvolvedor que estiver trabalhando no serviço precisará **começar do zero** para entender como funciona o serviço.

Documentação

- Pontos únicos de falha e problemas com o serviço passarão despercebidos até que causem uma interrupção.

- Novos recursos adicionados ao serviço frequentemente não terão uma ideia geral de como o serviço realmente funciona.

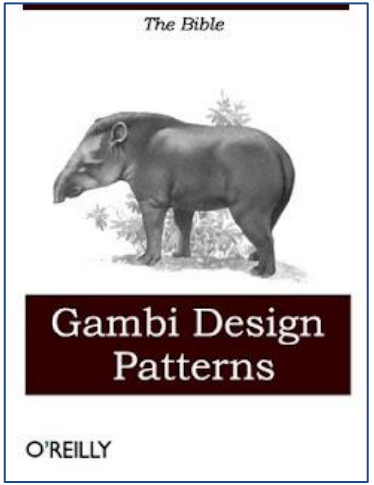


Documentação

- É nesse momento que as gambiarras começam a aparecer.







Documentação

- O objetivo de uma boa documentação pronta para produção é criar e manter um repositório central de conhecimento sobre o serviço.

- Compartilhar esse conhecimento tem dois componentes: os fatos objetivos sobre o serviço e a compreensão organizacional do que o serviço faz e onde ele se encaixa na organização como um todo.

Documentação

- O problema de uma documentação de baixa qualidade pode ser dividido em dois subproblemas: **falta de documentação (os fatos) e falta de compreensão**.

- Resolver esses dois subproblemas requer padronizar a documentação para todos os microsserviços e criar estruturas organizacionais para compartilhar o conhecimento sobre o microsserviço.

Documentação

- Um serviço pronto para produção é documentado e compreendido quando:
  - Ele tem uma documentação abrangente.
  - Sua documentação é atualizada regularmente.
  - Sua documentação contém uma **descrição do microsserviço**; **um diagrama da arquitetura**; informações de contato e um roteiro de plantão; links para informações importantes; um guia de bordo e desenvolvimento; **informações sobre o(s) fluxo(s) de solicitação, os endpoints e as dependências do serviço**; e as respostas para perguntas mais frequentes.
  - Ele é bem compreendido nos âmbitos do desenvolvedor, da equipe e da organização.
  - Ele mantém um conjunto de **padrões de disponibilidade de produção** e cumpre os requisitos associados.
  - Sua arquitetura é revista e auditada frequentemente.

Documentação

- A documentação de todos os microsserviços em uma organização de engenharia de software deve ser armazenada em um local centralizado, compartilhado e de fácil acesso.

- Qualquer desenvolvedor de qualquer equipe deve ser capaz de encontrar a documentação de cada microsserviço sem dificuldade. Um site interno contendo a documentação de todos os microsserviços e das ferramentas internas tende a ser o melhor meio para isso.

Documentação

- Arquivos README e comentários no código **não** são documentação.

- Muitos desenvolvedores limitam a documentação de seus microsserviços a um arquivo README em seu repositório ou comentários espalhados ao longo do código.

- Embora um arquivo README seja essencial e todo código de microsserviço deva conter comentários adequados, isso não é documentação pronta para produção, e exige que os desenvolvedores vasculhem o código à procura de informações.

Documentação

- A documentação deve ser atualizada regularmente. Sempre que uma mudança significativa for feita no serviço, a documentação deverá ser atualizada.
- Por exemplo:
  - Se um novo endpoint de API for adicionado, as informações sobre o endpoint também precisarão ser acrescentadas à documentação.
  - Se um novo alerta for adicionado, então, instruções passo a passo sobre como selecionar, mitigar e resolver o alerta também deverão ser acrescentadas ao roteiro de plantão do serviço.
  - Se uma nova dependência for adicionada, então, informações sobre essa dependência deverão ser acrescentadas à documentação.

Documentação

- A melhor maneira de fazê-lo é tornar o processo de atualizar a documentação parte do fluxo de trabalho de desenvolvimento.

- Se a atualização da documentação for considerada uma tarefa separada do (e secundária ao) desenvolvimento, então ela nunca será feita e se tornará parte da defasagem técnica do serviço. Para reduzir a defasagem técnica, os desenvolvedores devem ser encorajados (ou, se necessário, obrigados) a acompanhar toda mudança significativa de código com uma atualização da documentação.

Documentação

- A documentação deve ser abrangente e útil. Ela deve conter todos os fatos relevantes e importantes sobre o serviço.

- Depois de ler a documentação, um desenvolvedor deve saber como contribuir para o serviço e sua arquitetura, o contato e as informações do plantão para o serviço; como o serviço funciona (fluxos de solicitação, endpoints, dependências, etc.); como selecionar, mitigar e corrigir incidentes e interrupções, assim como resolver alertas gerados pelo serviço; e ter respostas para perguntas mais frequentes sobre o serviço.

Documentação

- E, o mais importante, a documentação deve ser escrita de forma clara e fácil de entender.

- Uma documentação cheia de jargões é inútil, uma documentação excessivamente técnica e que não explica os aspectos que podem ser exclusivos do serviço também é inútil, assim como a documentação que não entra em nenhum detalhe significativo.

Documentação

- O objetivo de escrever uma documentação boa e clara é que ela possa ser compreendida por qualquer desenvolvedor, gerente, gerente de produtos ou executivo dentro da empresa.

- Vamos agora nos aprofundar um pouco mais em cada um dos elementos de uma documentação de microsserviço pronto para produção.

Documentação

- **Descrição**:
  - A documentação de cada microsserviço deve começar com uma descrição do serviço. Ela deve ser curta, cativante e objetiva.

  - Por exemplo, se houver um microsserviço chamada **emissor-de-recibo** cuja finalidade seja enviar um recibo depois que o cliente concluir um pedido, a descrição deve ser:

Documentação

- **Descrição**:
  - Depois que um cliente faz um pedido, o emissor-de-recibo envia um recibo ao cliente via e-mail.

  - Parece óbvio, mas tratando-se de documentação o óbvio não existe. A descrição é essencial, pois garante que qualquer pessoa que encontrar a documentação saberá qual o papel desempenhado pelo microsserviço no ecossistema.

Sumário

- Documentação
  - **Diagrama da arquitetura**
  - Fluxos de solicitação, endpoints e dependências
  - Documentando as rotas

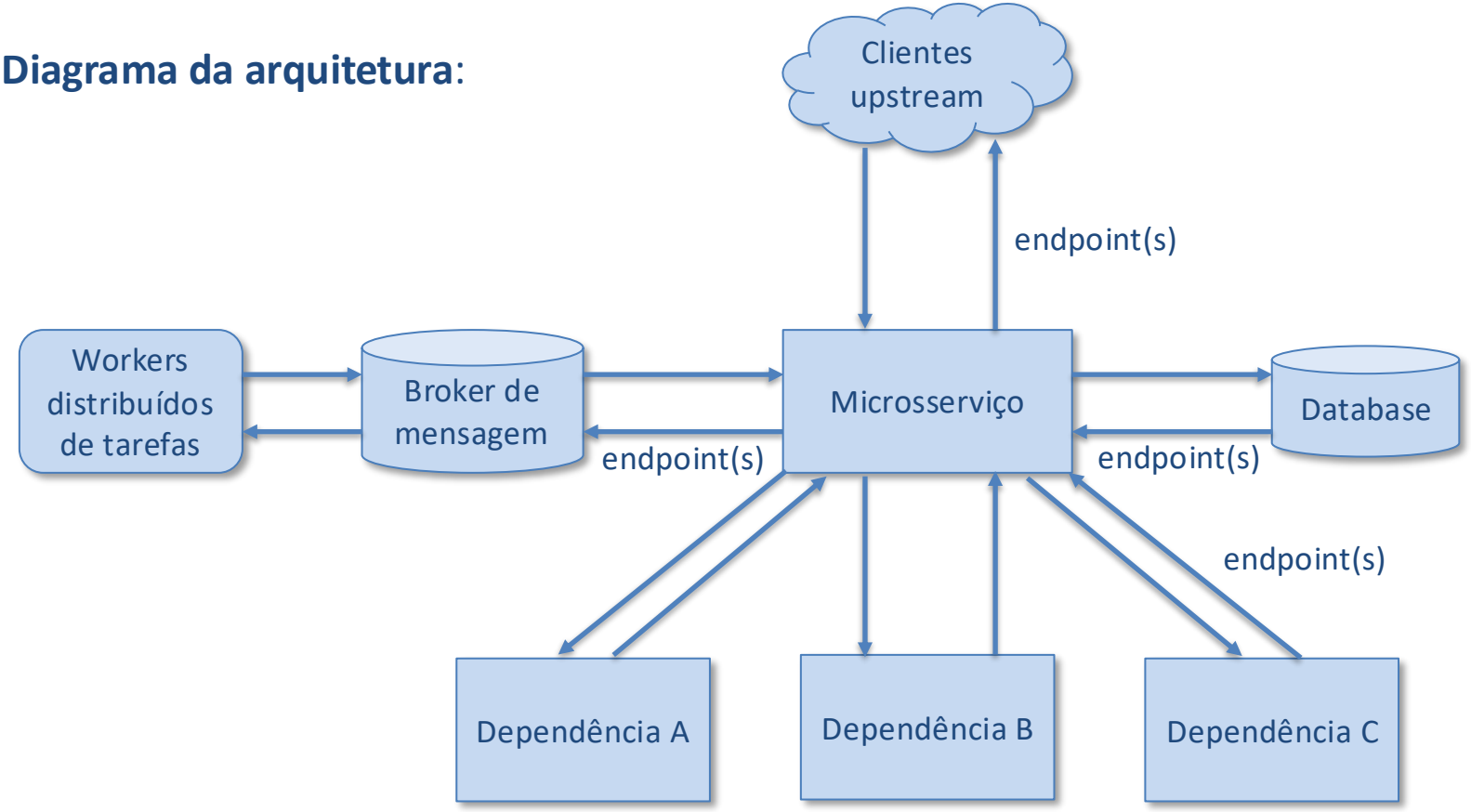
Documentação

- **Diagrama da arquitetura**:
  - A descrição do serviço deve ser seguida de um diagrama da arquitetura. Este diagrama deve detalhar a arquitetura do serviço, incluindo seus componentes, seus endpoints, o fluxo de solicitações, suas dependências (tanto upstream como downstream) e informações sobre quaisquer databases ou caches.

  - Vejamos um exemplo de diagrama de arquitetura.

Documentação

- **Diagrama da arquitetura**:



Documentação

- **Diagrama da arquitetura**:
  - Alguns conceitos importantes:
    - Workers:
      - Trabalhador (Worker) em computação distribuída: Em sistemas distribuídos, um “worker” é muitas vezes um nó de processamento que executa tarefas específicas. Essas tarefas podem ser atribuídas por um "coordenador" central em um ambiente de computação distribuída.

Documentação

- **Diagrama da arquitetura**:
  - Alguns conceitos importantes:
    - Workers:
      - Trabalhador (Worker) em sistemas Web: Em contextos relacionados a servidores web e aplicações assíncronas, um “worker” pode ser um processo que executa tarefas em segundo plano, muitas vezes desvinculado da resposta direta a requisições do usuário. Por exemplo, processar filas de tarefas, processamento assíncrono de solicitações, etc.

Documentação

- **Diagrama da arquitetura**:
  - Alguns conceitos importantes:
    - Workers:
      - Trabalhador (Worker) em sistemas de filas de mensagens: Em arquiteturas baseadas em filas de mensagens, um “worker” é um componente que consome mensagens de uma fila e executa tarefas associadas a essas mensagens. Isso é comum em sistemas de processamento de filas, onde várias instâncias de “workers” podem processar em paralelo para aumentar a eficiência.

Documentação

- **Diagrama da arquitetura**:
  - Alguns conceitos importantes:
    - Brokers:
      - Corretor (Broker) em microsserviços: Em microsserviços, um “broker” pode se referir a um serviço ou componente que facilita a comunicação e a coordenação entre microsserviços. Por exemplo, um “message broker” (corretor de mensagens) pode ser usado para permitir a troca assíncrona de mensagens entre microsserviços, auxiliando na integração e na comunicação entre eles.

Documentação

- **Diagrama da arquitetura**:
  - Alguns conceitos importantes:
    - Upstream:
      - Upstream em microsserviços: “Upstream” refere-se à direção que está na direção oposta ao fluxo de dados. Geralmente, isso implica a fonte de dados ou serviços. Por exemplo, se um microsserviço A chama um microsserviço B para obter dados, então A é considerado “upstream” de B.

Documentação

- **Diagrama da arquitetura**:
  - Alguns conceitos importantes:
    - Downstream:
      - Downstream em microsserviços: “Downstream” refere-se à direção que está na direção do fluxo de dados. É a parte do processo que consome os dados ou serviços fornecidos por um microsserviço “upstream”. Por exemplo, se um microsserviço B fornece dados para um microsserviço A, então A é considerado “downstream” de B.

Documentação

- **Diagrama da arquitetura**:
  - Diagramas de arquitetura são essenciais por vários motivos.
  - É quase impossível entender como e por que um microsserviço funciona apenas lendo o código, portanto um diagrama de arquitetura bem desenhado é uma descrição visual e um resumo de fácil entendimento do microsserviço.

Documentação

- **Diagrama da arquitetura**:
  - Esses diagramas também ajudam desenvolvedores a acrescentar novos recursos, isolando o funcionamento interno do serviço de modo que eles possam ver onde e como os novos recursos irão (ou não) se encaixar.

  - Eles revelam questões e problemas com o serviço que passariam despercebidos sem uma representação visual completa de sua arquitetura: é difícil descobrir os pontos de falha de um serviço apenas percorrendo as linhas de código, mas eles tendem a se destacar claramente em um diagrama de arquitetura preciso.

Sumário

- Documentação
  - Diagrama da arquitetura
  - **Fluxos de solicitação, endpoints e dependências**
  - Documentando as rotas

Documentação

- **Fluxos de solicitação, endpoints e dependências**:
  - A documentação também deve conter informações críticas sobre fluxos de solicitações, endpoints e dependências do microsserviço.

  - A documentação do fluxo de solicitações pode conter um diagrama dos fluxos de solicitações da aplicação. Isso pode ser o diagrama de arquitetura, se o fluxo de solicitações for detalhado adequadamente dentro do diagrama de arquitetura.

Documentação

- **Fluxos de solicitação, endpoints e dependências**:
  - Qualquer diagrama deve ser acompanhado por uma **descrição qualitativa** dos tipos de solicitações que são feitas para o microsserviço e de como elas são tratadas.

  - Este é também o lugar para documentar todos os endpoints de API do serviço. Geralmente é suficiente uma lista dos endpoints com seus nomes e uma descrição qualitativa de cada um, junto a suas respostas.

Documentação

- **Fluxos de solicitação, endpoints e dependências**:
  - Esta deve ser clara e compreensível o suficiente para que outro desenvolvedor que esteja trabalhando em uma equipe diferente possa ler as descrições dos endpoints de API de seu serviço e tratar seu microsserviço com superficialidade, acessando os endpoints com sucesso e recebendo as respostas esperadas.

  - Iremos inserir essas informações no próprio diagrama de arquitetura.

Documentação

- **Fluxos de solicitação, endpoints e dependências**:
  - Por último, temos as informações sobre as dependências do serviço.

  - Relacione as dependências, os endpoints relevantes dessas dependências e quaisquer solicitações que o serviço faça a elas, junto das informações sobre seus SLAs, de quaisquer alternativas/caches/backups em caso de falha e dos links para sua documentação e seus dashboards.

- Leituras sugeridas:

  Diagramação:

  - https://www.edrawsoft.com/pt/article/microservices-architecture-diagram.html
  - https://www.figma.com/community/file/989635781221754599/architecture-diagram-components
  - https://www.figma.com/community/file/989634471195357925/architecture-diagram-example-multiplayer

  Documentação:

  - https://helpjuice.com/blog/technical-documentation-software

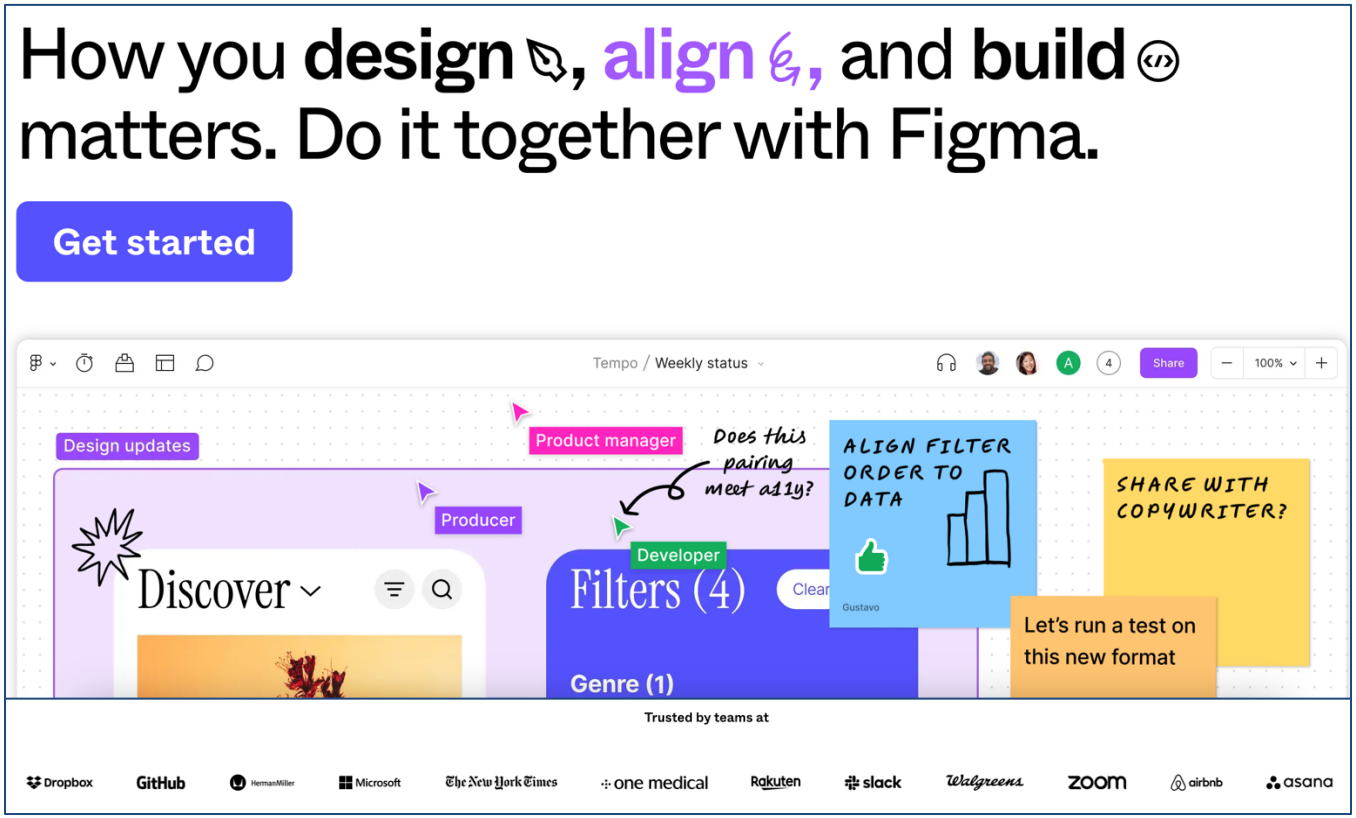
  Exemplos:

  - https://fga-eps-mds.github.io/mdseps.github.io/
    - https://fga-eps-mds.github.io/2018.2-CarDefense/docs/docarquitetura.html
    - https://fga-eps-mds.github.io/2018.2-Integra-Vendas/docs/doc-arquitetura
    - https://fga-eps-mds.github.io/2019.1-MaisMonitoria/docs/doc-arquitetura
    - https://fga-eps-mds.github.io/2020.1-eSaudeUnB-Wiki/documento_arquitetura/
    - https://fga-eps-mds.github.io/2020.1-Minacademy-Wiki/documentos/doc_arqui/
  - https://github.com/fga-eps-mds

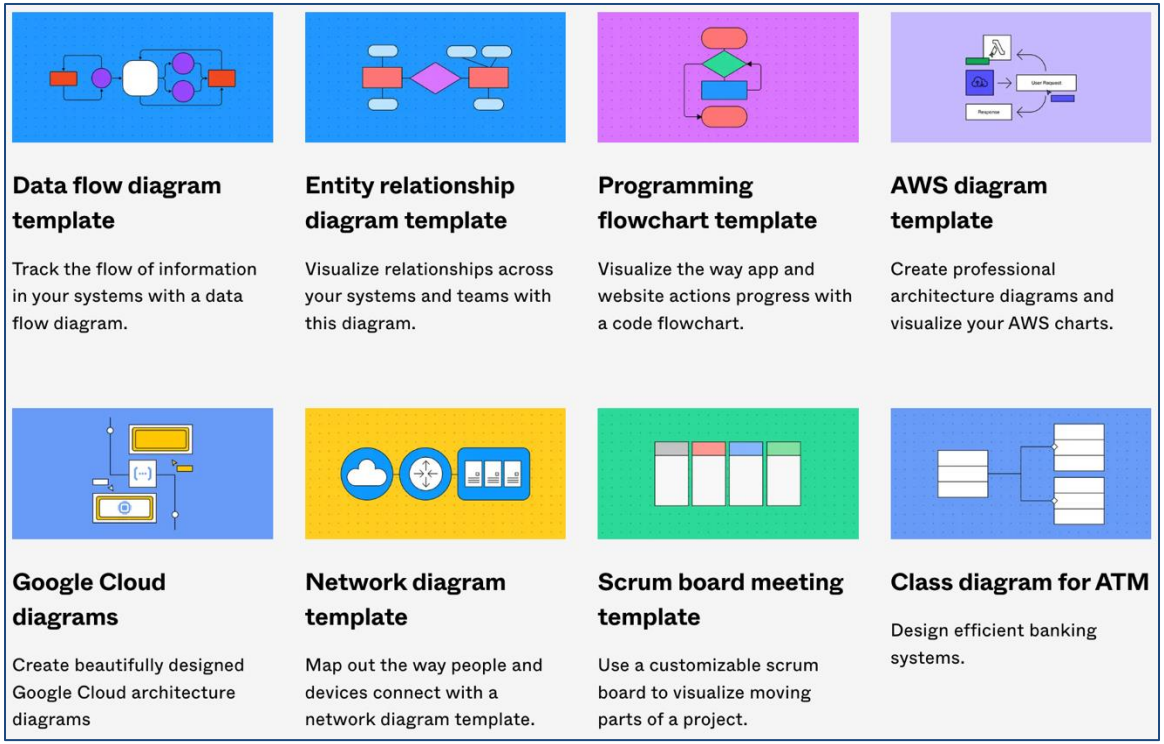
Documentação

- **Ferramentas para diagramação da arquitetura de microsserviços**:
  - Existem diversas ferramentas para auxiliar no processo de diagramação de uma arquitetura. Como por exemplo:
    - [Figma] https://www.figma.com/templates/engineering/
    - [Miro] https://miro.com/pt/modelos/desenvolvedores/
    - [EdrawMax] https://www.edrawmax.com/online/pt/store
    - [Lucidchart] https://www.lucidchart.com/pages/pt/modelos
    - [Draw.io] https://www.drawio.com/
    - [Gliffy] https://www.gliffy.com/

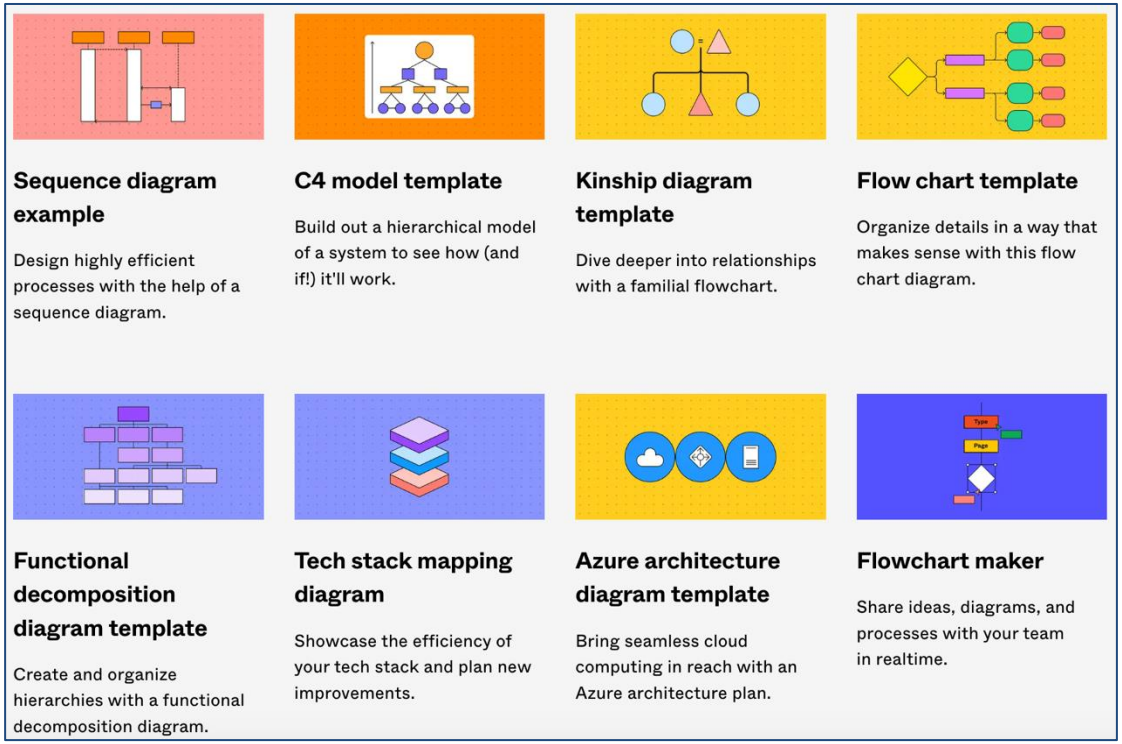
- https://www.figma.com/templates/engineering/
- Nesta disciplina, de Arquitetura de Aplicações Web, utilizaremos o **Figma**.
- **Figma**:



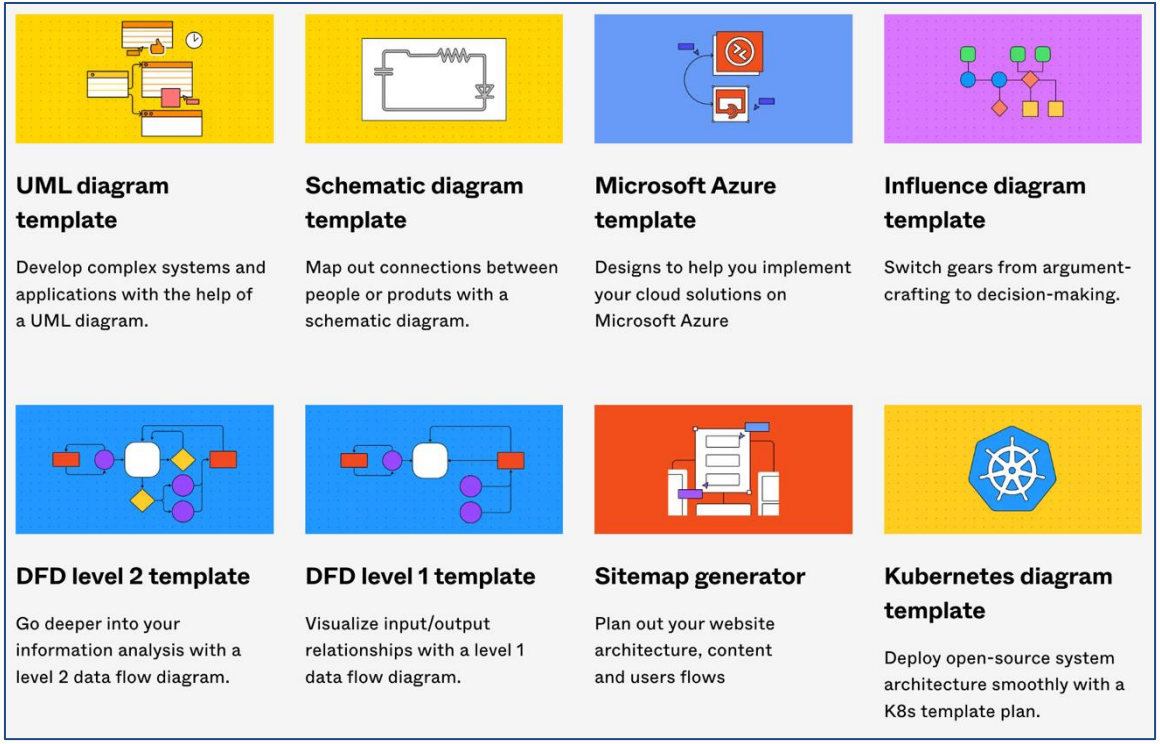
- **Figma**:
  - https://www.figma.com/templates/engineering/
  - Nesta disciplina, de Arquitetura de Aplicações Web, utilizaremos o **Figma**.



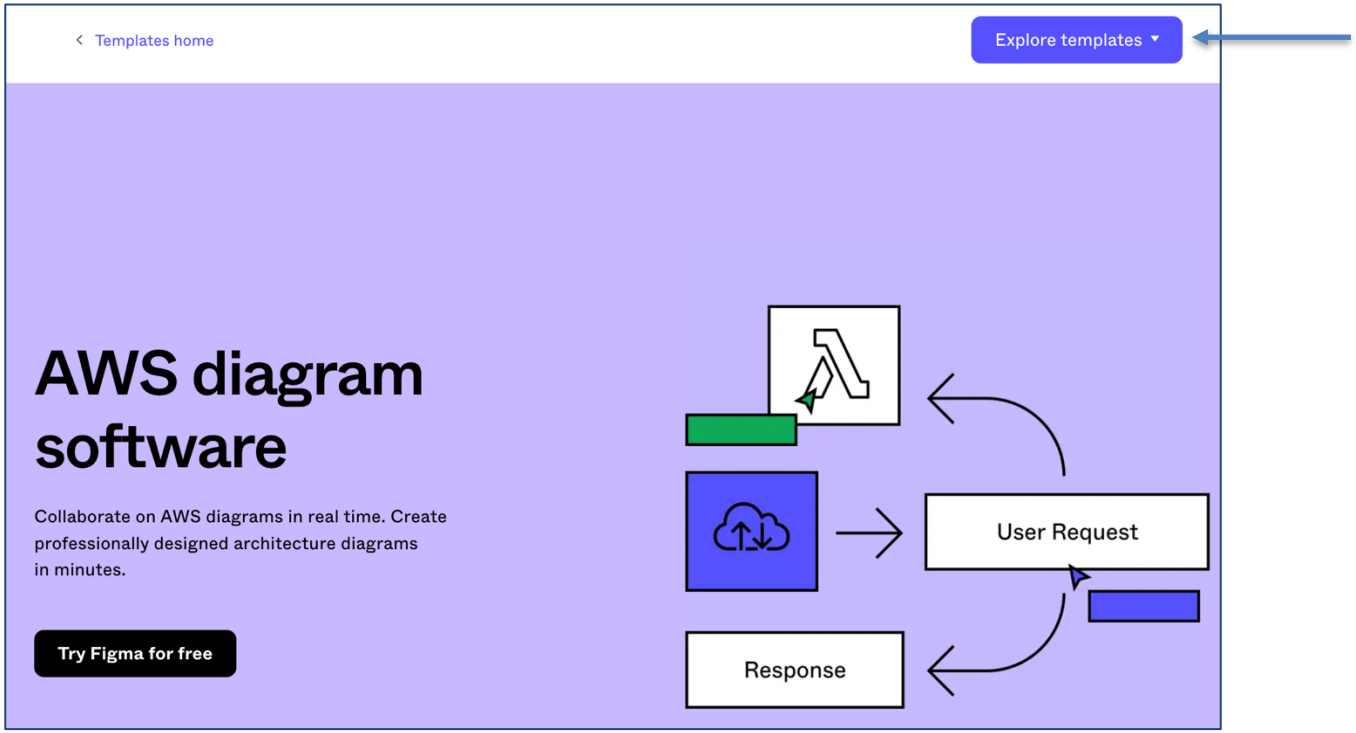
- **Figma**:
  - https://www.figma.com/templates/engineering/
  - Nesta disciplina, de Arquitetura de Aplicações Web, utilizaremos o **Figma**.



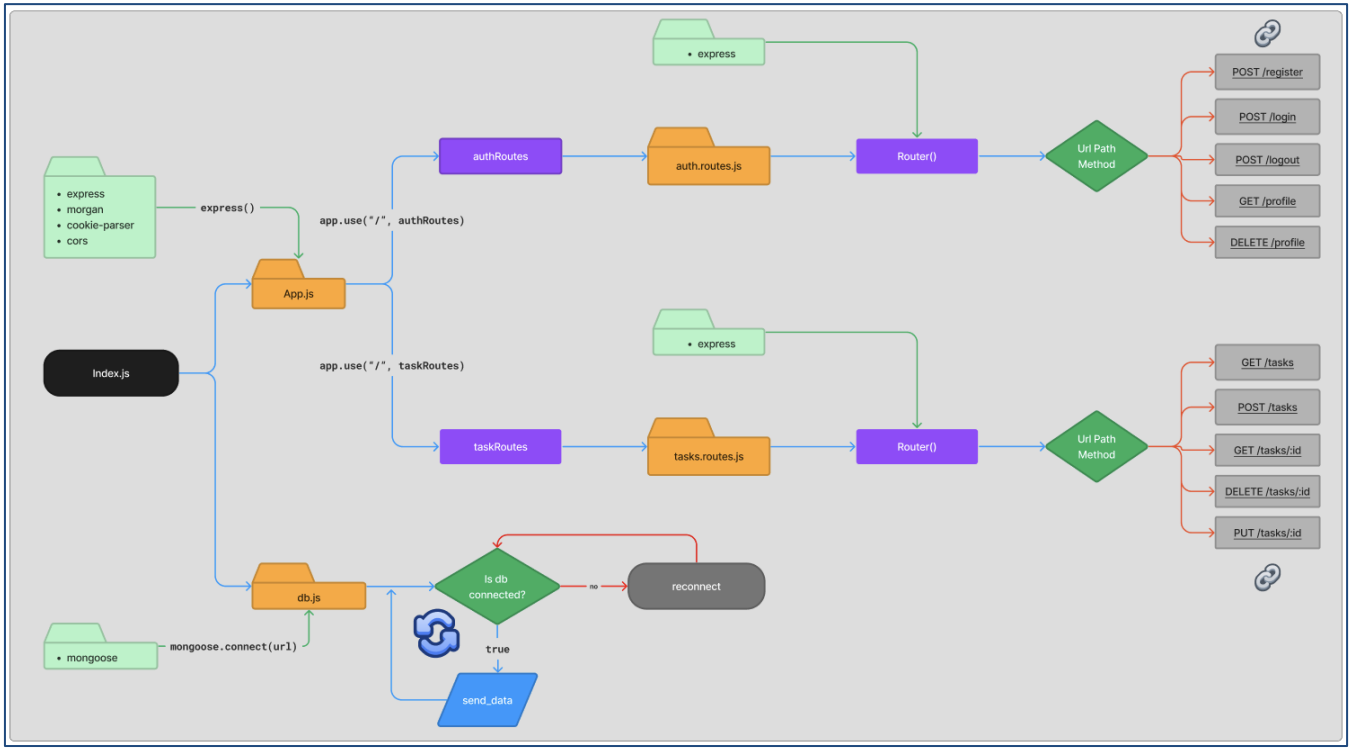
- **Figma**:
  - https://www.figma.com/templates/engineering/
  - Nesta disciplina, de Arquitetura de Aplicações Web, utilizaremos o **Figma**.



- **Figma**:
  - https://www.figma.com/templates/aws-diagram-software/
  - Sugestão: Explore os templates disponíveis.

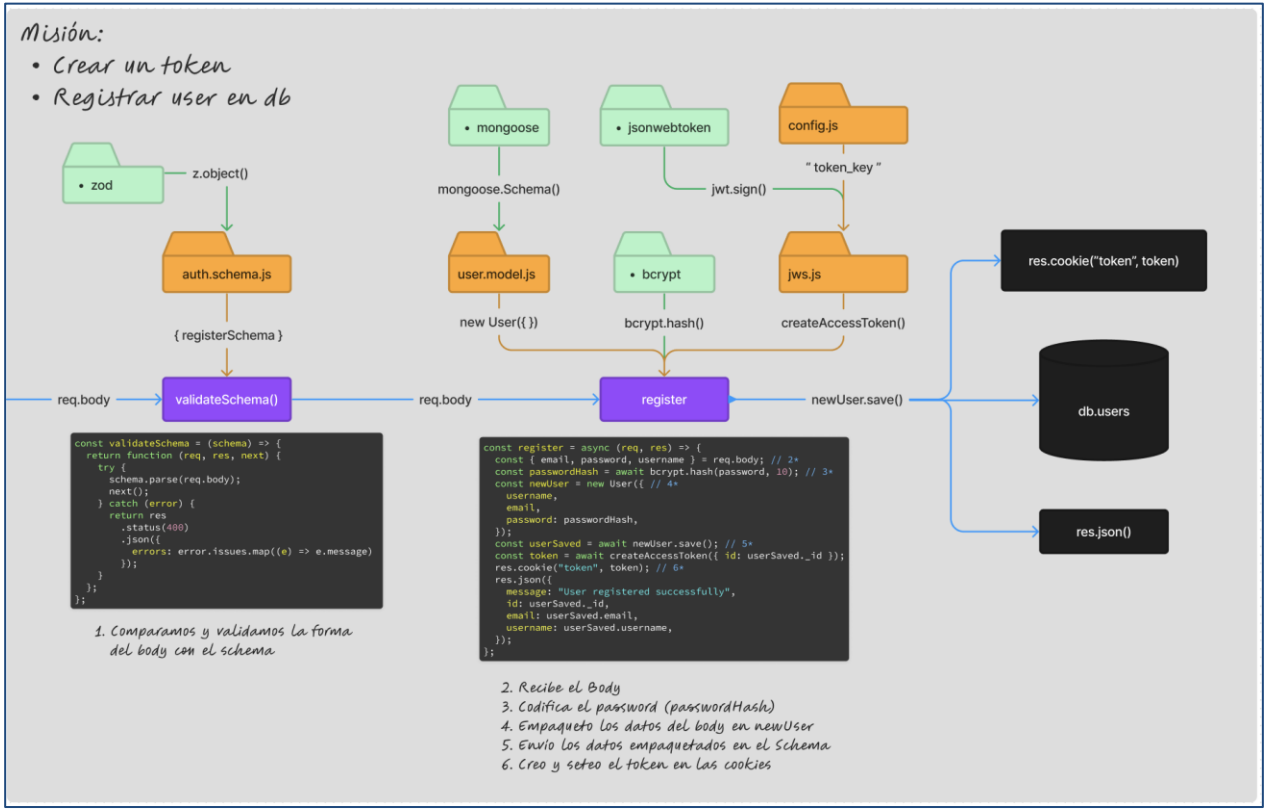


- **Figma**:
  - Comunidade: [REST API Diagram]
  - https://www.figma.com/community/file/1275291490973723744/rest-api-diagram-eng-esp



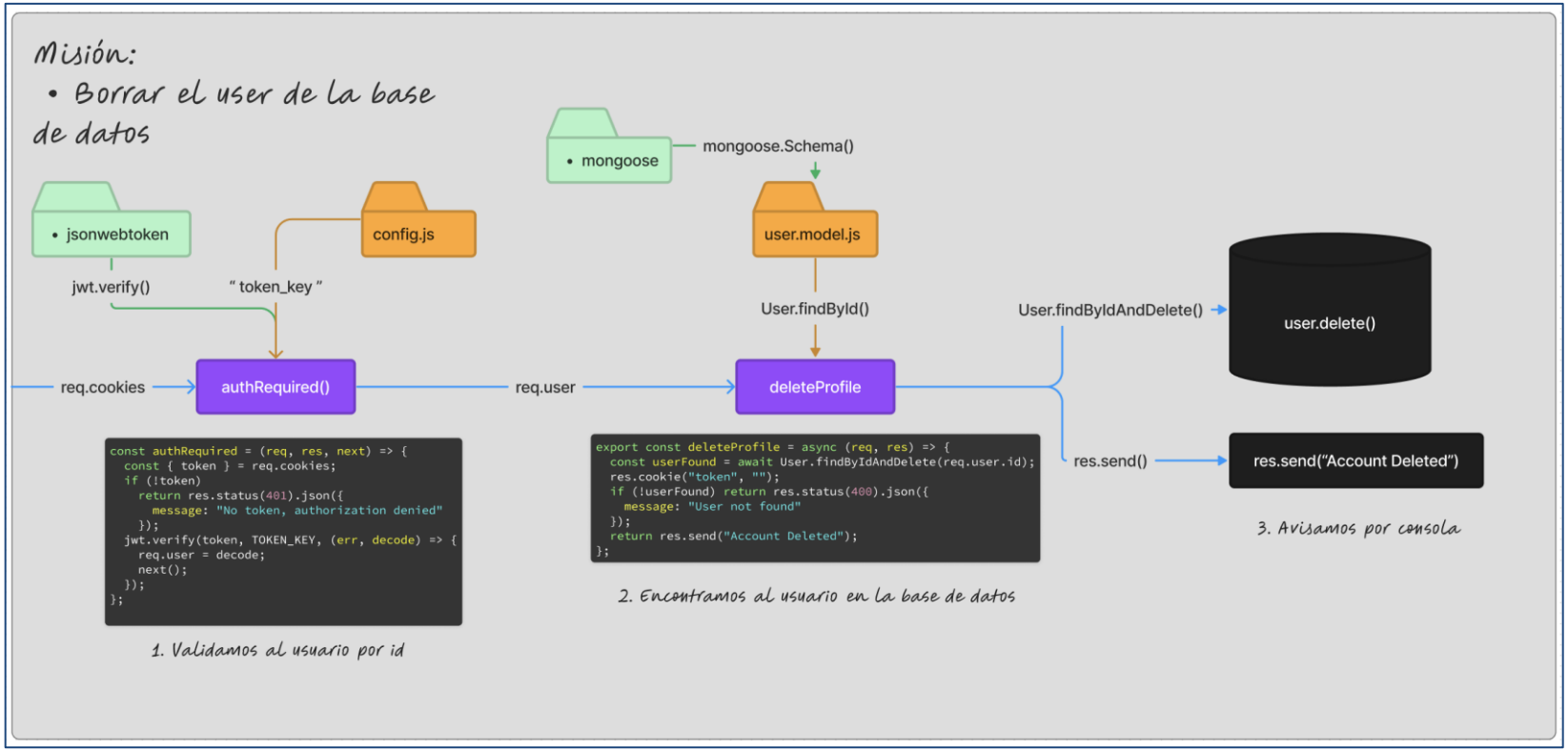
- Comunidade: [REST API Diagram]
- https://www.figma.com/community/file/1275291490973723744/rest-api-diagram-eng-esp

- **Figma**:

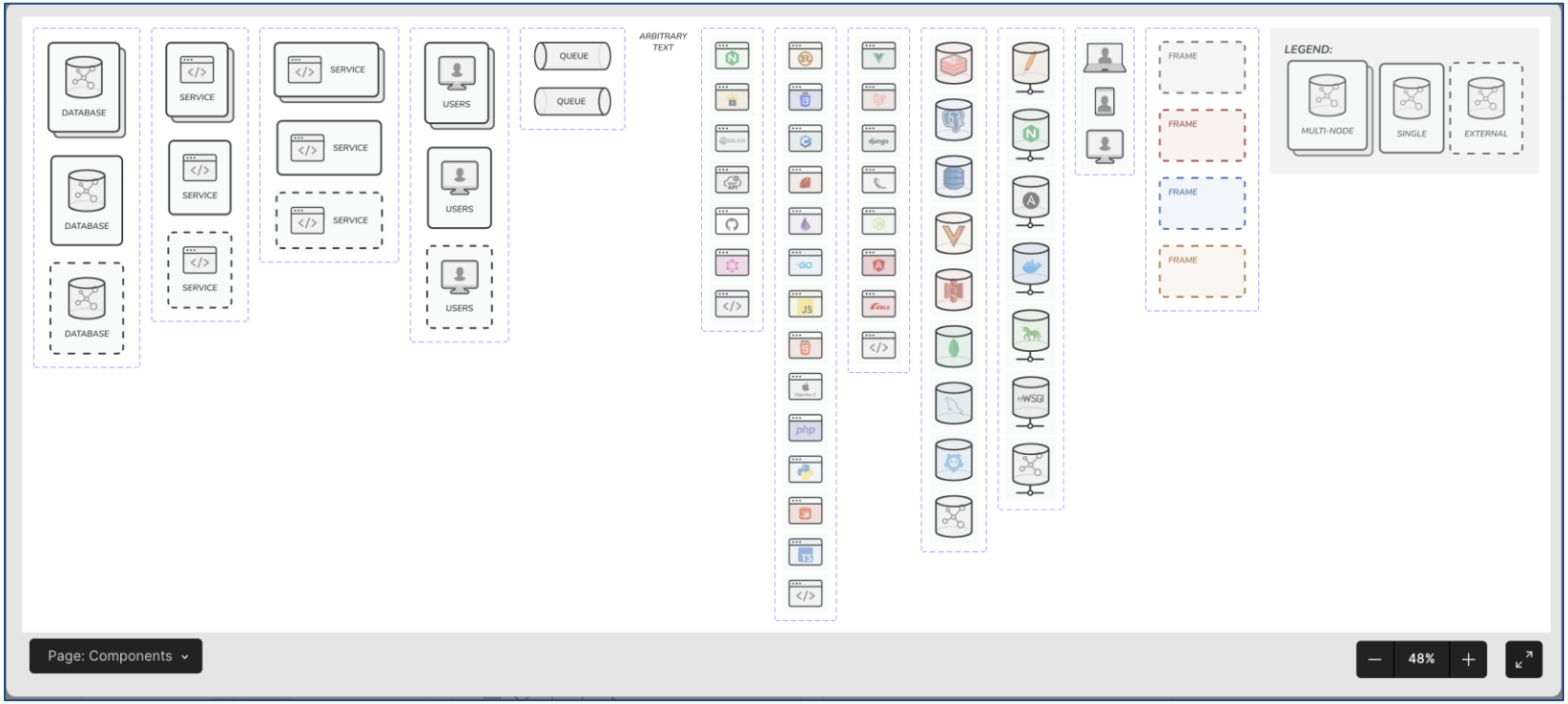


- Comunidade: [REST API Diagram]
- https://www.figma.com/community/file/1275291490973723744/rest-api-diagram-eng-esp

- **Figma**:

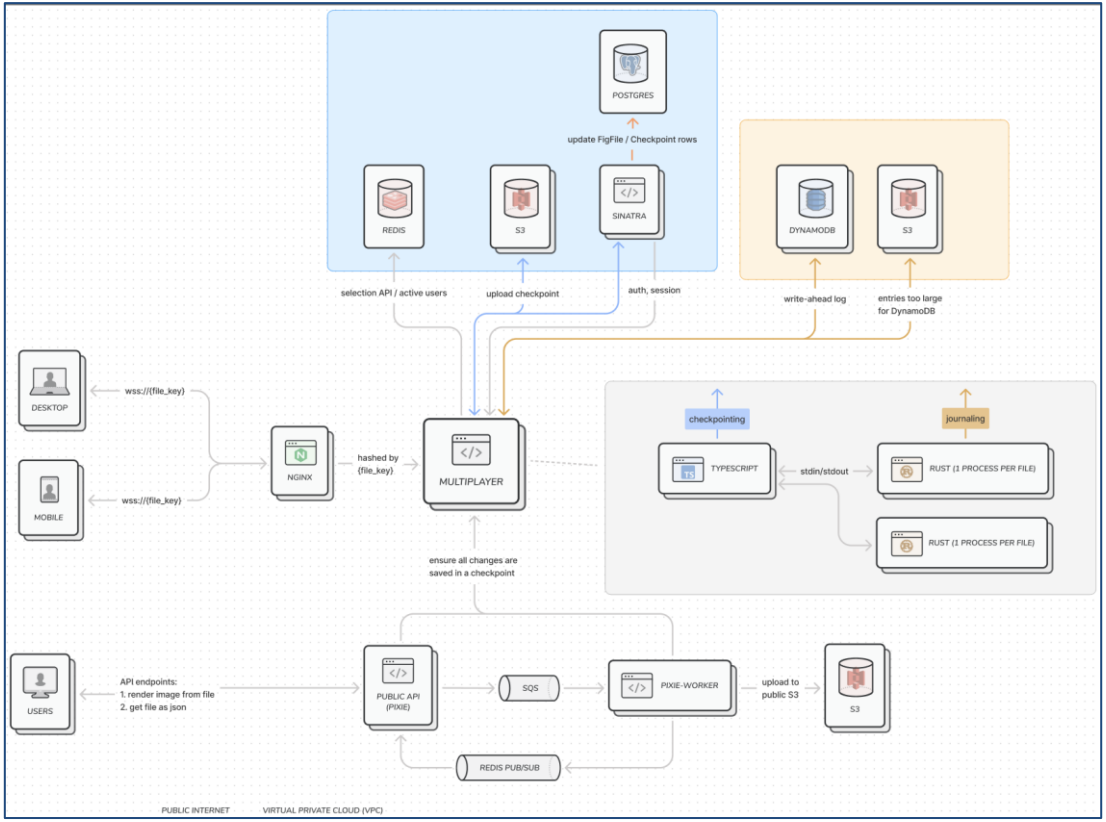


- **Figma**:
  - Comunidade: [Architecture Diagram Components]
  - https://www.figma.com/community/file/989635781221754599/architecture-diagram-components

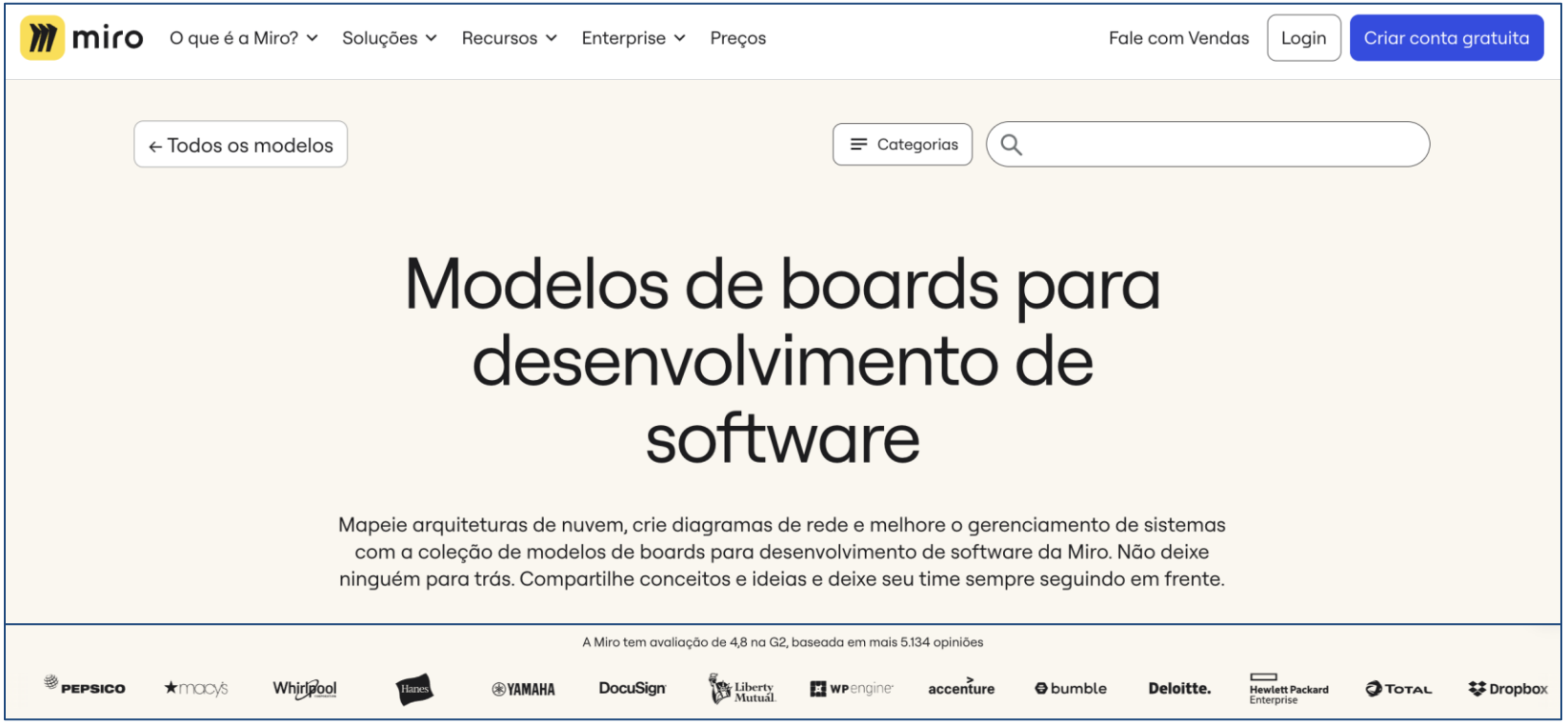


- Comunidade: [Exemplo feito com os componentes do diagrama de arquitetura]
- https://www.figma.com/community/file/989634471195357925/architecture-diagram-example-multiplayer

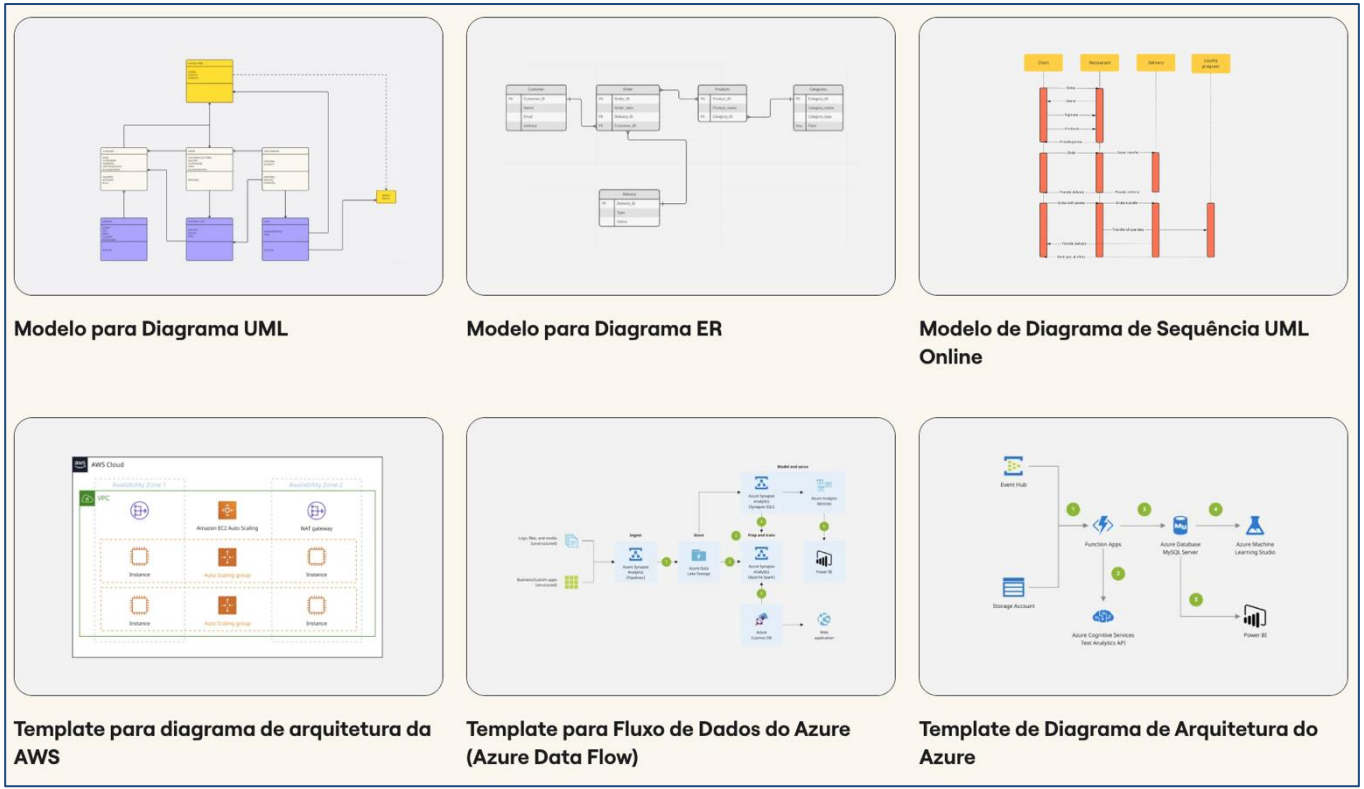
- **Figma**:



- **Miro**:
  - https://miro.com/pt/modelos/desenvolvedores/



- **Miro**:
  - https://miro.com/pt/modelos/desenvolvedores/



Modelo para Diagrama UML

Modelo para Diagrama ER

Modelo de Diagrama de Sequência UML Online

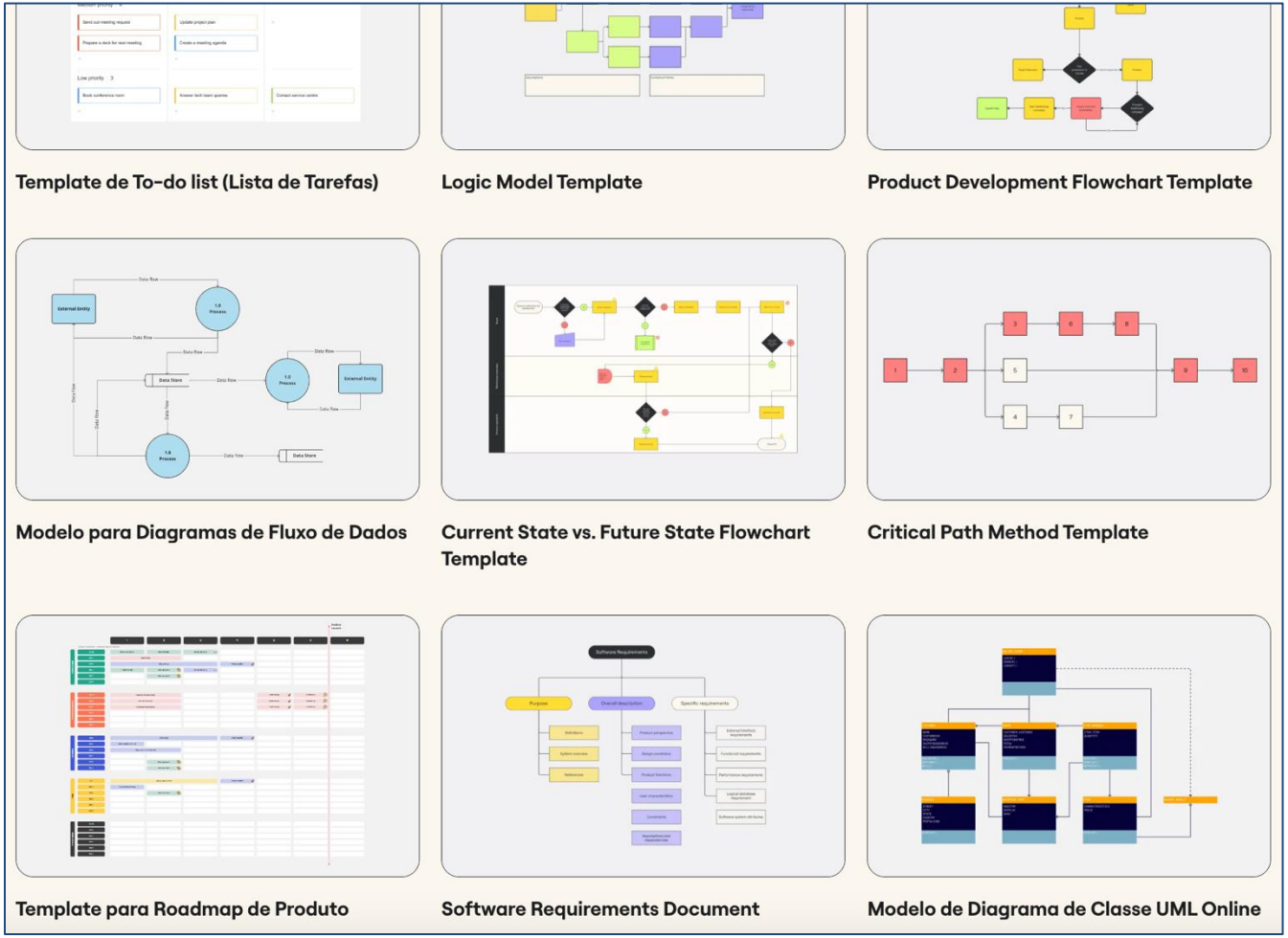
Template para diagrama de arquitetura da AWS

Template para Fluxo de Dados do Azure (Azure Data Flow)

Template de Diagrama de Arquitetura do Azure

- **Miro**:

Interface muito semelhante ao Figma

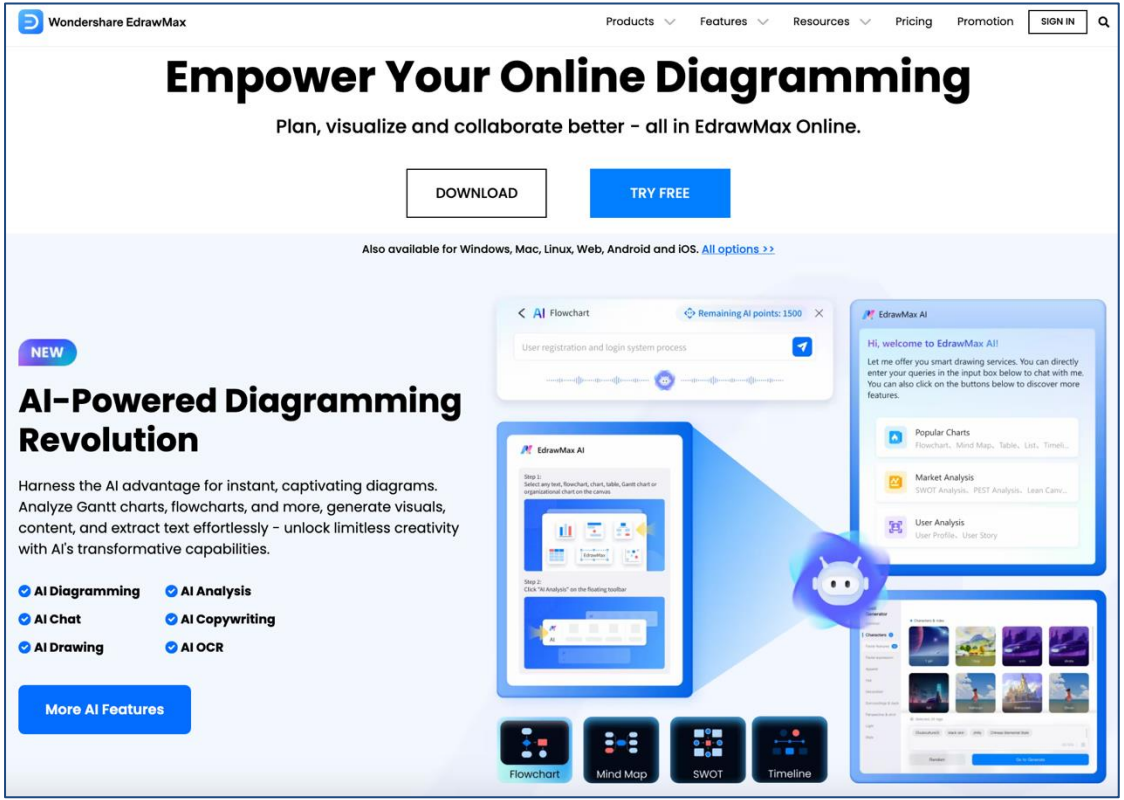


- **EdrawMax**:
  - **[Desktop]** https://www.edrawsoft.com/pt/edraw-max/
  - **[Online]** https://www.edrawmax.com/online/pt/store



- **EdrawMax**:
  - https://www.edrawmax.com/

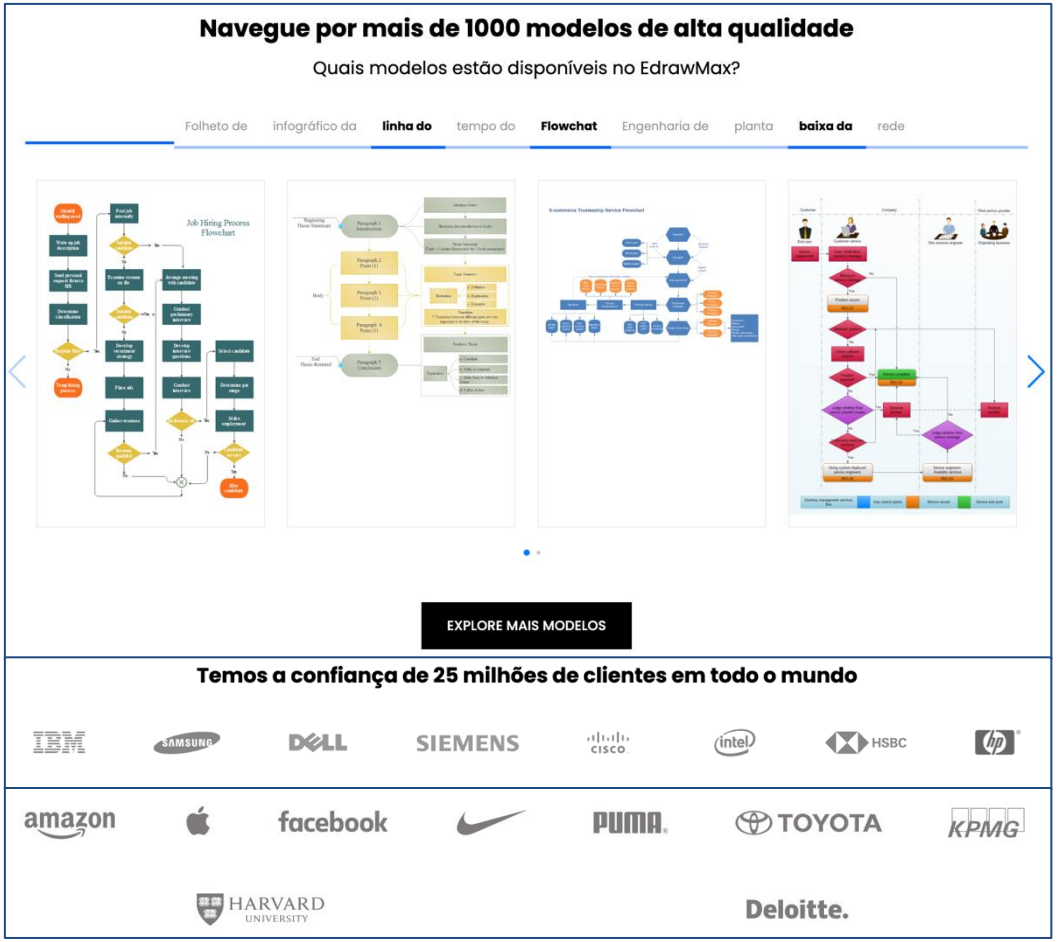
A maioria das ferramentas de documentação estão utilizando Inteligência Artificial



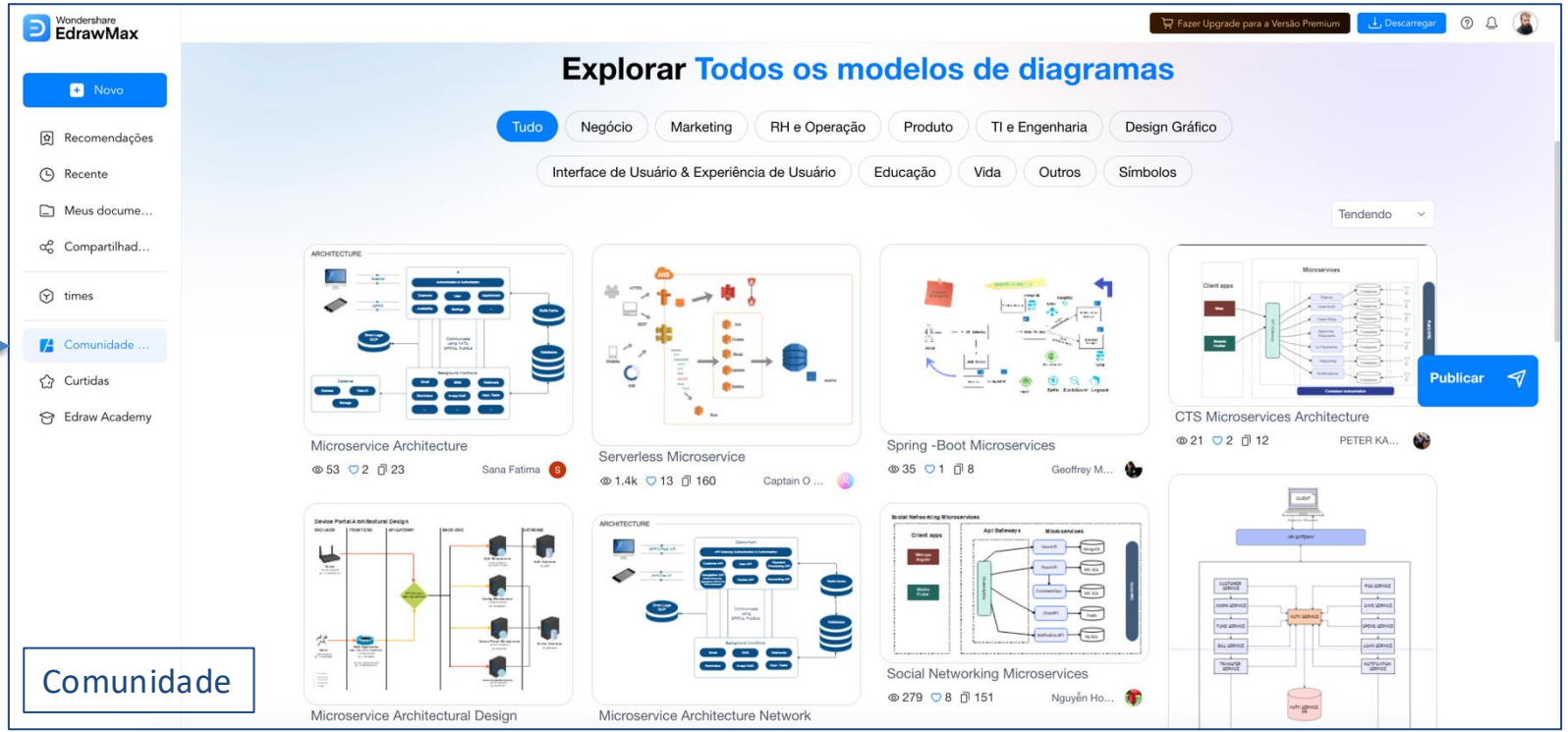
- **EdrawMax:**
  - https://www.edrawmax.com/pt/



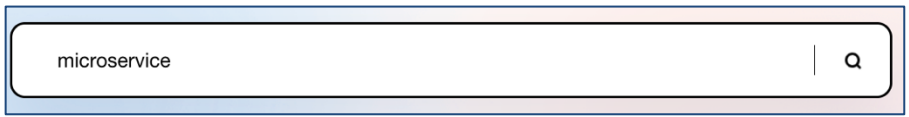
EdrawMax possui diferentes aplicações

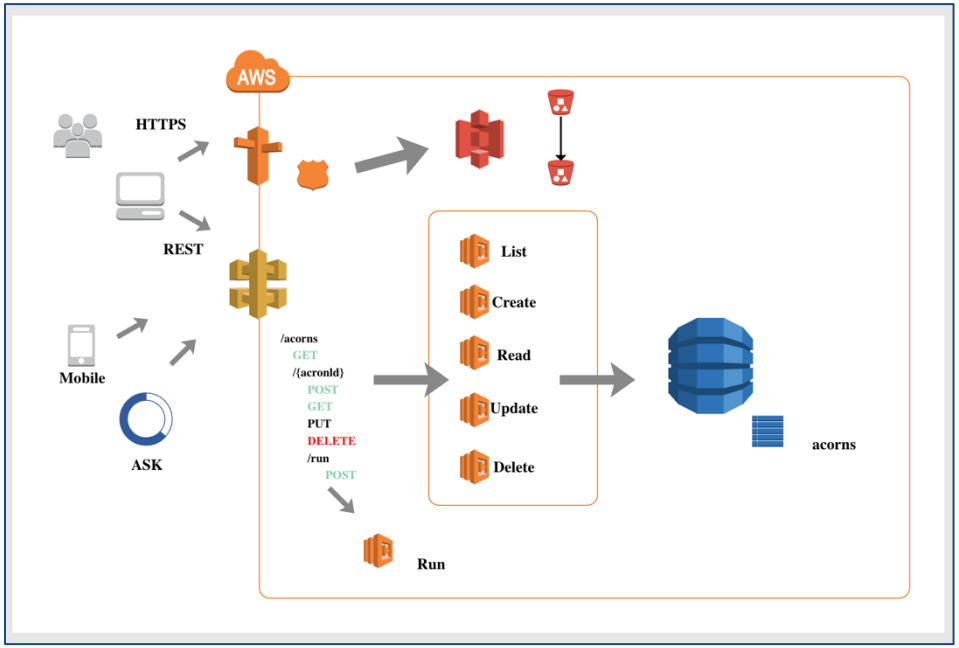


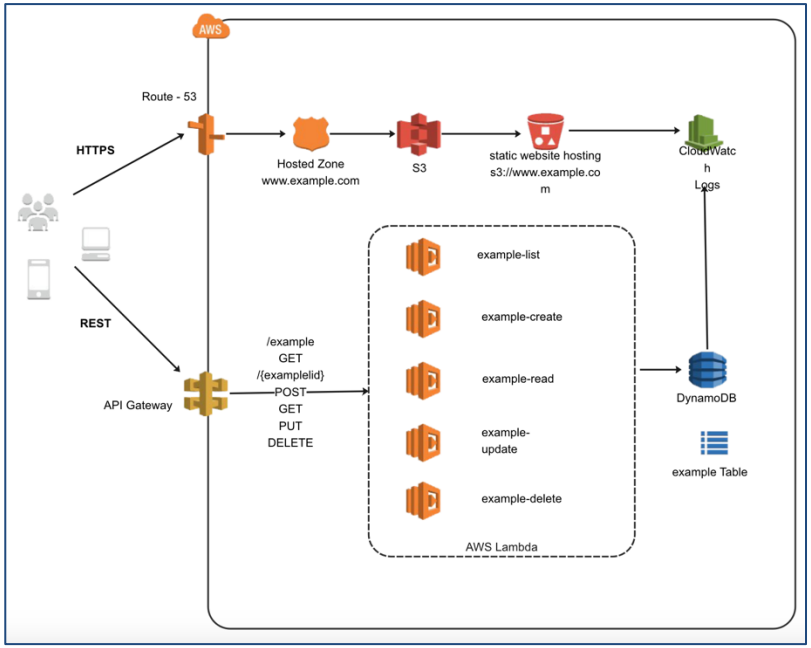
- **EdrawMax**:
  - Comunidade: https://www.edrawmax.com/online/pt/store
  - https://www.edrawsoft.com/pt/article/microservices-architecture-diagram.html



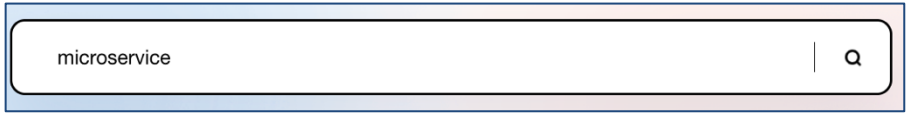
- **EdrawMax**:
  - Comunidade: https://www.edrawmax.com/online/pt/store

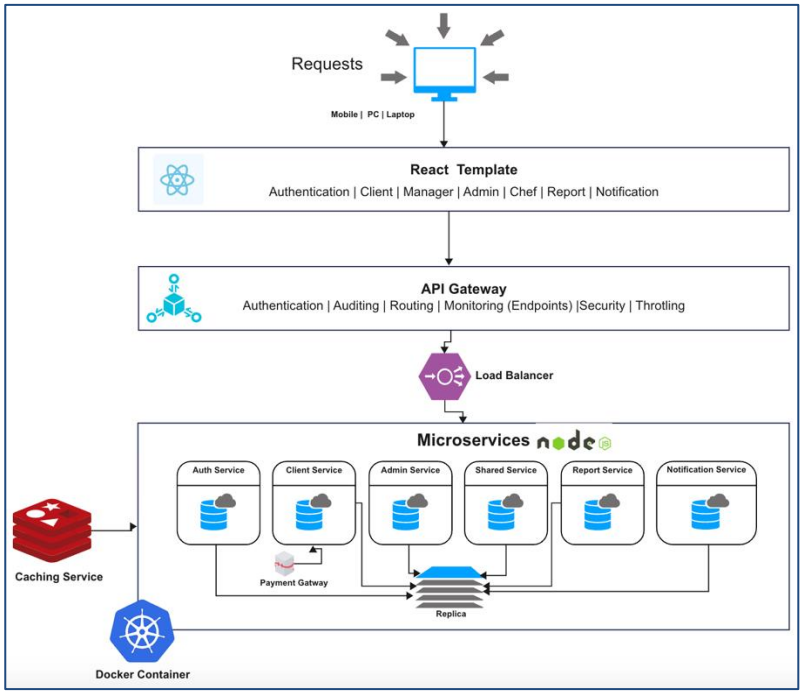


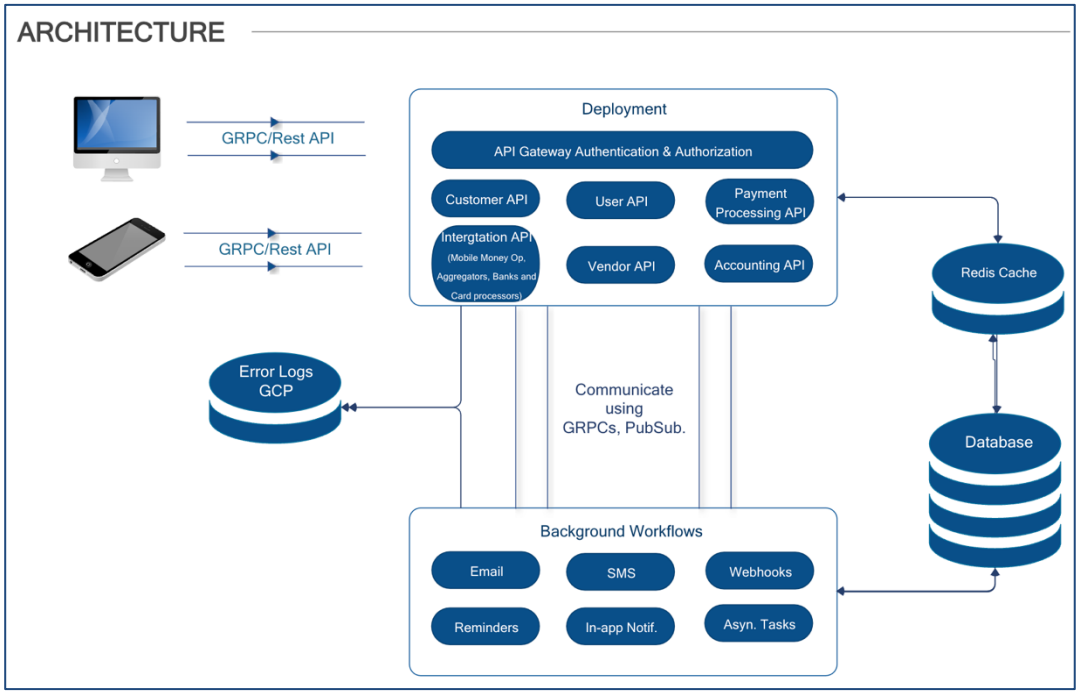




- **EdrawMax**:
  - Comunidade: https://www.edrawmax.com/online/pt/store







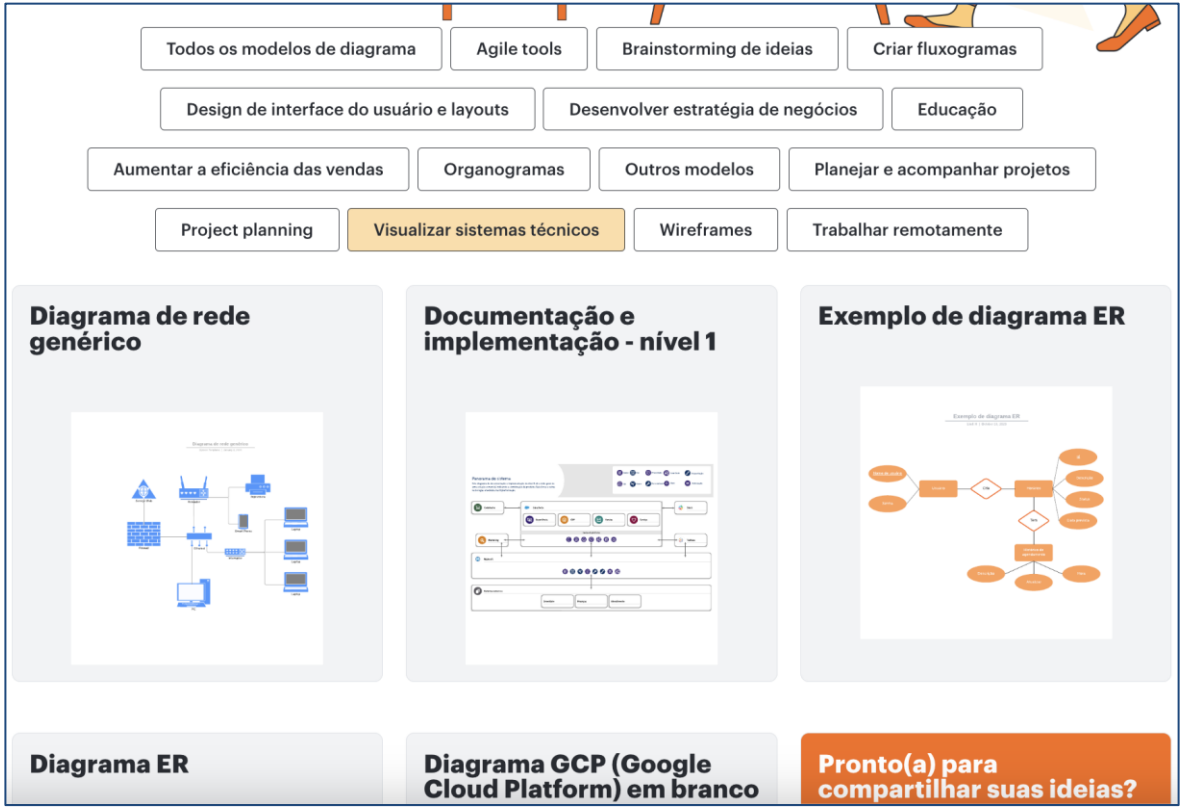
- **Lucidchart**:
  - https://www.lucidchart.com/pages/pt



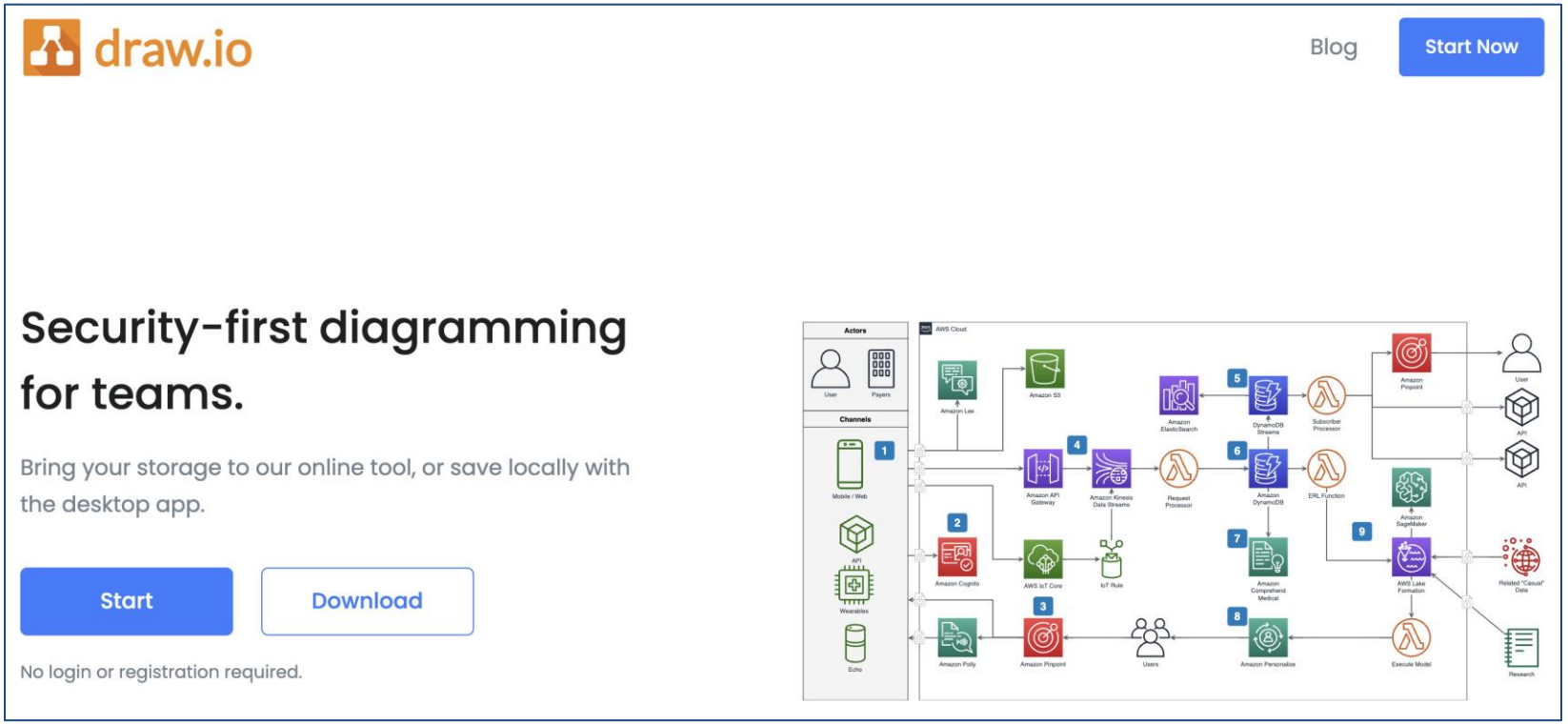
- **Lucidchart**:
  - https://www.lucidchart.com/pages/pt/modelos



- **Lucidchart**:
  - https://www.lucidchart.com/pages/pt/modelos/visualize-technical-systems

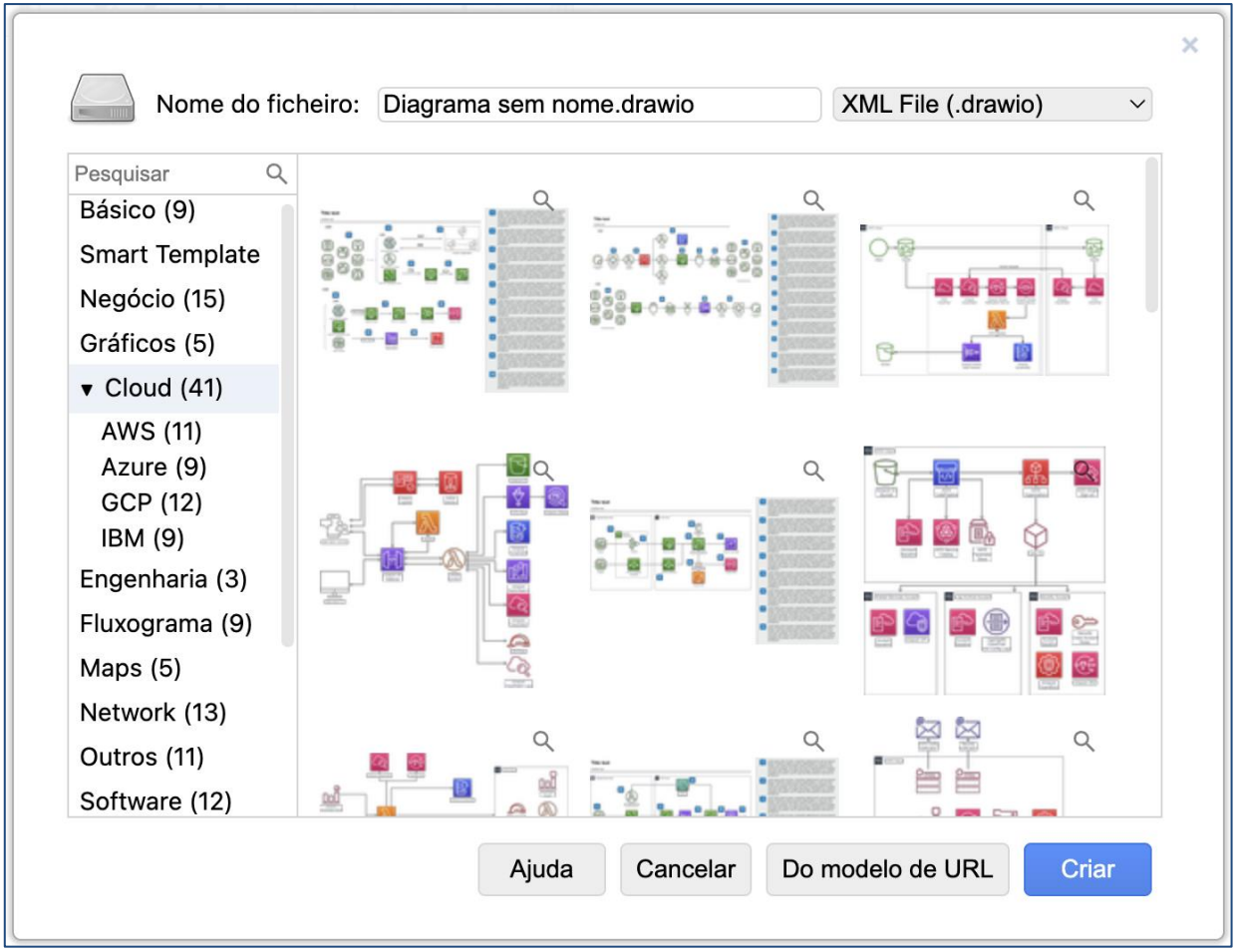


- **Draw.io**:
  - https://www.drawio.com/

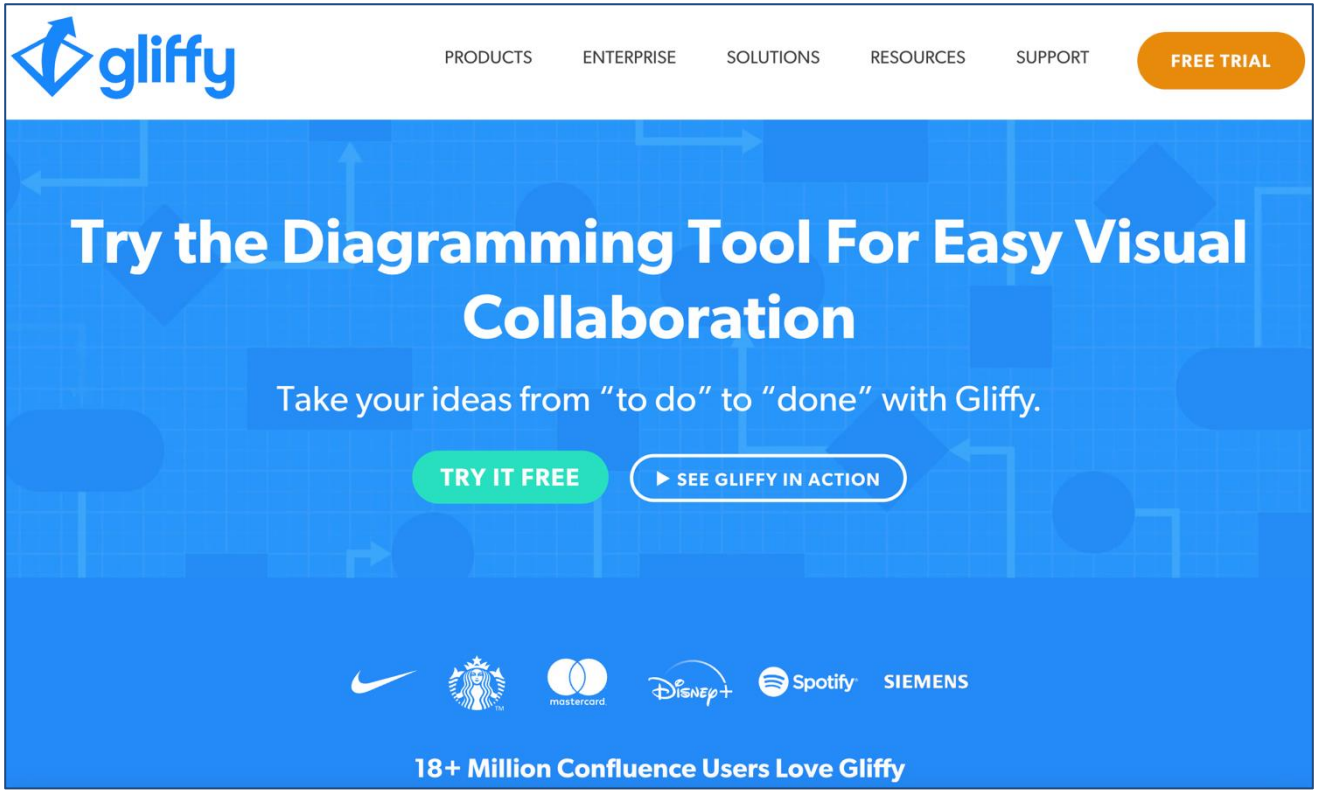


- **Draw.io**:
  - https://www.drawio.com/





- **Gliffy**:
  - https://www.gliffy.com/

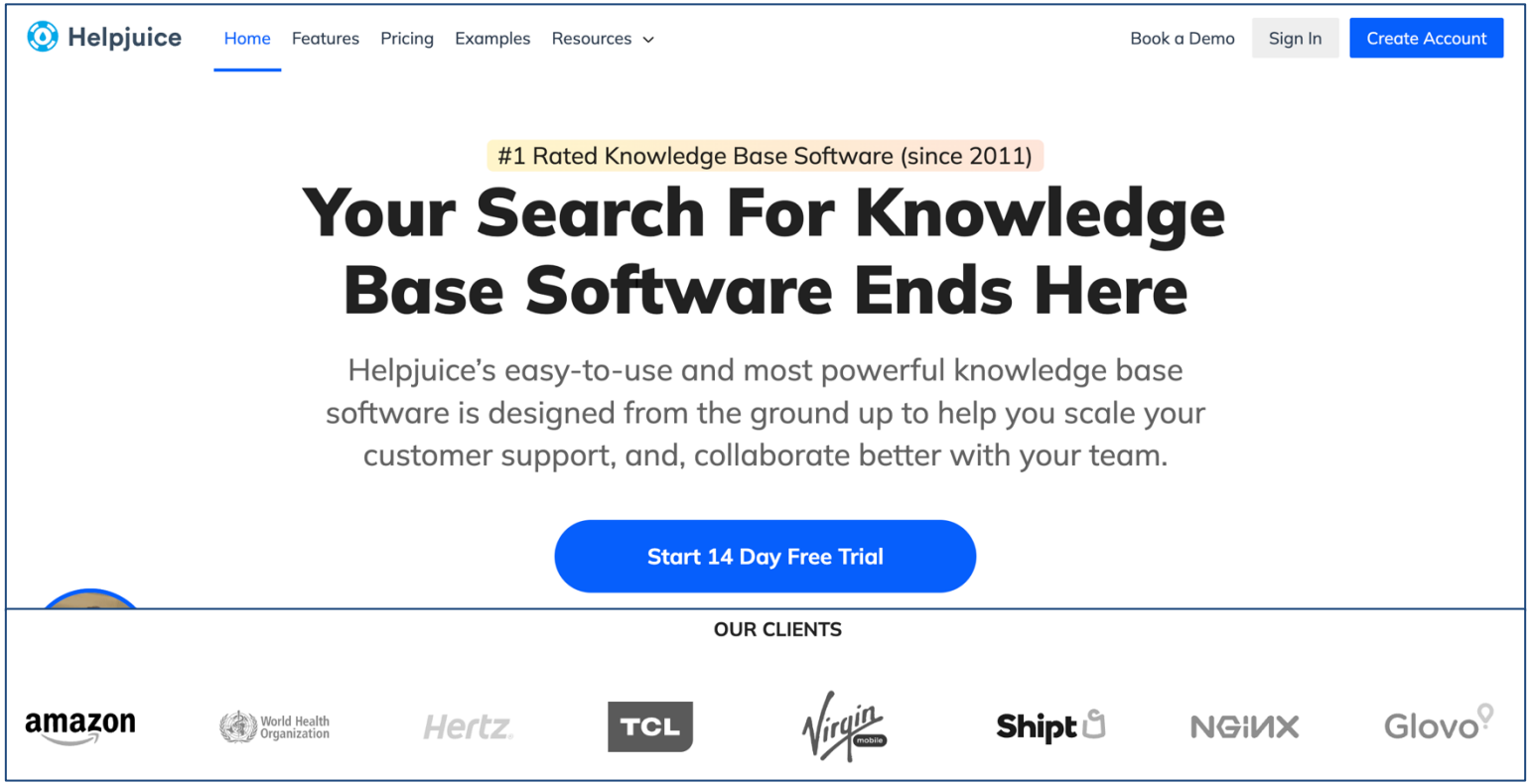


Documentação

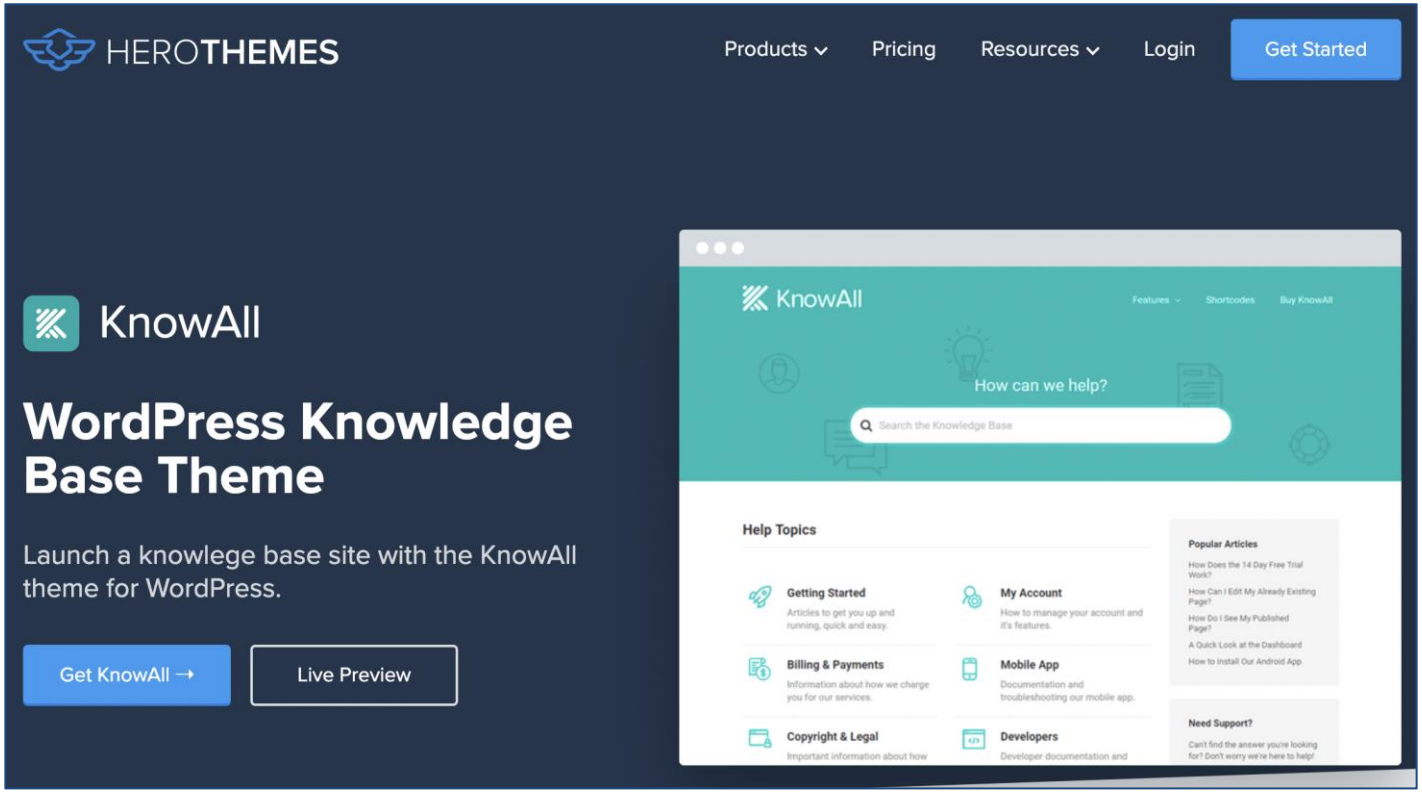
- **Ferramentas para documentação do software**:
  - Existem diversas ferramentas para auxiliar no processo de documentação de um software.

    Como por exemplo:
    - [HelpJuice] https://helpjuice.com/
    - [KnowAll] https://herothemes.com/themes/knowall-wordpress-knowledge-base/
    - [ClickHelp] https://clickhelp.com/
    - [HelpDocs] https://www.helpdocs.io/
    - [KnowledgeOwl] https://www.knowledgeowl.com/
    - [Confluence] https://www.atlassian.com/br/software/confluence

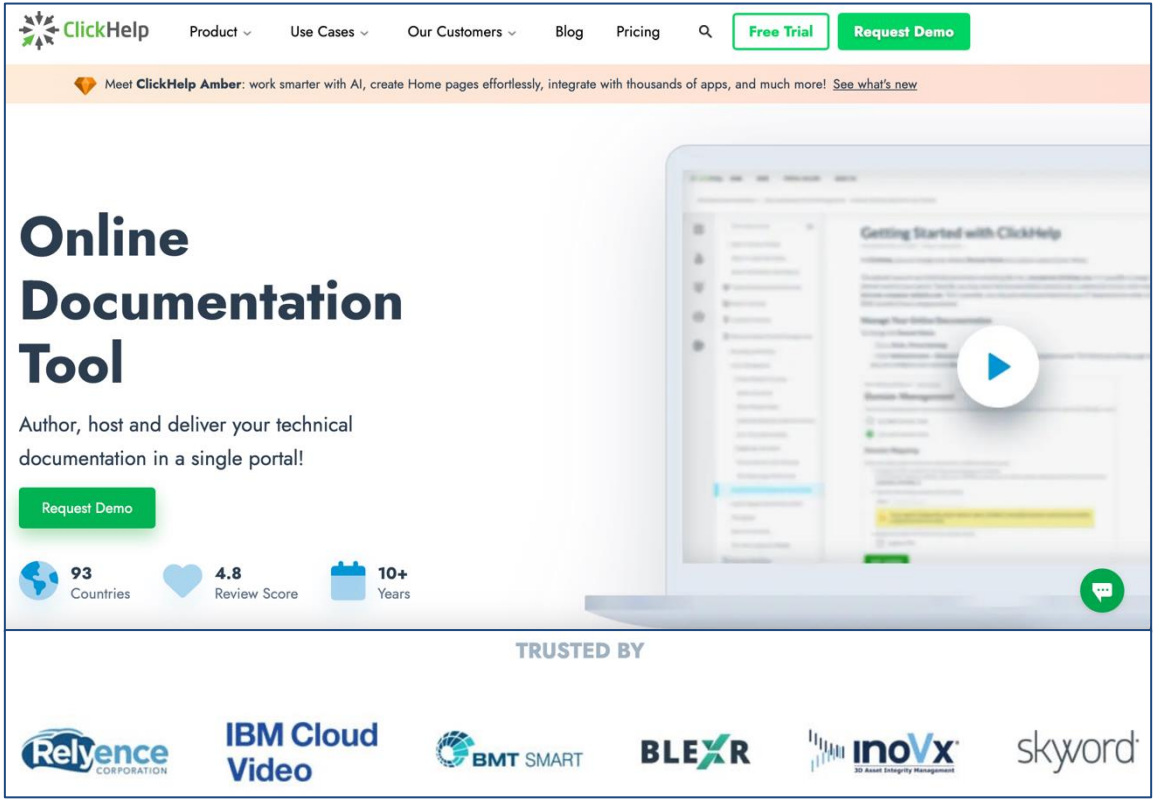
- **HelpJuice**:
  - https://helpjuice.com/



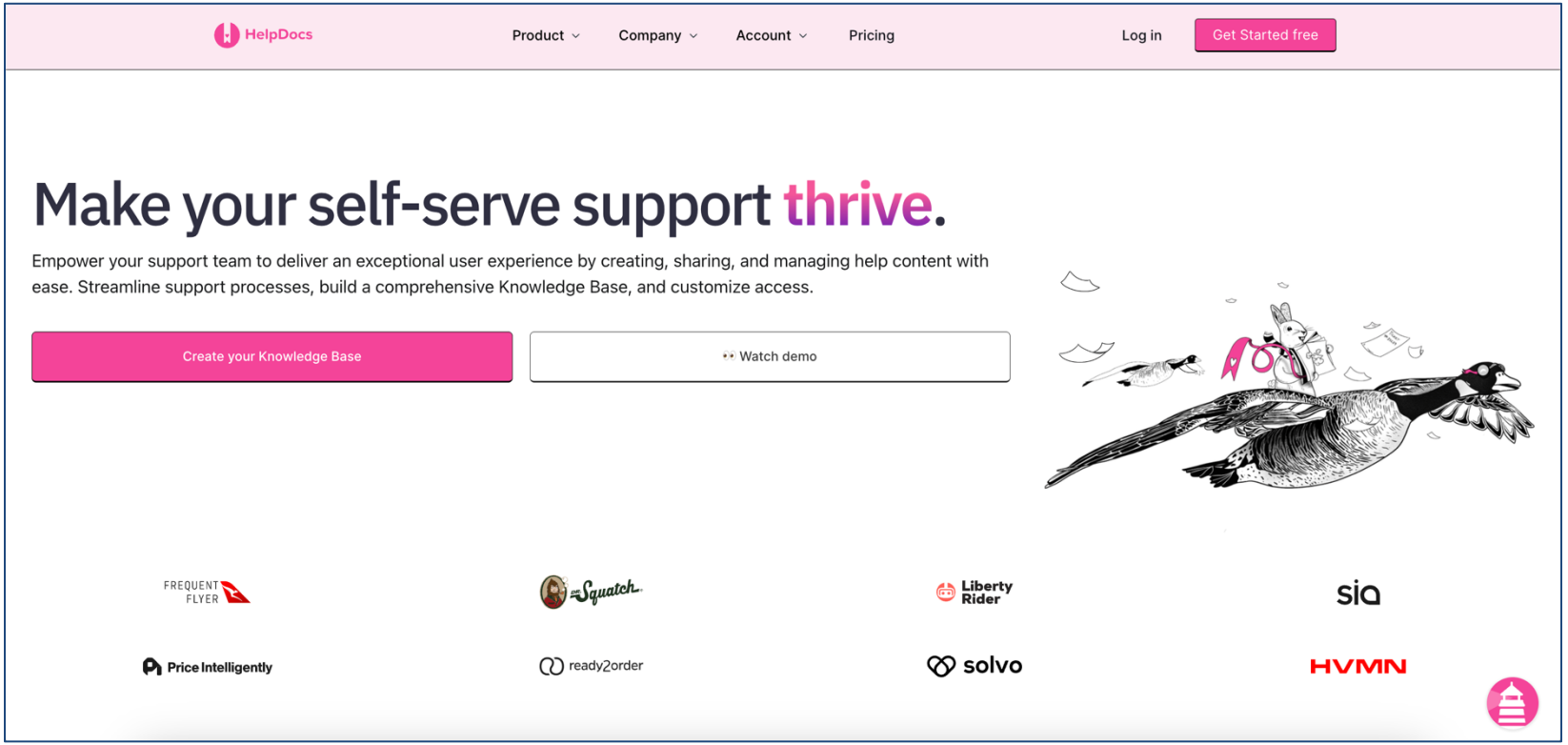
- **KnowAll**:
  - https://herothemes.com/themes/knowall-wordpress-knowledge-base/



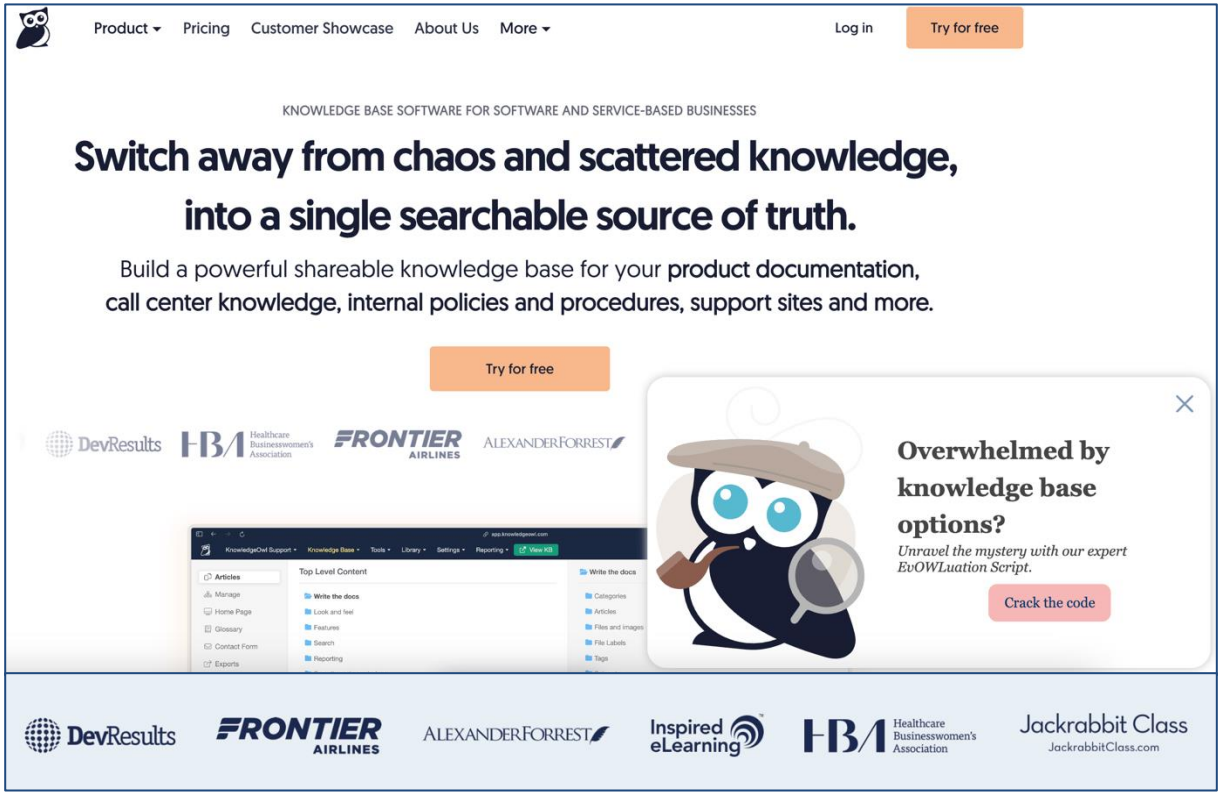
- **ClickHelp**:
  - https://clickhelp.com/



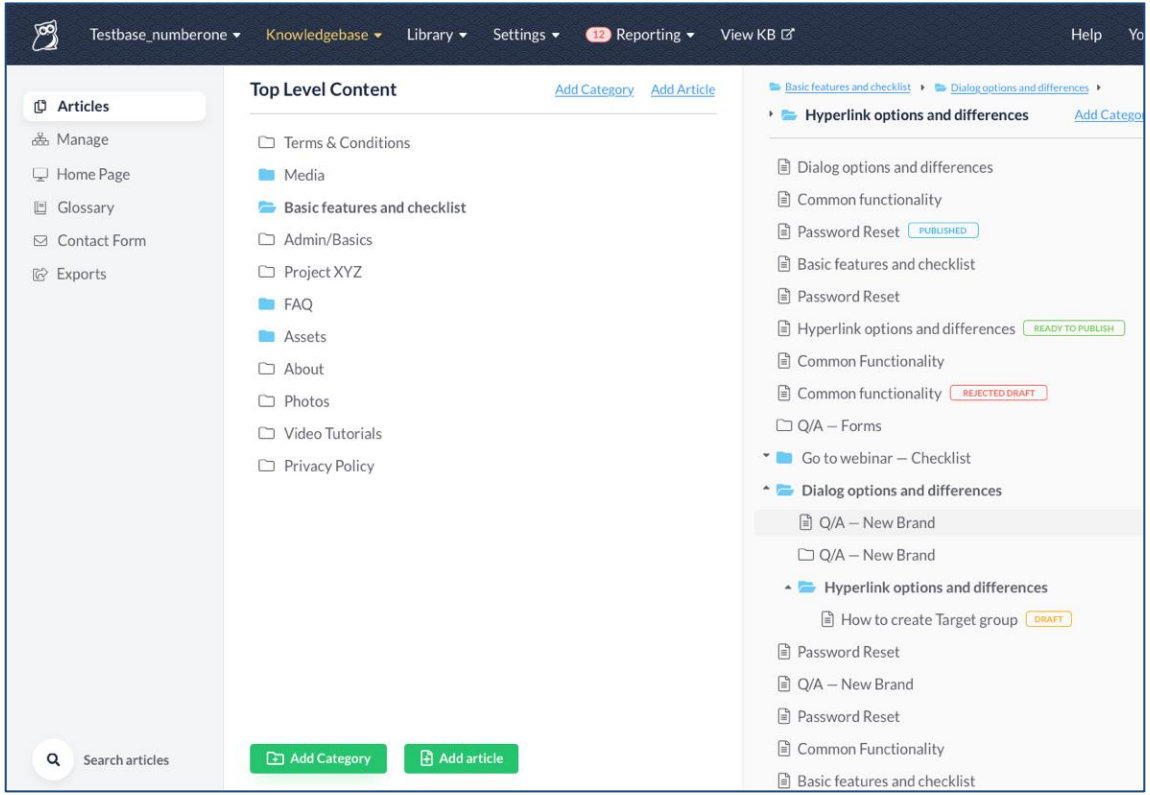
- **HelpDocs**:
  - https://www.helpdocs.io/



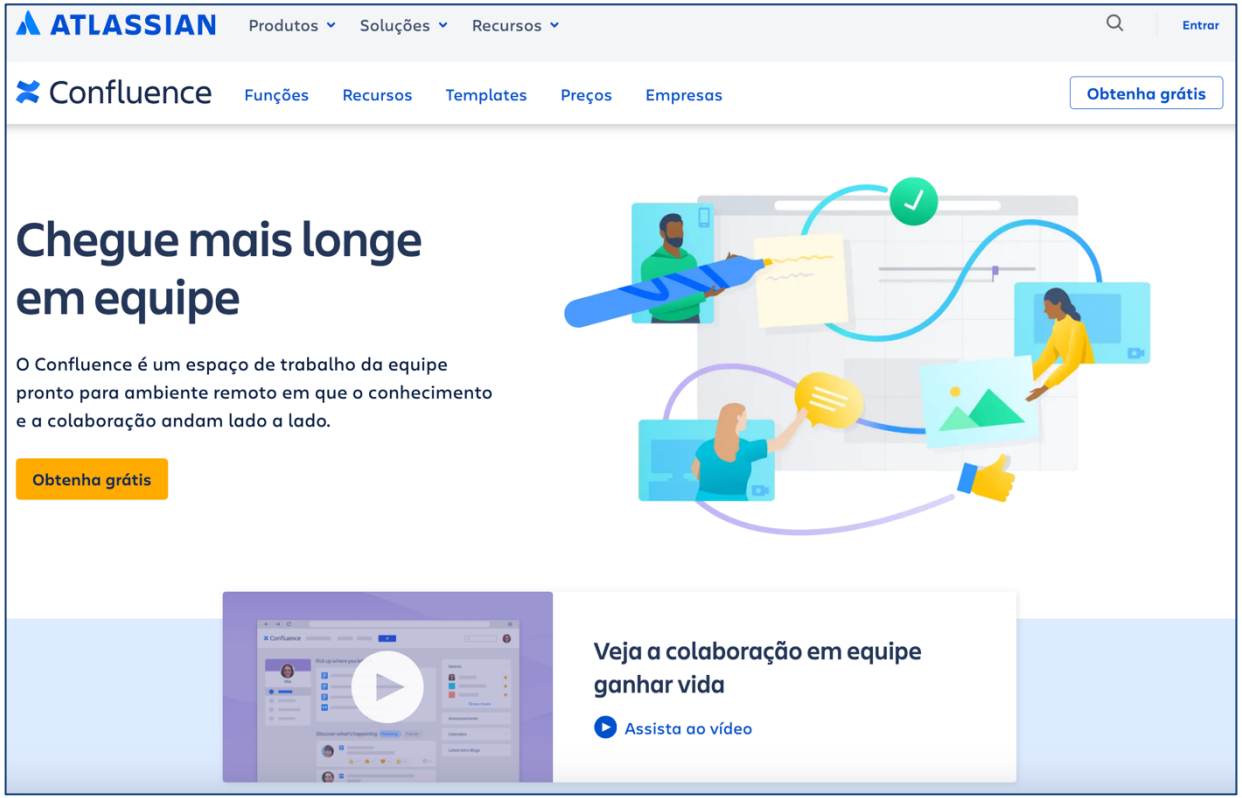
- **KnowledgeOwl**:
  - https://www.knowledgeowl.com/



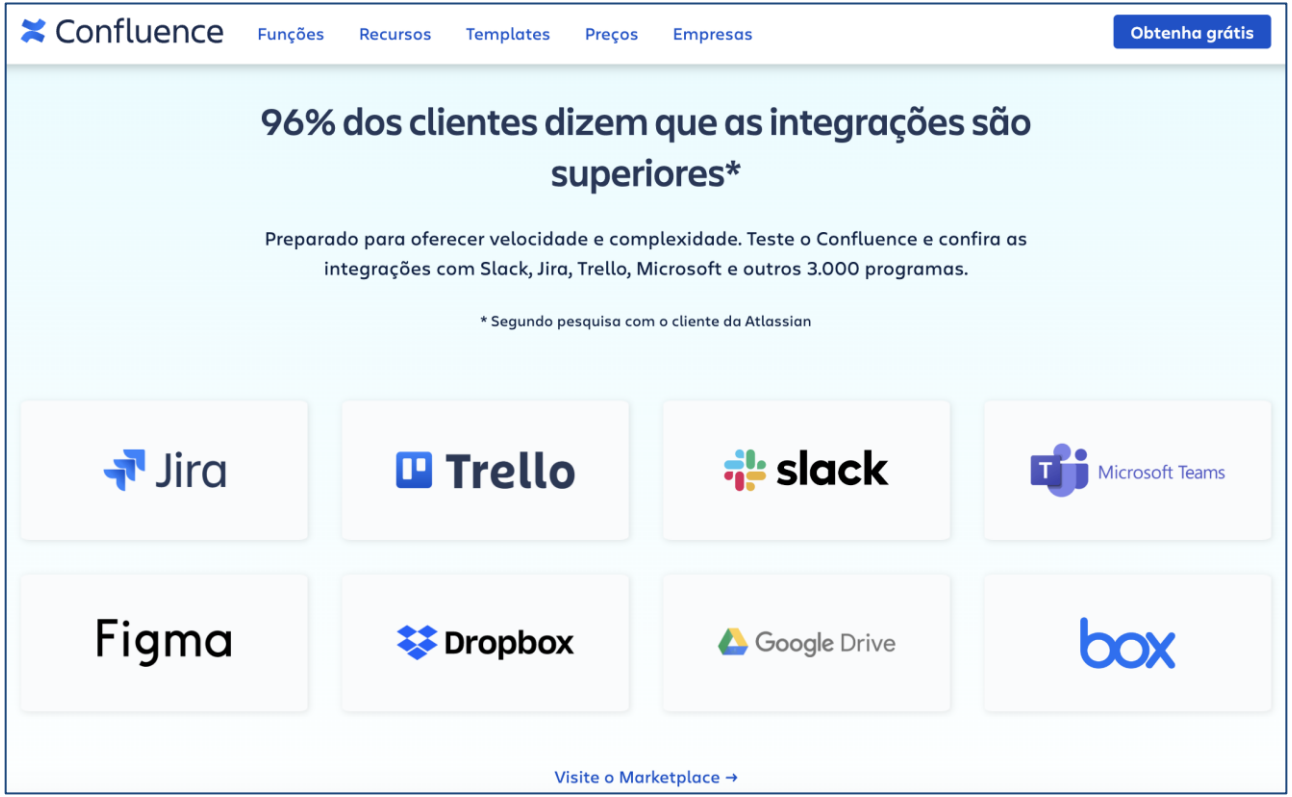
- **KnowledgeOwl**:
  - https://www.knowledgeowl.com/



- **Confluence**:
  - https://www.atlassian.com/br/software/confluence



- **Confluence**:
  - https://www.atlassian.com/br/software/confluence



- **Confluence**:
  - https://www.atlassian.com/br/software/confluence



  - No Brasil, o Confluence já é bem difundido em fábricas de software/empresas de tecnologia.

- **No caso desta disciplina**:
  - Para **diagramação** da arquitetura de microsserviços:
    - [Figma] https://www.figma.com/
  - Para **documentação** das APIs RESTful:
    - [SpringDoc] https://springdoc.org/
    - [SpringDoc] https://github.com/springdoc/springdoc-openapi
    - [Swagger UI] https://swagger.io/tools/swagger-ui/
    - [Swagger UI] https://github.com/swagger-api/swagger-ui
    - [Postman] https://learning.postman.com/docs/publishing-your-api/documenting-your-api/
  - Para **hospedagem** da documentação dos microsserviços:
    - [GitHub Pages] https://pages.github.com/
    - [GitHub Pages] https://docs.github.com/pt/pages

Documentação

- **Resumo** - A documentação de microsserviços prontos para produção inclui:
  - Uma descrição do microsserviço e de seu lugar no ecossistema e no negócio;
  - Um diagrama de arquitetura detalhando a arquitetura do serviço, seus clientes e dependências em um alto nível de abstração;
  - Informações de contato e plantão sobre a equipe de desenvolvimento do microsserviço;
  - Links para o repositório, os dashboards, a RFC (*Request for Comments*) para o serviço, revisões de arquitetura e outras informações relevantes ou úteis;
  - Uma seção FAQ (“perguntas mais frequentes”);



Documentação

- **Resumo** - A documentação de microsserviços prontos para produção inclui:
  - Um guia de bordo e desenvolvimento contendo detalhes sobre o processo de desenvolvimento, o pipeline de deployment e quaisquer outras informações que serão úteis para os devs que contribuem com código para o serviço;
  - Informações detalhadas sobre os fluxos de solicitações, o SLA, o status de disponibilidade de produção, os endpoints de API, importantes clientes e dependências do microsserviço;
  - Um roteiro de plantão contendo procedimentos gerais de resposta a incidentes e interrupções, instruções passo a passo sobre como selecionar, mitigar e resolver cada alerta e uma seção geral de solução de problemas e depuração.



Documentação

- **Avalie seu microsserviço**:
  - Agora que você tem mais entendimento a respeito de documentação, use a lista de perguntas a seguir para avaliar a disponibilidade de produção de seu(s) microserviço(s) e do ecossistema de microsserviços.



Documentação

- **Avaliação** - Documentação do Microsserviço:
  - 1) A documentação de todos os microsserviços está armazenada em um local centralizado, compartilhado e de fácil acesso?
  - 2) É fácil pesquisar a documentação?
  - 3) As mudanças significativas no microsserviço são acompanhadas por atualizações da documentação do microsserviço?
  - 4) A documentação do microsserviço contém uma descrição do microsserviço?
  - 5) A documentação do microsserviço contém um diagrama da arquitetura?



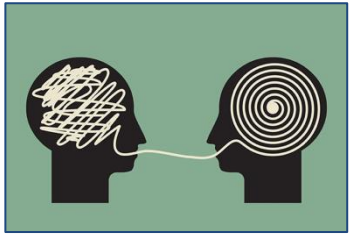
Documentação

- **Avaliação** - Documentação do Microsserviço:
  - 6) A documentação do microsserviço contém informações de contato e plantão?
  - 7) A documentação do microsserviço contém links para informações importantes?
  - 8) A documentação do microsserviço contém um guia de bordo e desenvolvimento?
  - 9) A documentação do microsserviço contém informações sobre o fluxo de solicitações, os endpoints e as dependências do microsserviço?
  - 10) A documentação do microsserviço contém um roteiro de plantão?
  - 11) A documentação do microsserviço contém uma seção FAQ?



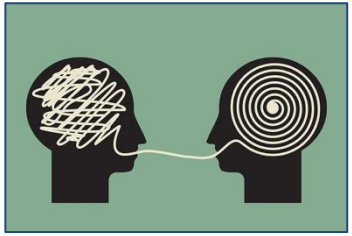
Documentação

- **Avaliação** - Compreensão do Microsserviço:
  - 1) Todo desenvolvedor da equipe é capaz de responder a perguntas sobre a disponibilidade de produção do microsserviço?
  - 2) Existe um conjunto de princípios e padrões que todos os microsserviços devem seguir?
  - 3) Existe um processo de RFC (*Request for Comments*) para cada novo microsserviço?
  - 4) Os microsserviços existentes são revisados e auditados com frequência?
  - 5) São realizadas revisões de arquitetura para cada equipe de microsserviço?



Documentação

- **Avaliação** - Compreensão do Microsserviço:
  - 6) Existe um processo de auditoria de disponibilidade de produção?
  - 7) São usados planos estratégicos de disponibilidade de produção para levar o microsserviço a um estado de pronto para produção?
  - 8) Padrões de disponibilidade de produção orientam os OKRs (*Objective Key Results*) da organização?
  - 9) O processo de disponibilidade de produção é automatizado?



Sumário

- Documentação
  - Diagrama da arquitetura
  - Fluxos de solicitação, endpoints e dependências
  - **Documentando as rotas**

Vamos à prática!

- **Documentando as rotas**:
  - Como vimos, iremos utilizar o SpringDoc com o Swagger UI para documentar a nossa API:
    - [SpringDoc] https://springdoc.org/
    - [SpringDoc] https://github.com/springdoc/springdoc-openapi
    - [Swagger UI] https://swagger.io/tools/swagger-ui/
    - [Swagger UI] https://github.com/swagger-api/swagger-ui





Vamos à prática!

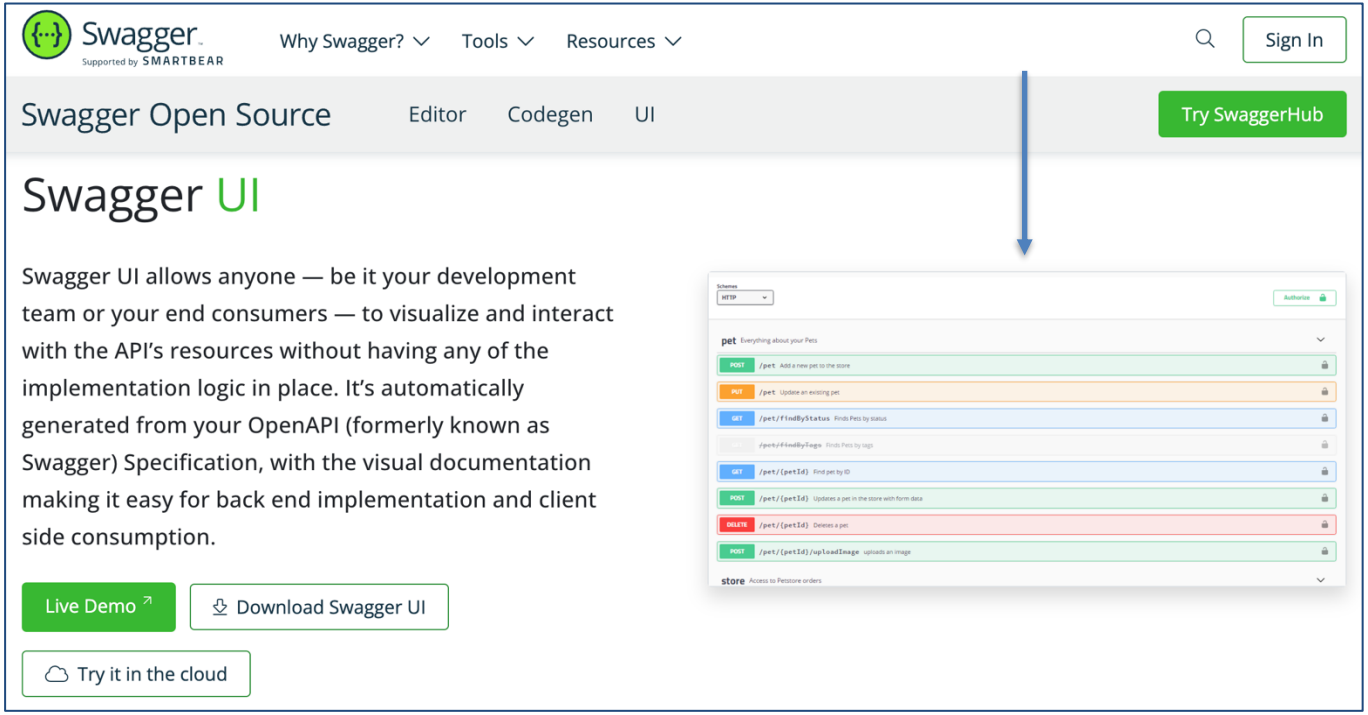
- **Documentando as rotas**:
  - **Não** iremos utilizar o SpringFox:
    - https://springfox.github.io/springfox/
    - https://springfox.github.io/springfox/docs/current/
  - É muito importante pesquisar sobre as bibliotecas antes de inseri-las em seu projeto. Elas podem estar defasadas, incompatíveis com versões mais atuais do Spring Boot ou simplesmente foram descontinuadas.

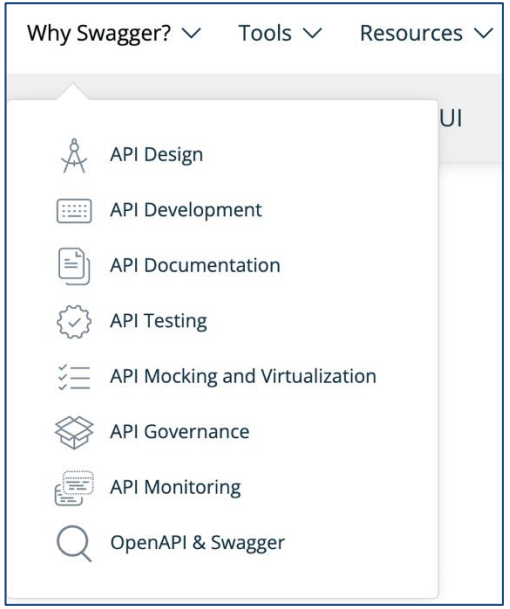
- **Documentando as rotas:**
  - https://springdoc.org/



- Este é um projeto baseado na comunidade. O Spring Doc não é mantido pelo Spring Framework (Pivotal).

- **Documentando as rotas:**
  - https://swagger.io/tools/swagger-ui/





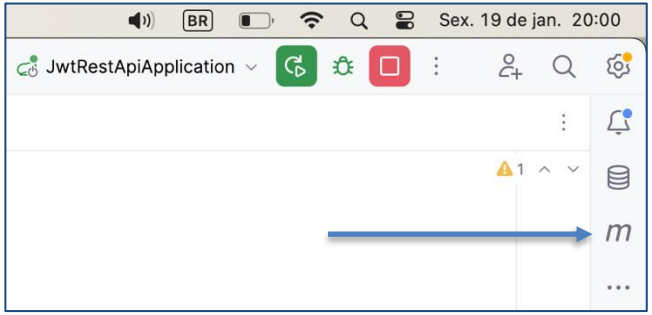
Vamos à prática!

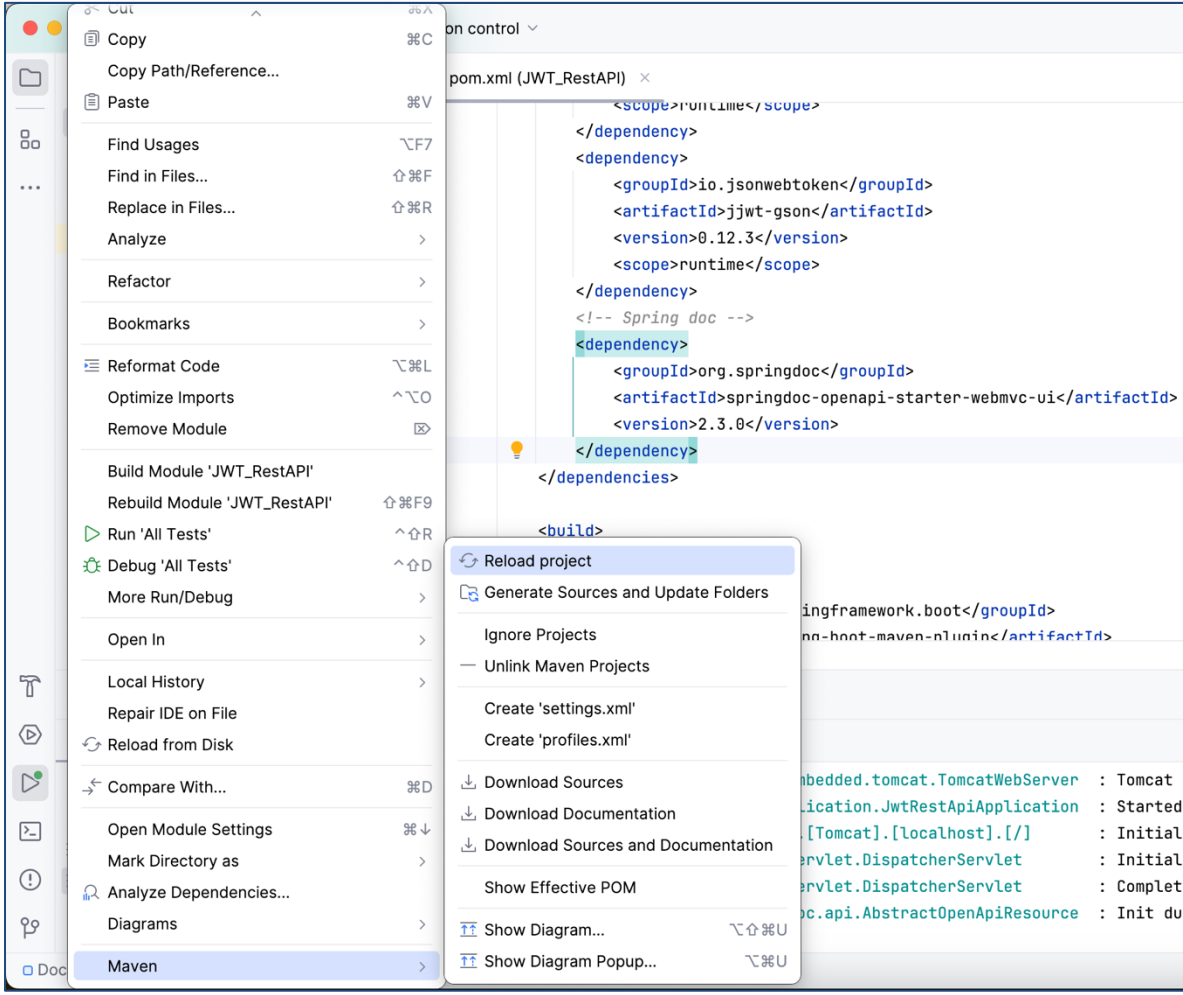
- **Documentando as rotas**:
  - Usaremos como base o projeto da aula passada, **JWT_RestAPI**.
  - Iremos gerar a sua documentação de forma automatizada, utilizando o Spring Doc e o Swagger UI.
  - Primeiro adicionamos a dependência do Spring Doc no arquivo **pom.xml** do nosso projeto.

```
<!-- Spring doc -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.3.0</version>
</dependency>
```

Lembre-se de clicar com o botão direito no projeto e ir em **Maven** > **Reload project** para atualizar as dependências.

Você também pode acessar a Maven no canto superior direito da tela.

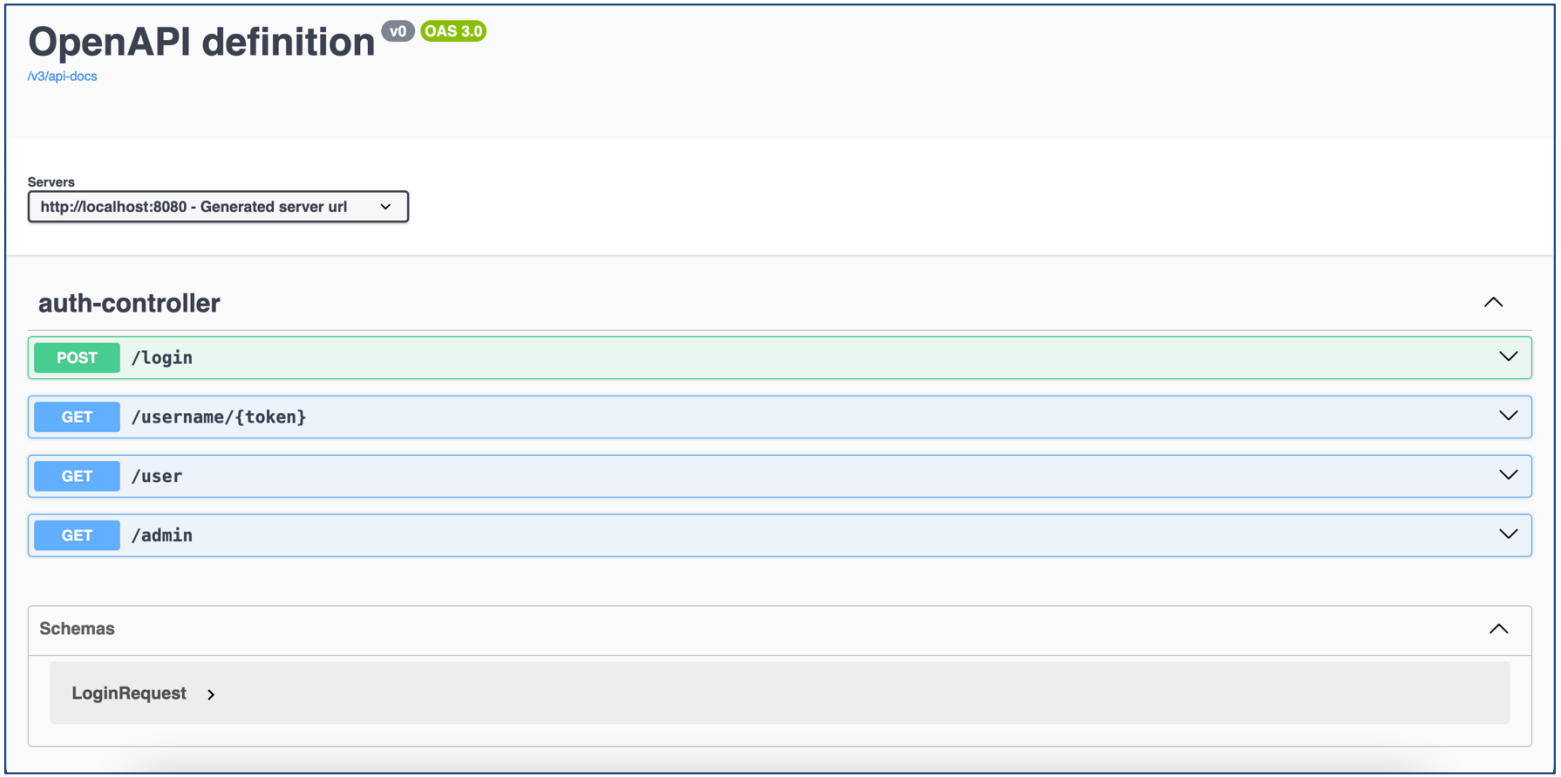




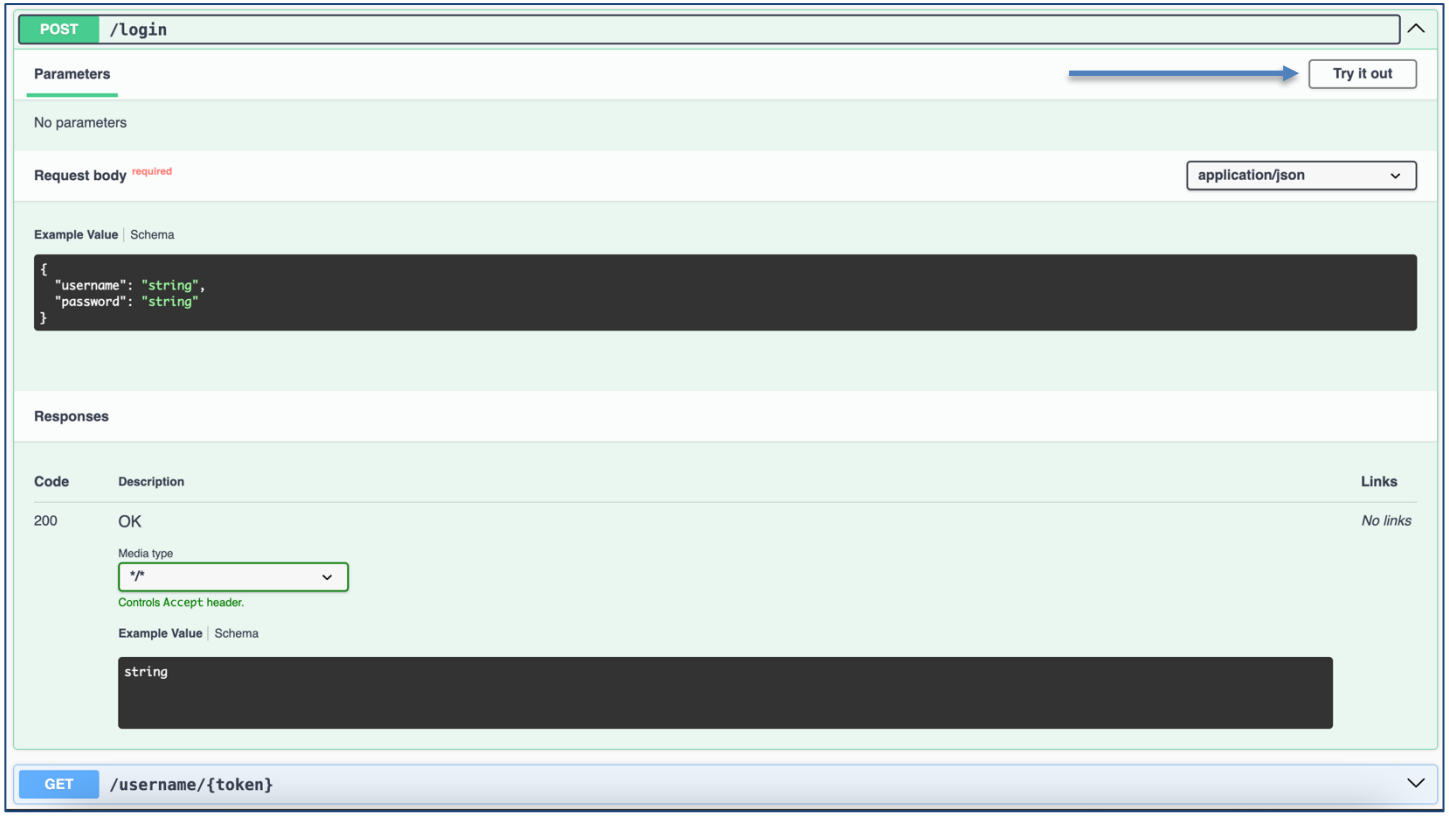
Vamos à prática!

- **Documentando as rotas**:
  - Agora basta acessar a documentação do projeto nos formatos HTML e JSON:
    - [HTML] http://localhost:8080/swagger-ui/index.html
    - [JSON] http://localhost:8080/v3/api-docs
  - Vejamos como ficou =D

- **Documentando as rotas:**
  - http://localhost:8080/swagger-ui/index.html

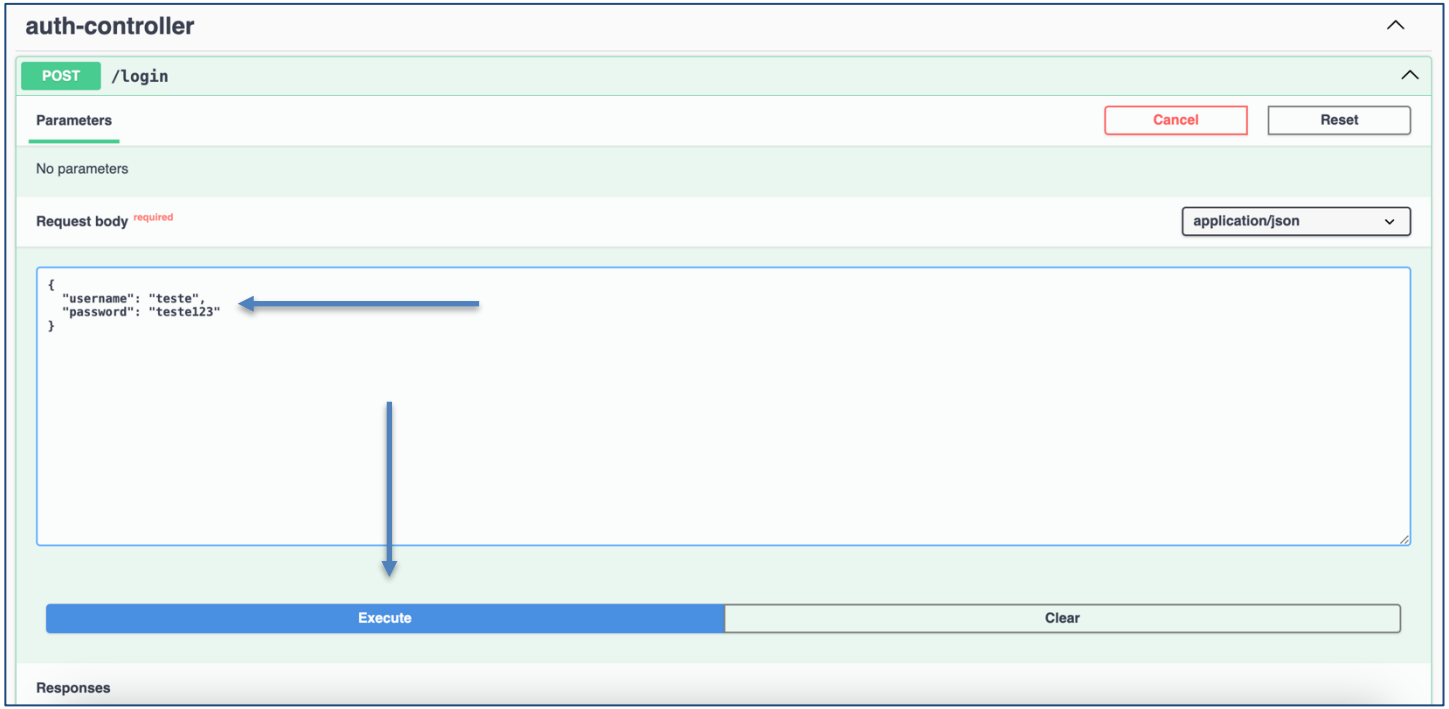


- **Documentando as rotas:**



Podemos inclusive testar requisições simples por aqui, sem recorrer ao Postman ou Insomnia.

- **Documentando as rotas:**



Faça o teste do endpoint POST /login:

HTTP Status Code

Token JWT

Cabeçalho da resposta

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "teste",
  "password": "teste123"
}'
```

Request URL

```
http://localhost:8080/login
```

Server response

Code	Details
200	**Response body**

```
eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJ0ZXN0ZSIsImV4cCI6MTcwNjU3MDU0OH0.Xa1KbrlOfi6WYdnNqu6AbRpwrl2VEl-vL-sgGF925JOzcRtG_IEaBP5wVkDzzf4rB96GFpH68z-9XYl0-9hSjg
```

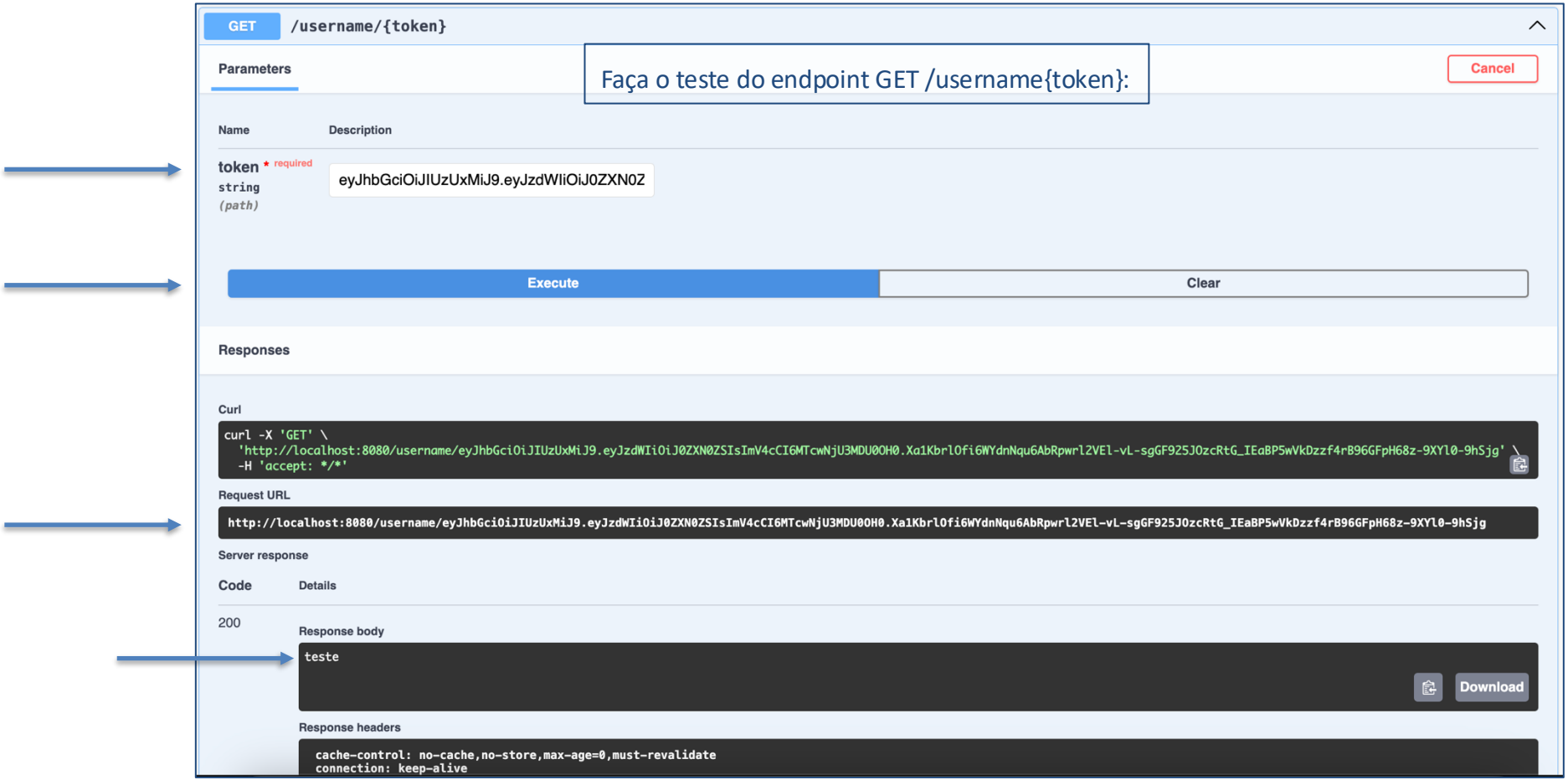
Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-length: 151
content-type: text/plain;charset=UTF-8
date: Fri,19 Jan 2024 23:22:28 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

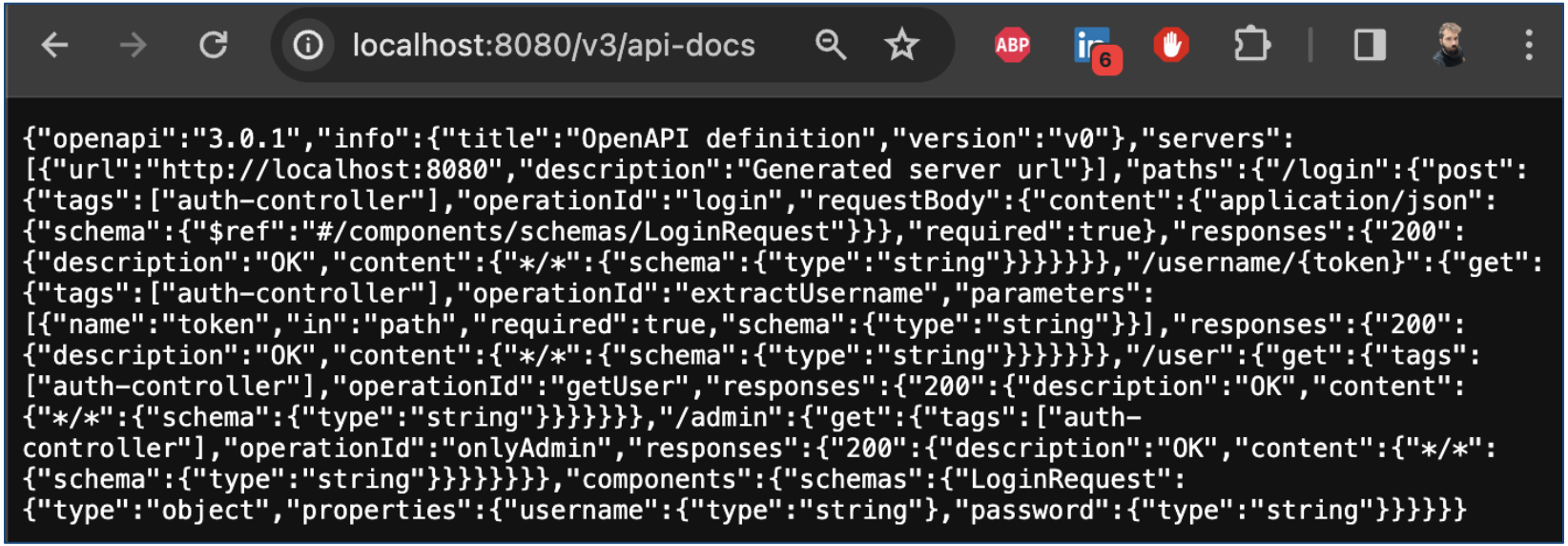
Code	Description	Links
200	OK	*No links*


GET
/username/{token}
Parameters
Faça o teste do endpoint GET /username{token}:
Cancel
Name
Description
token * required
string
(path)
eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJ0ZXN0Z
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:8080/username/eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJ0ZXN0ZSIsImV4cCI6MTcwNjU3MDU0OH0.Xa1KbrlOfi6WYdnNqu6AbRpwrl2VEl-vL-sgGF925JOzcRtG_IEaBP5wVkDzzf4rB96GFpH68z-9XYl0-9hSjg' \
-H 'accept: */*'
Request URL
http://localhost:8080/username/eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJ0ZXN0ZSIsImV4cCI6MTcwNjU3MDU0OH0.Xa1KbrlOfi6WYdnNqu6AbRpwrl2VEl-vL-sgGF925JOzcRtG_IEaBP5wVkDzzf4rB96GFpH68z-9XYl0-9hSjg
Server response
Code
Details
200
Response body
teste
Download
Response headers
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive

- **Documentando as rotas**:
  - Também foi gerada uma especificação OpenAPI no formato JSON.
  - http://localhost:8080/v3/api-docs

  - Este JSON representa a especificação OpenAPI da nossa API REST. Ele descreve os endpoints disponíveis, as operações permitidas, os parâmetros aceitos, os esquemas de dados, entre outras informações. E
  - Essa descrição pode ser usada por ferramentas e clientes que suportam OpenAPI para gerar documentação interativa, testar a API e até mesmo gerar código cliente em várias linguagens de programação.
  - A URL (http://server:port/context-path/v3/api-docs) é um ponto de acesso para obter a descrição OpenAPI no formato JSON.

- **Documentando as rotas**:
  - http://localhost:8080/v3/api-docs
  - Especificação OpenAPI



Especificação OpenAPI

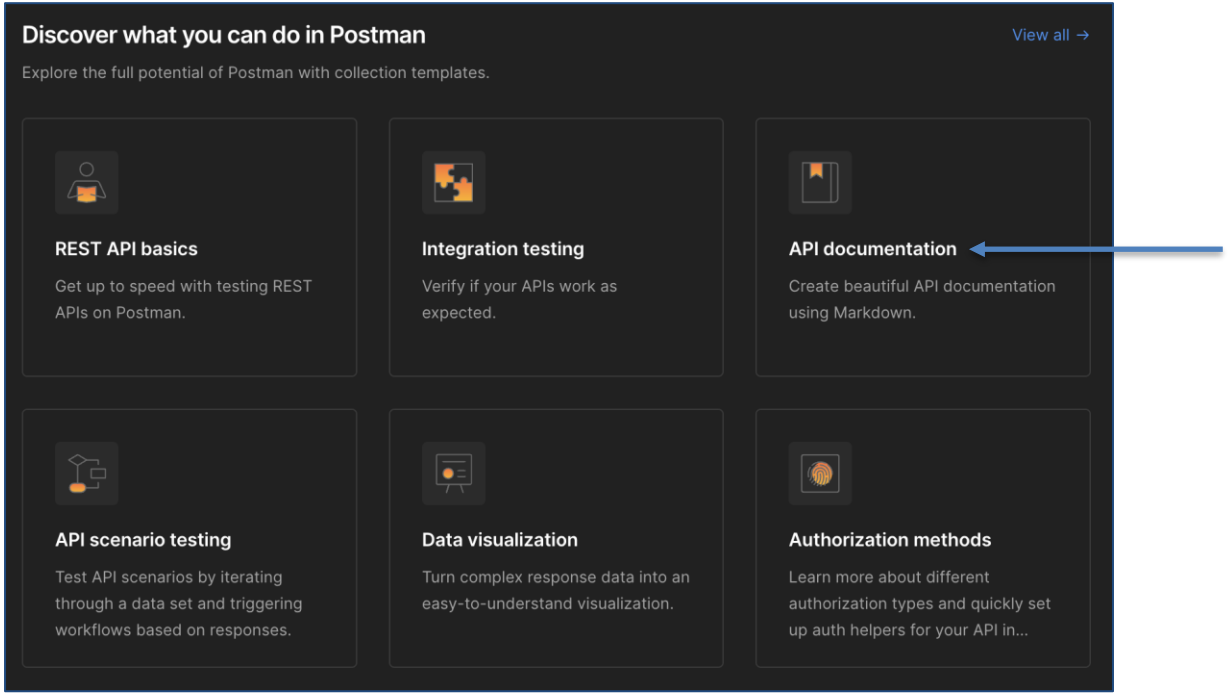
- Vamos analisar as partes principais desse JSON e entender o que significa:
  - **openapi**: Indica a versão da especificação OpenAPI que está sendo usada. No caso, é "3.0.1".
  - **info**: Contém informações básicas sobre a API, como o título e a versão. No exemplo, o título é "OpenAPI definition" e a versão é "v0".
  - **servers**: Fornece informações sobre os servidores nos quais a API está hospedada. No exemplo, há um servidor listado com a URL "http://localhost:8080" e uma descrição "Generated server url".
  - **paths**: Define os caminhos (endpoints) disponíveis na API e as operações suportadas nesses caminhos. Cada caminho é representado como uma chave, e as operações (como POST, GET) são listadas dentro desses caminhos. O exemplo mostra os endpoints "/login", "/username/{token}", "/user" e "/admin", cada um com operações associadas e informações sobre parâmetros, respostas, etc.
  - **components**: Contém definições reutilizáveis, como esquemas (schemas) que são usados em várias partes da especificação. No exemplo, há um esquema chamado "LoginRequest", que é usado como o corpo da requisição na operação POST do endpoint "/login".

- **Documentando as rotas**:
  - **Postman** API Documentation
    - https://learning.postman.com/docs/publishing-your-api/documenting-your-api/
    - Como vimos, o Postman é uma plataforma popular para desenvolvimento de APIs que fornece uma variedade de recursos, incluindo um cliente RESTful intuitivo, testes automatizados, monitoramento de API e, também, uma API de documentação que permite gerenciar e publicar documentação de API.

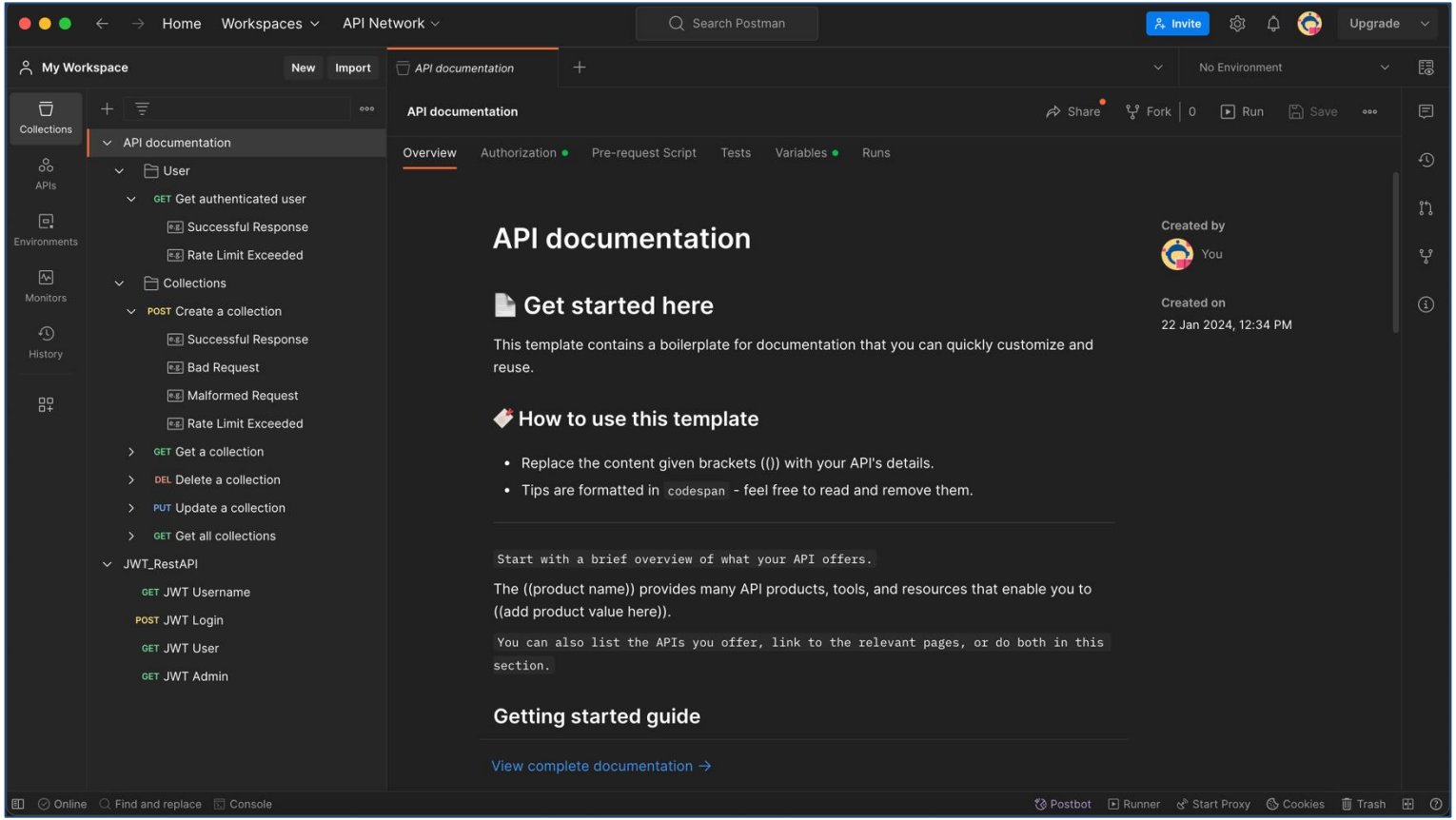
POSTMAN



- **Documentando as rotas**:
  - **Postman** API Documentation
    - https://learning.postman.com/docs/publishing-your-api/documenting-your-api/



- **Postman** API Documentation

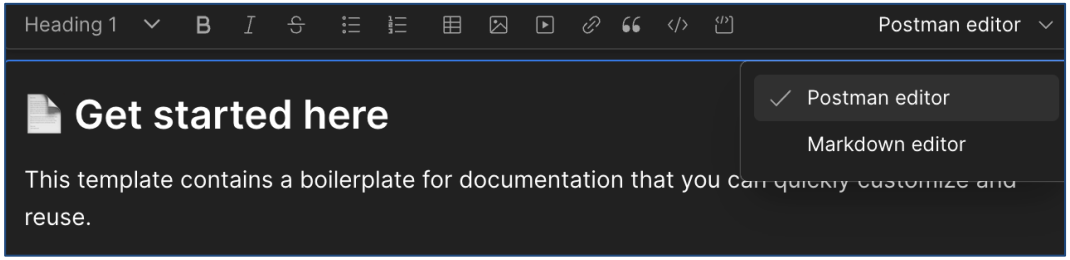


Exercício 1

- Com auxílio do template (e boilerplate) anterior do Postman, construa a documentação para o projeto JWT RestAPI. Crie também os exemplos de requisições bem-sucedidas e exemplos de requisições que falharam.

- Se preferir, utilize também o **Markdown editor** (Markdown é a mesma linguagem usada nos READMEs.md do GitHub).
  - https://docs.github.com/pt/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes
  - https://www.markdownguide.org/

Exercício 1



Authentication

`Add details on the authorization keys/tokens required, steps that cover how to get them, and the relevant error codes.`

The ((product name)) API uses ((add your API's authorization type)) for authentication.

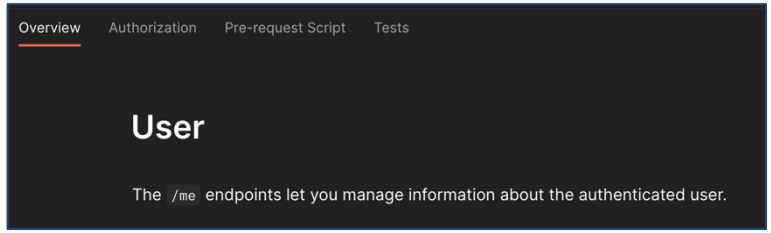
Getting started guide

`List the steps or points required to start using your APIs. Make sure to cover everything required to reach success with your API as quickly as possible.`

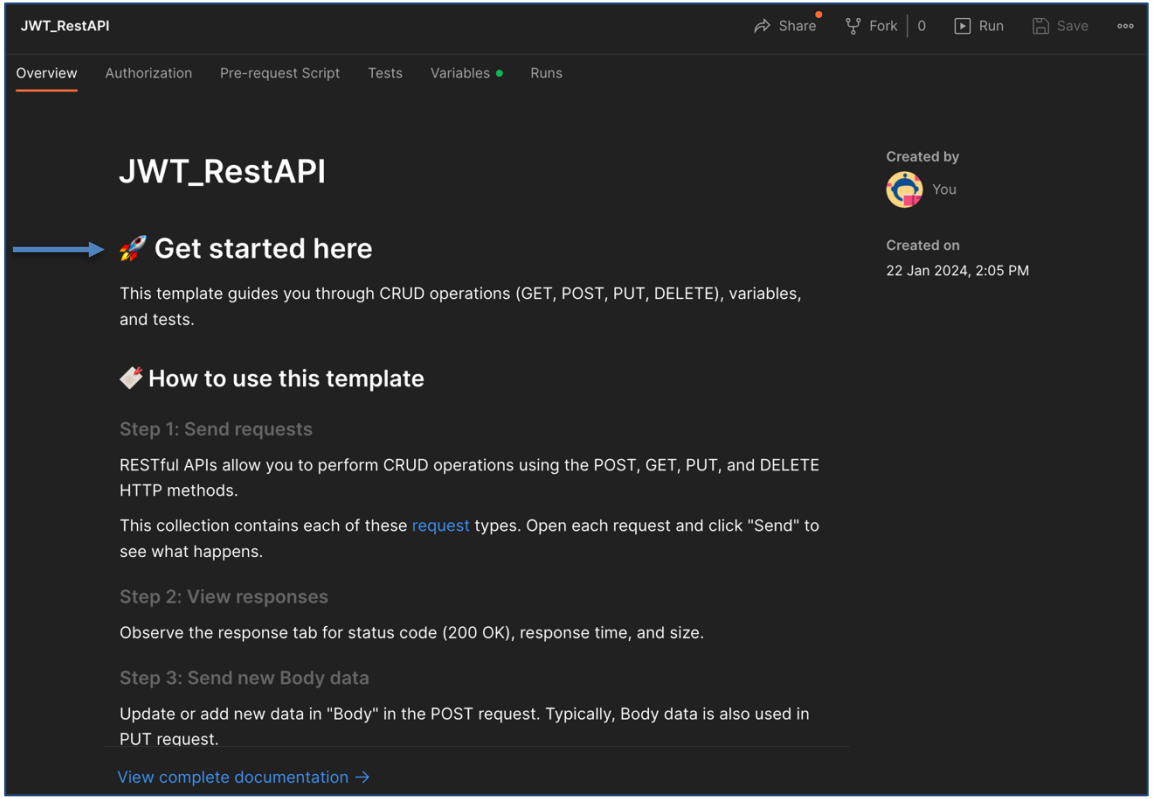
To start using the ((add APIs here)), you need to -

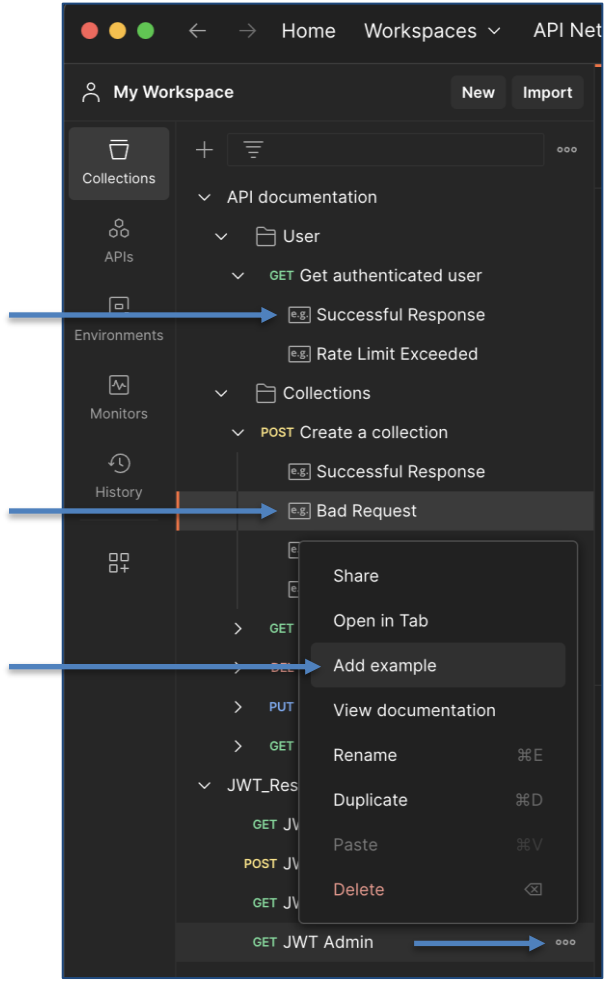
How to use this template

- Replace the content given brackets (()) with your API's details.
- Tips are formatted in `codespan` - feel free to read and remove them.



Exercício 1


JWT_RestAPI
Share
Fork 0
Run
Save
Overview
Authorization
Pre-request Script
Tests
Variables
Runs
JWT_RestAPI
Created by
You
Created on
22 Jan 2024, 2:05 PM
🚀 Get started here
This template guides you through CRUD operations (GET, POST, PUT, DELETE), variables, and tests.
How to use this template
Step 1: Send requests
RESTful APIs allow you to perform CRUD operations using the POST, GET, PUT, and DELETE HTTP methods.
This collection contains each of these request types. Open each request and click "Send" to see what happens.
Step 2: View responses
Observe the response tab for status code (200 OK), response time, and size.
Step 3: Send new Body data
Update or add new data in "Body" in the POST request. Typically, Body data is also used in PUT request.
View complete documentation →


Home
Workspaces
API Net
My Workspace
New
Import
Collections
APIs
Environments
Monitors
History
API documentation
User
GET Get authenticated user
Successful Response
Rate Limit Exceeded
Collections
POST Create a collection
Successful Response
Bad Request
Share
Open in Tab
Add example
View documentation
Rename ⌘E
Duplicate ⌘D
Paste ⌘V
Delete
GET JWT Admin

JavaDoc

- Adicionalmente, você também pode gerar a documentação JavaDoc.
  - https://www.jetbrains.com/help/idea/javadocs.html

  - O JavaDoc é uma ferramenta de documentação automática padrão para a linguagem Java.

  - Ele é usado para gerar documentação a partir de **comentários** incorporados diretamente no código-fonte Java.

JavaDoc

- Esses comentários são escritos em um formato específico e incluem informações sobre classes, métodos, campos e outros elementos do código.

- Os comentários do JavaDoc são delimitados por **/**** no início e ***/** no final.

- Documentação:
  - https://docs.oracle.com/javase/8/docs/technotes/tools/windows/javadoc.html
  - https://www.oracle.com/br/technical-resources/articles/java/javadoc-tool.html

JavaDoc

- Os comentários Javadoc geralmente são colocados acima de classes, métodos ou campos em seu código-fonte.

- Um Javadoc fornece uma descrição do elemento de código localizado abaixo dele e contém tags de bloco marcadas com @ com metadados específicos.

- Vejamos um exemplo.

JavaDoc

- Comentários JavaDoc em **classes**:

AuthService.java

```java
package com.example.JWT_RestAPI.service;

import com.example.JWT_RestAPI.security.JwtUtil;
import org.springframework.stereotype.Service;

/**
 * Serviço responsável por operações relacionadas à autenticação e geração de tokens JWT.
 *
 * @author <a href="mailto:joaopauloaramuni@gmail.com">João Paulo Aramuni</a>
 * @version 1.0
 * @since 2024-01-20
 */
@Service
public class AuthService {
```

JavaDoc

- Comentários JavaDoc em **métodos**:

AuthService.java

```java
@Service
public class AuthService {

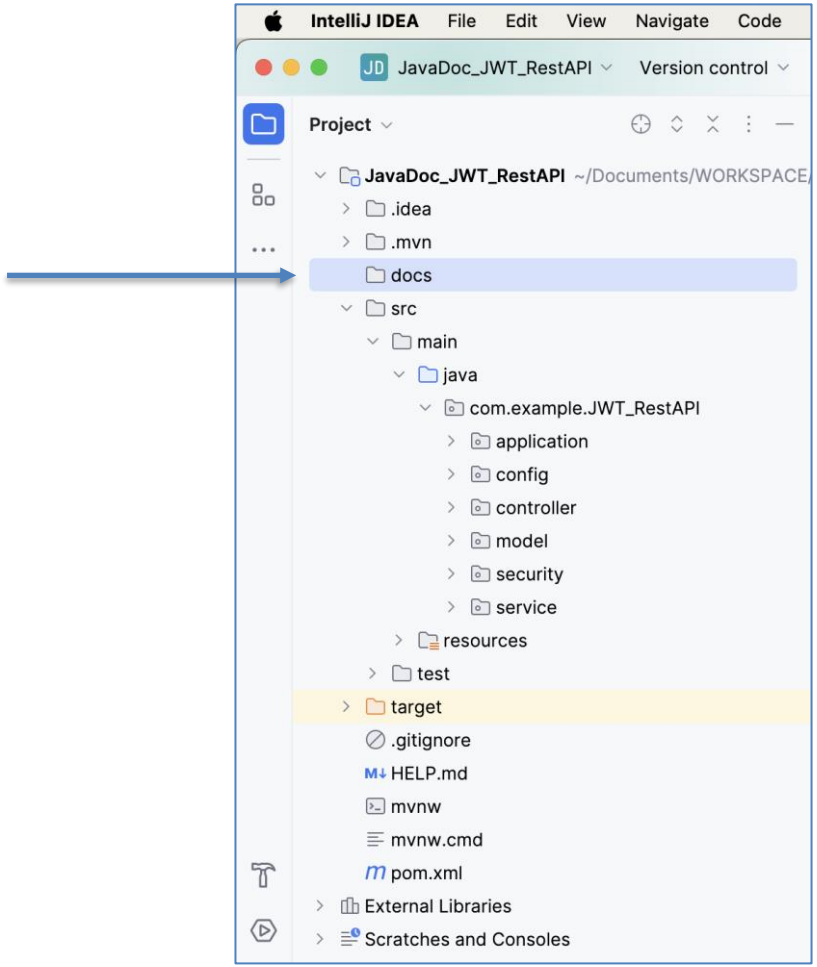
    /**
     * Gera um token JWT para o nome de usuário fornecido.
     *
     * @param username O nome de usuário para o qual um token JWT será gerado.
     * @return Uma string contendo o token JWT gerado.
     */
    public String generateToken(String username) {
        String token = JwtUtil.generateToken(username);
        return token;
    }

    /**
     * Extrai o nome de usuário de um token JWT.
     *
     * @param token O token JWT do qual o nome de usuário será extraído.
     * @return Uma string contendo o nome de usuário extraído do token JWT.
     */
    public String extractUsername(String token) {
        String username = JwtUtil.extractUsername(token);
        return username;
    }
}
```

JavaDoc

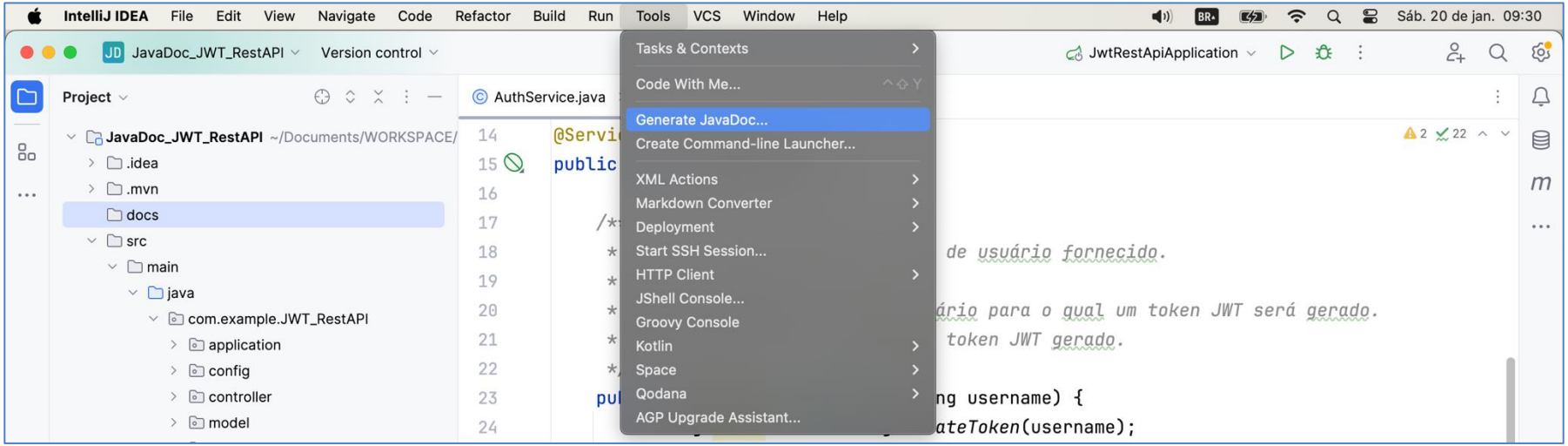
- Crie uma pasta **docs** na raiz do projeto.

- Aqui ficarão os arquivos HTML da documentação gerados pelo JavaDoc.



JavaDoc

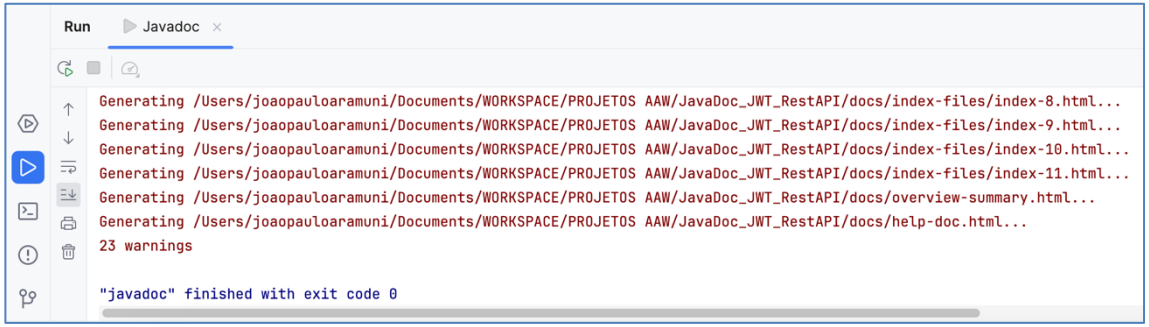
- No menu do IntelliJ IDEA navegue até **Tools** > **Generate JavaDoc...**

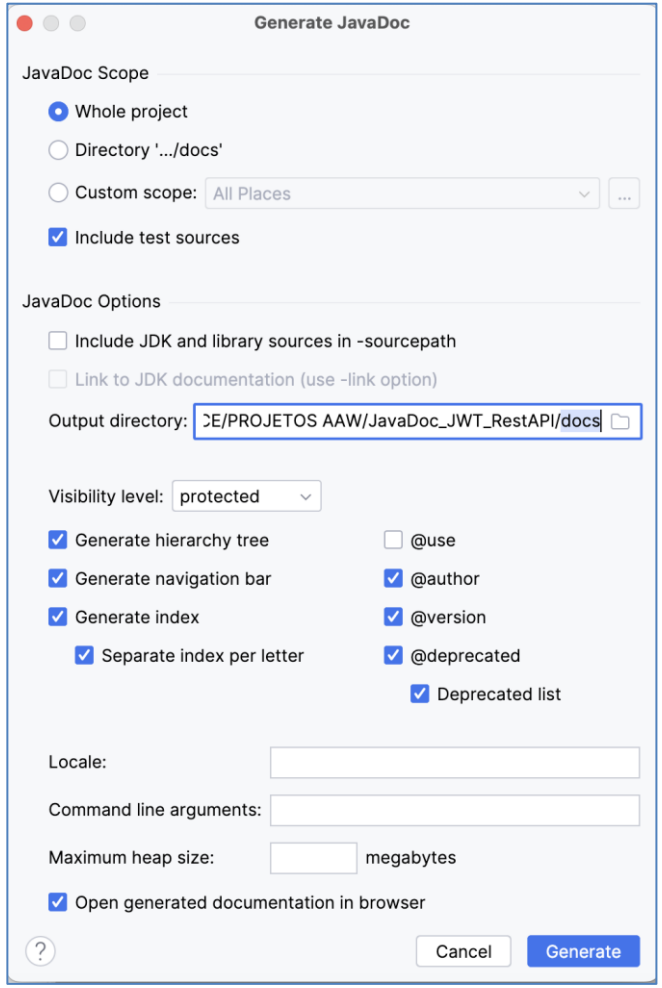


JavaDoc

- Adicione a pasta **docs** em **Output directory**.

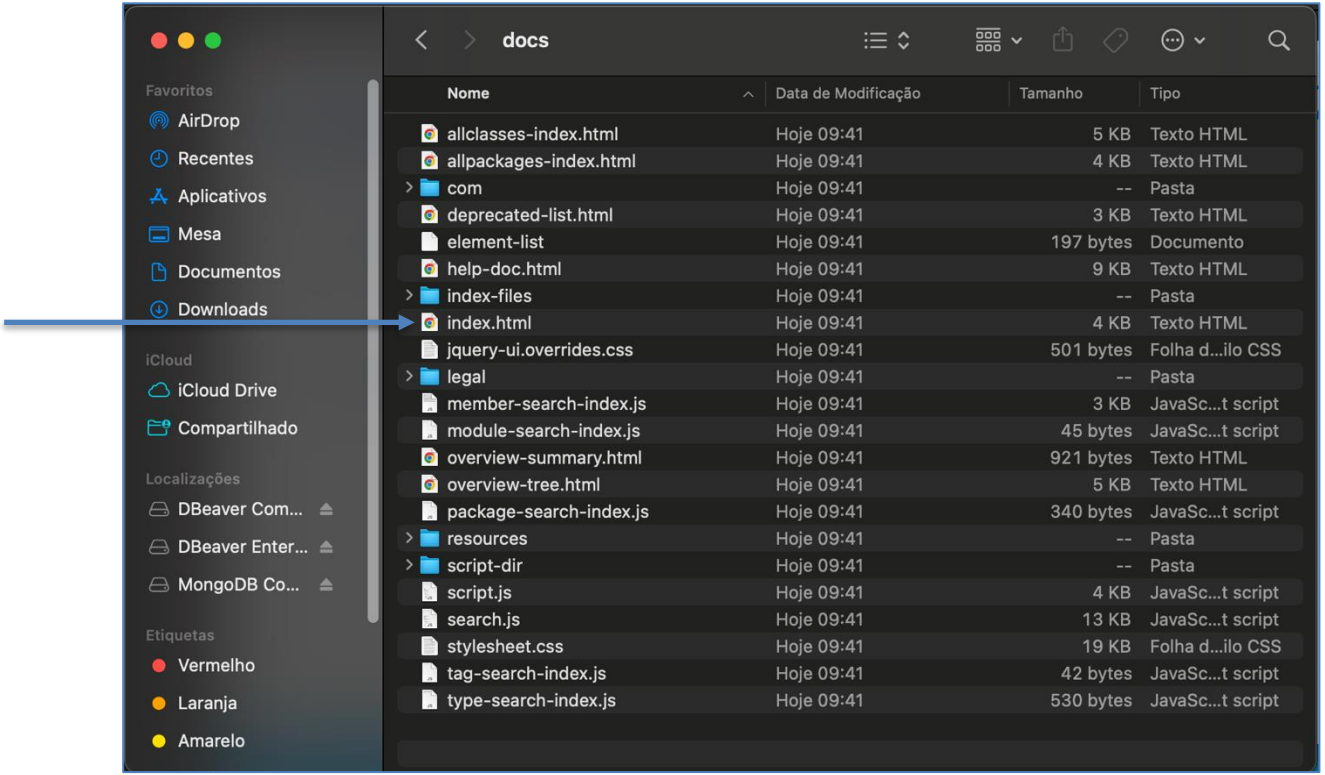
- Clique em **Generate**.



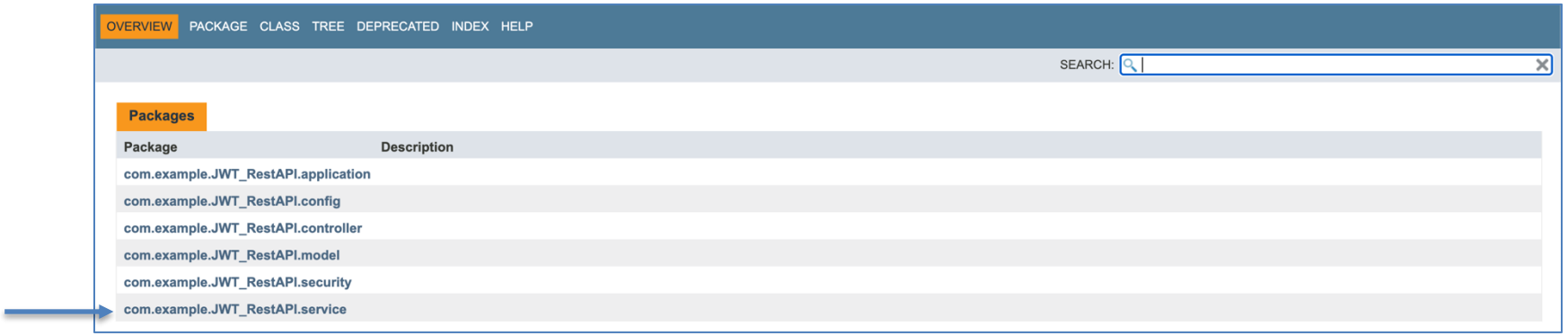


JavaDoc

- Abra o arquivo **index.html**.

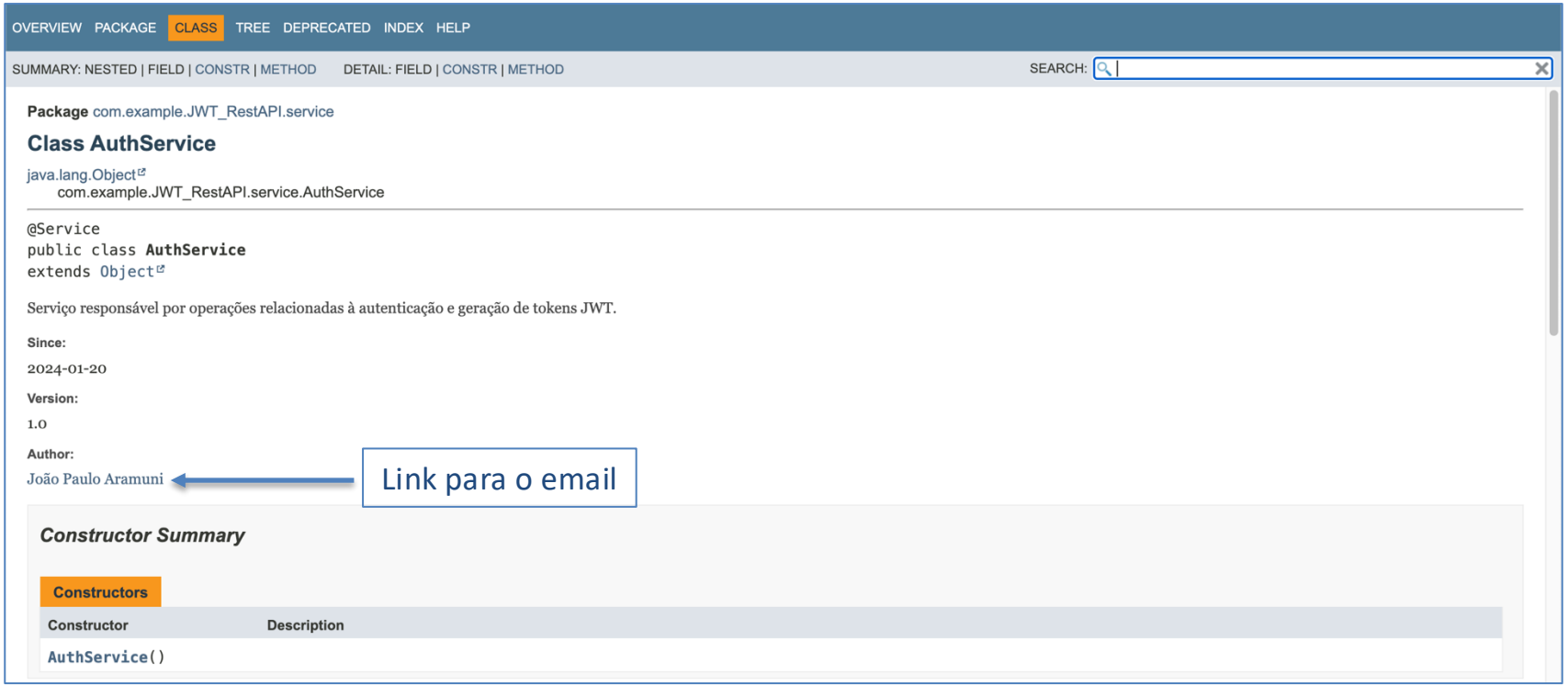


JavaDoc

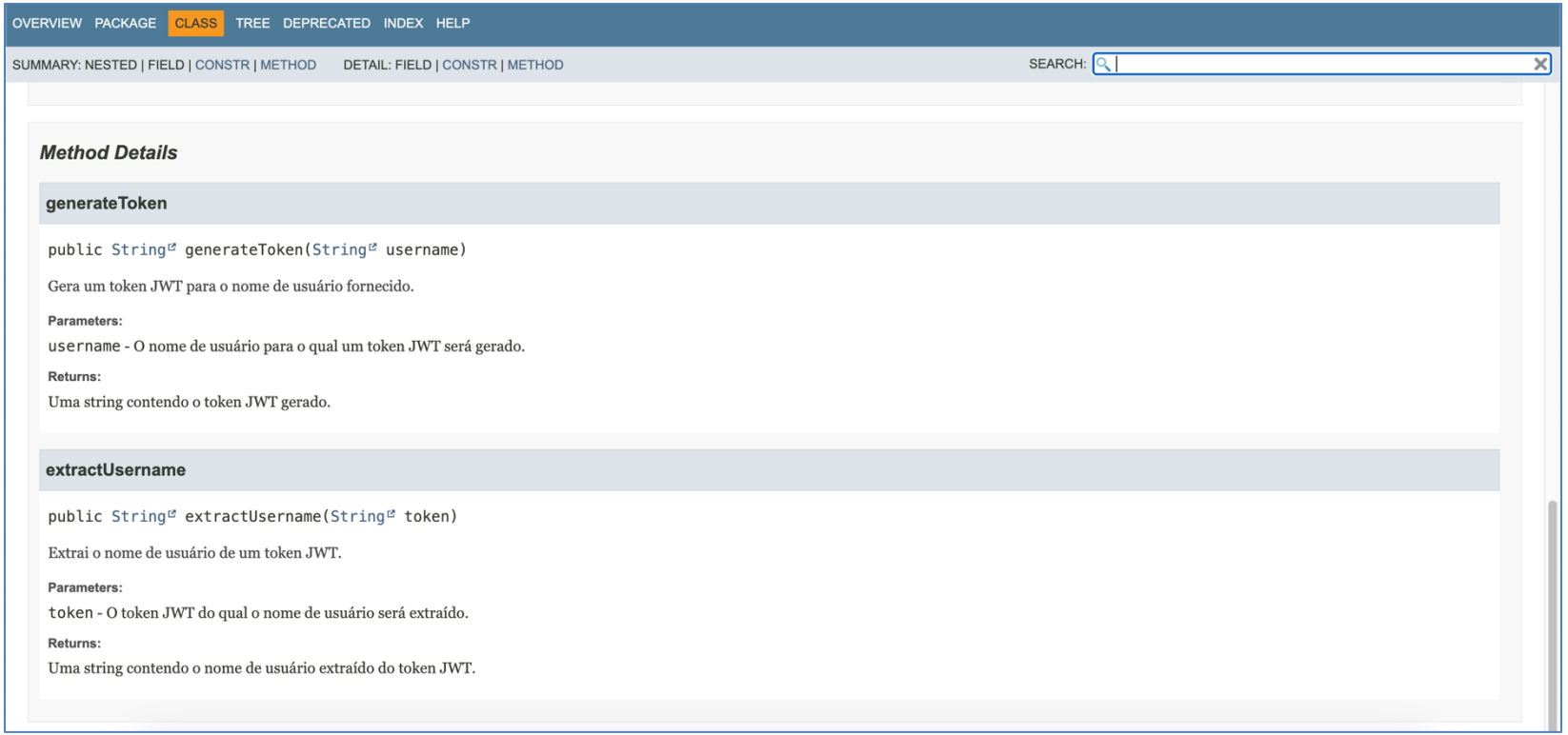

OVERVIEW PACKAGE CLASS TREE DEPRECATED INDEX HELP
SEARCH:
Packages
Package
Description
com.example.JWT_RestAPI.application
com.example.JWT_RestAPI.config
com.example.JWT_RestAPI.controller
com.example.JWT_RestAPI.model
com.example.JWT_RestAPI.security
com.example.JWT_RestAPI.service


OVERVIEW PACKAGE CLASS TREE DEPRECATED INDEX HELP
PACKAGE: DESCRIPTION | RELATED PACKAGES | CLASSES AND INTERFACES
SEARCH:
Package com.example.JWT_RestAPI.service
package com.example.JWT_RestAPI.service
Classes
Class
Description
AuthService
Serviço responsável por operações relacionadas à autenticação e geração de tokens JWT.

JavaDoc


OVERVIEW PACKAGE CLASS TREE DEPRECATED INDEX HELP
SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD
SEARCH:
Package com.example.JWT_RestAPI.service
Class AuthService
java.lang.Object
com.example.JWT_RestAPI.service.AuthService
@Service
public class AuthService
extends Object
Serviço responsável por operações relacionadas à autenticação e geração de tokens JWT.
Since:
2024-01-20
Version:
1.0
Author:
João Paulo Aramuni
Link para o email
Constructor Summary
Constructors
Constructor
Description
AuthService()

JavaDoc


OVERVIEW PACKAGE CLASS TREE DEPRECATED INDEX HELP
SUMMARY: NESTED | FIELD | CONSTR | METHOD
DETAIL: FIELD | CONSTR | METHOD
SEARCH:
Method Details
generateToken
public String generateToken(String username)
Gera um token JWT para o nome de usuário fornecido.
Parameters:
username - O nome de usuário para o qual um token JWT será gerado.
Returns:
Uma string contendo o token JWT gerado.
extractUsername
public String extractUsername(String token)
Extrai o nome de usuário de um token JWT.
Parameters:
token - O token JWT do qual o nome de usuário será extraído.
Returns:
Uma string contendo o nome de usuário extraído do token JWT.

Exercício 2

- Construa os comentários JavaDoc para as demais classes e métodos do projeto JWT_RestAPI.



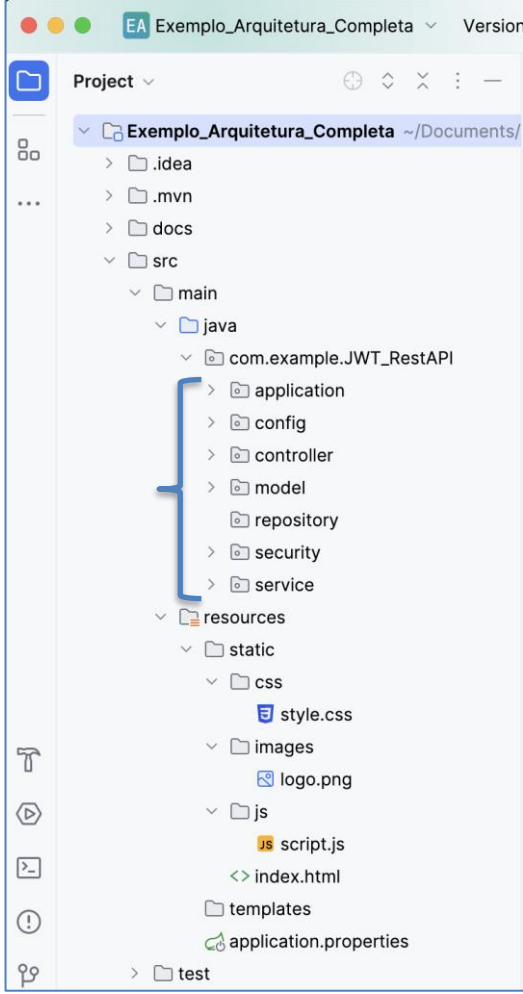
Exercício 3

- Construa o diagrama de arquitetura para o projeto JWT RestAPI utilizando o figma.

- Links úteis:
  - https://www.figma.com/templates/engineering/
  - Exemplo no Figma de diagrama de arquitetura REST API com JSON Web Token (JWT):
    - https://www.figma.com/community/file/1275291490973723744
    - Link do projeto no Figma.

Juntando tudo que aprendemos

- Veja um exemplo de arquitetura completa do projeto:
  - Pasta **docs** para a documentação **JavaDoc**.
  - Pasta **src** com os pacotes **application**, **config**, **controller**, **model**, **repository**, **security** e **service**.
  - Pasta **resources** com os recursos estáticos e templates.
  - Pasta **test** para testes com **JUnit Jupiter** e **Mockito**.

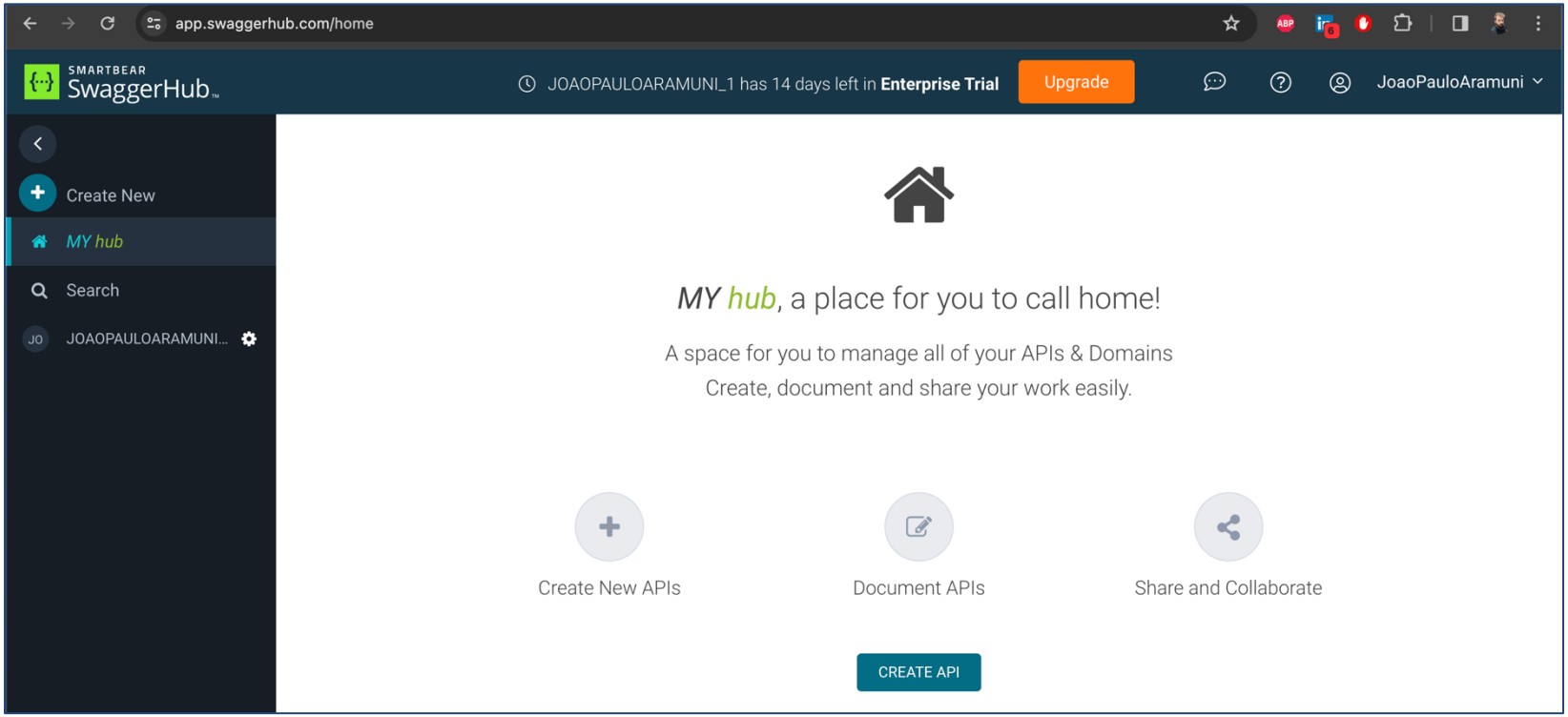


Bônus

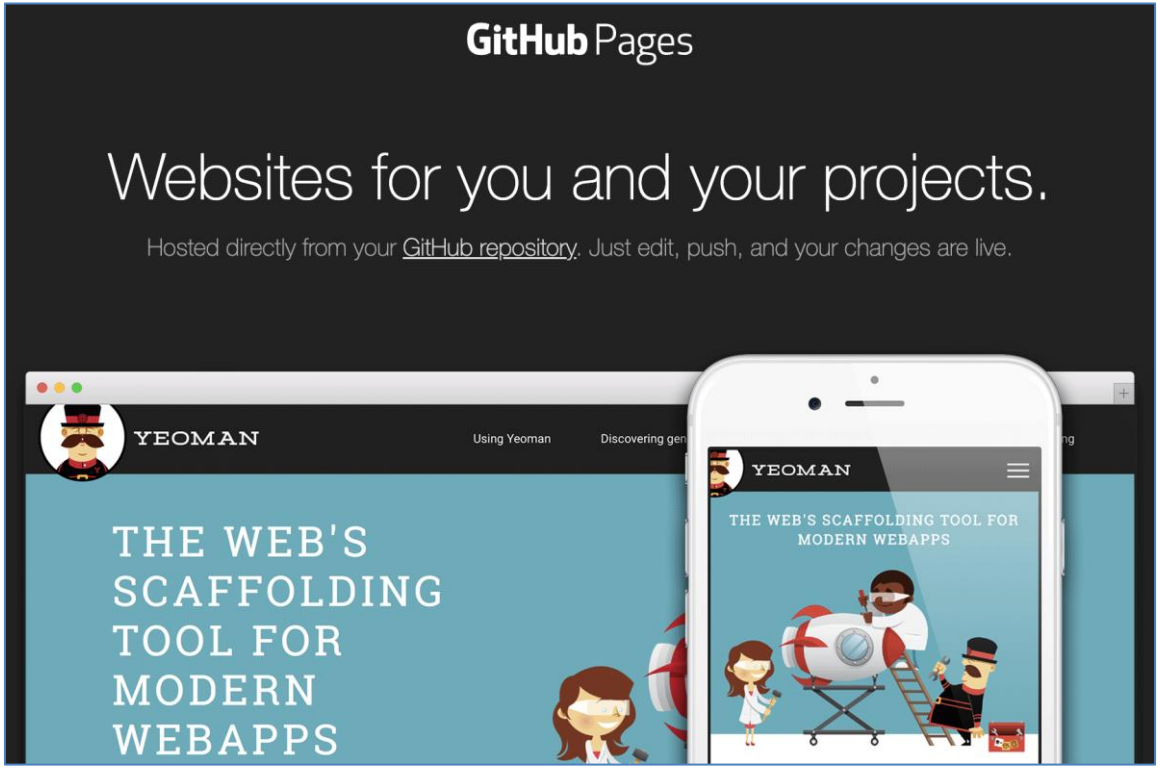
- **Hospedando a sua documentação no GitHub Pages**:
  - Agora que já geramos a documentação da JWT_RestAPI, podemos hospedá-la (juntamente com o diagrama de arquitetura), no **GitHub Pages**.
    - https://pages.github.com/
    - https://docs.github.com/pt/pages

  - O Swagger também possui um Hub para gestão de APIs (e hospedagem de documentação), porém ele oferece apenas 14 dias para teste.
    - https://swagger.io/tools/swaggerhub/

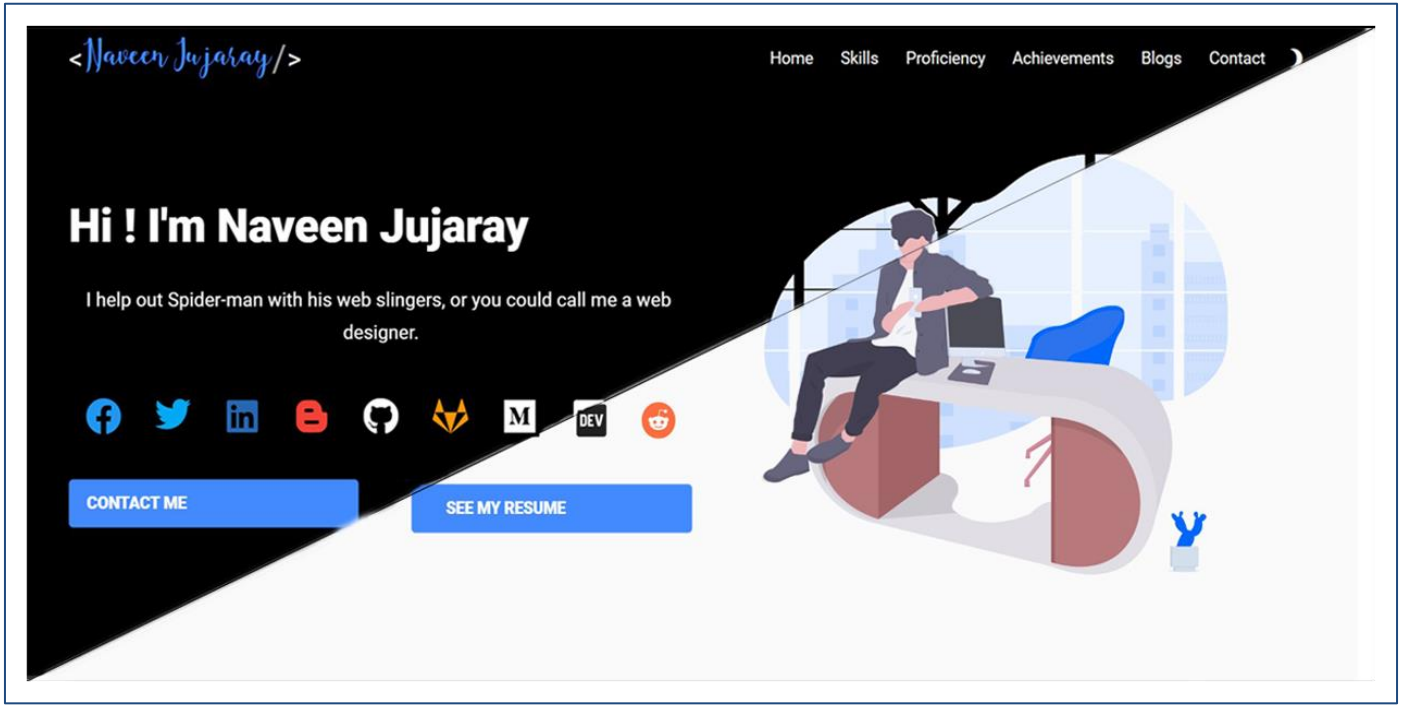
- **SwaggerHub**: O SwaggerHub é uma plataforma da SmartBear Software projetada para auxiliar no desenvolvimento, design e documentação de APIs.



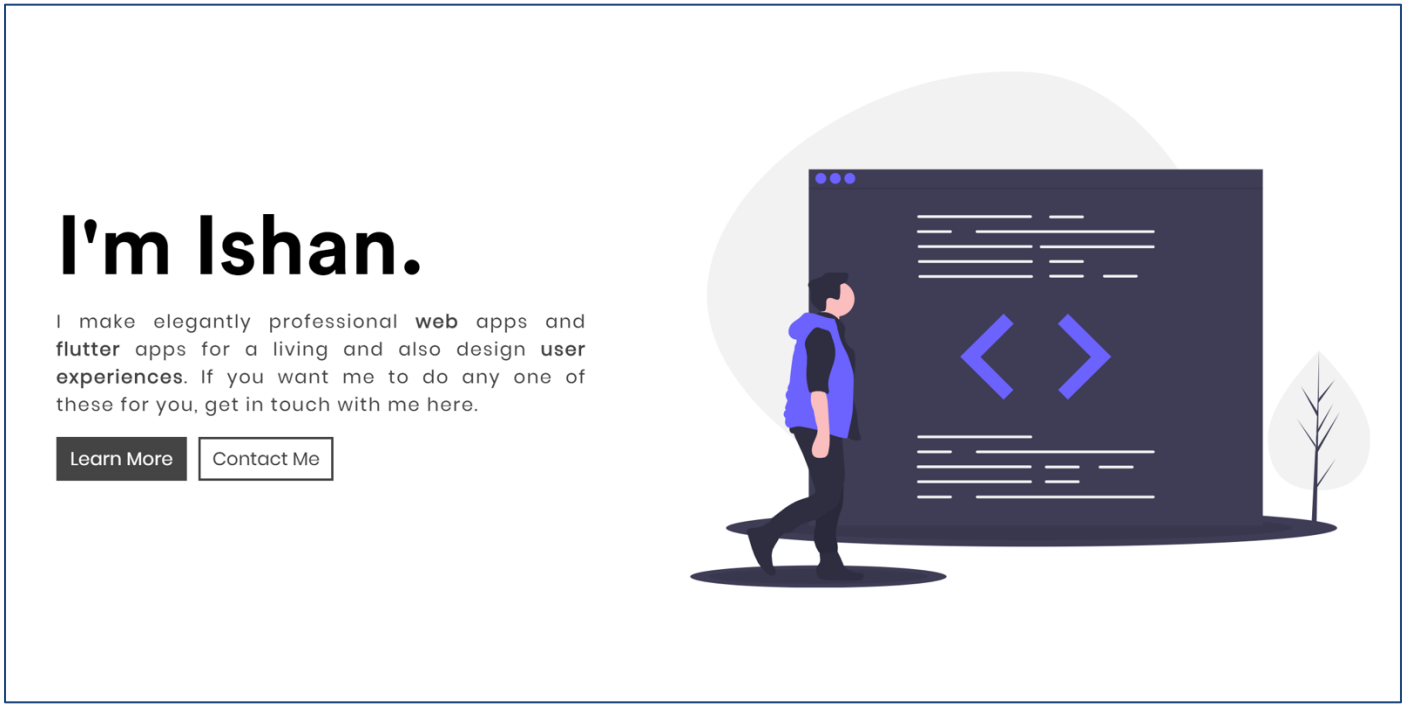
- **GitHub Pages**: Websites para você e seus projetos.
  - ***https://seunomeusuario.github.io/****seurepositorio*



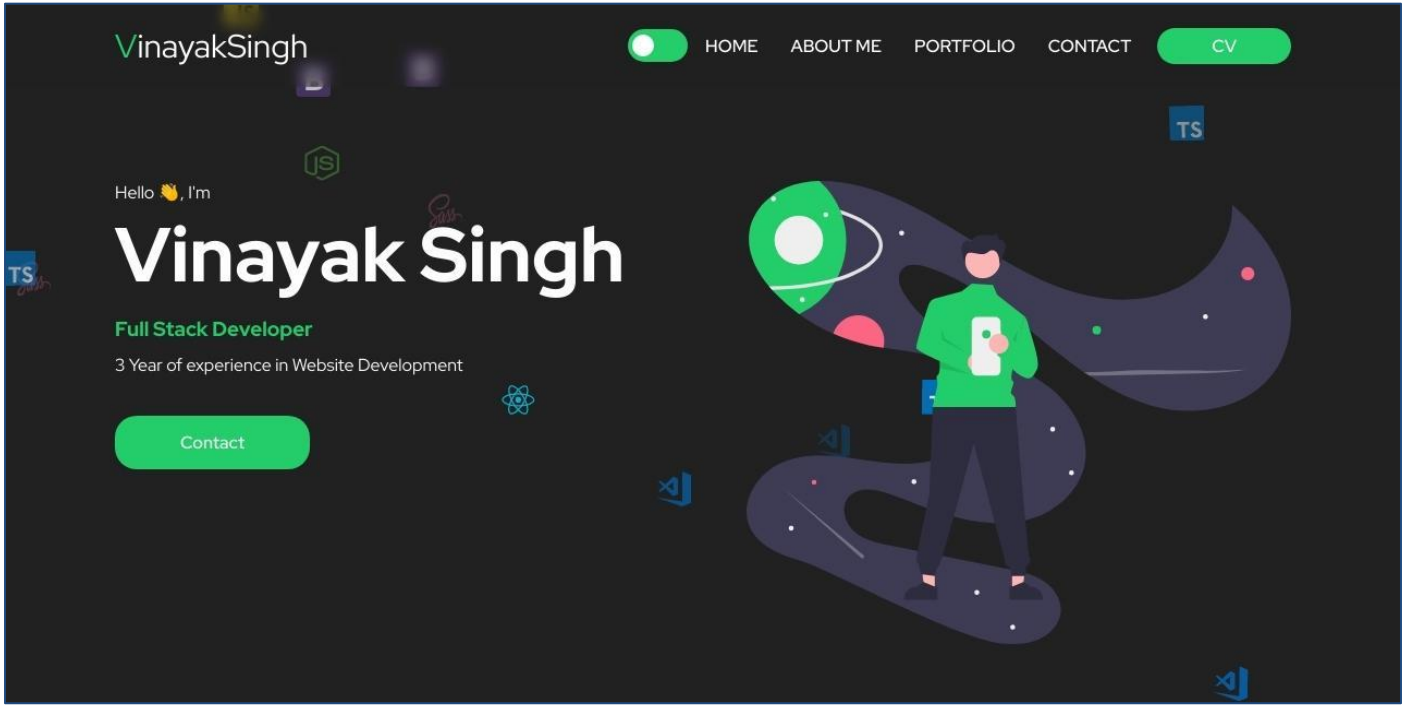
- **GitHub Pages**: Websites para você e seus projetos.
  - Além dos projetos, é possível também construir um perfil para você utilizando diversos templates.



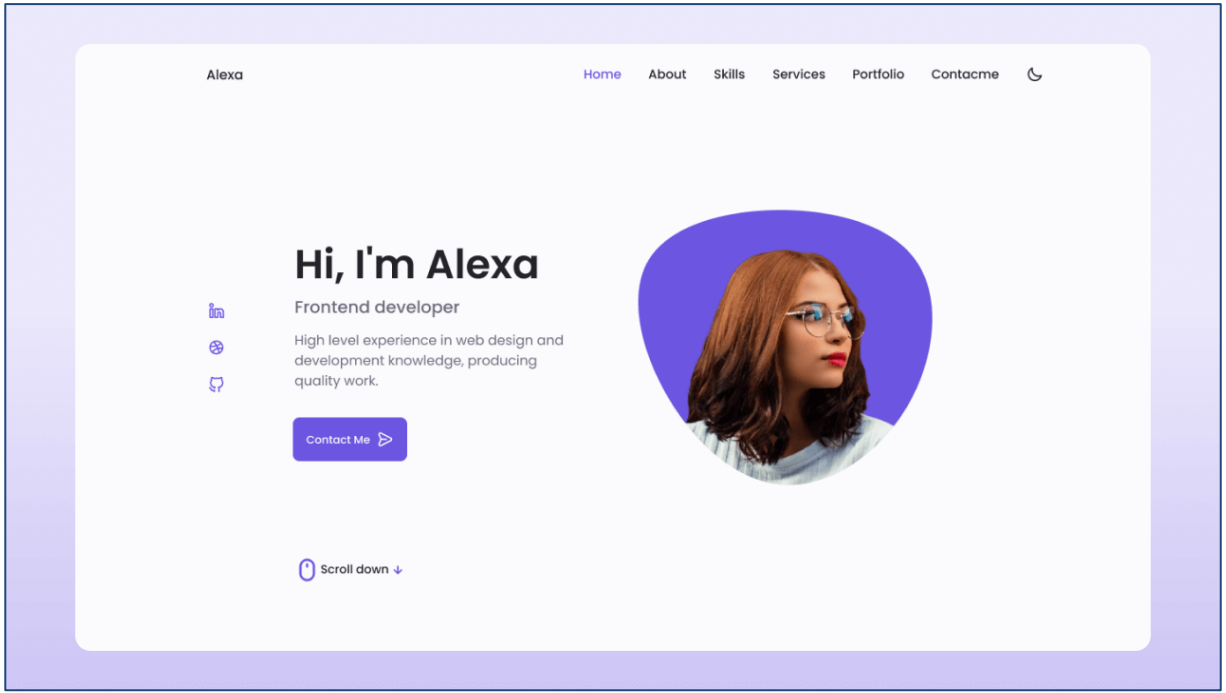
- **GitHub Pages**: Websites para você e seus projetos.
  - Além dos projetos, é possível também construir um perfil para você utilizando diversos templates.



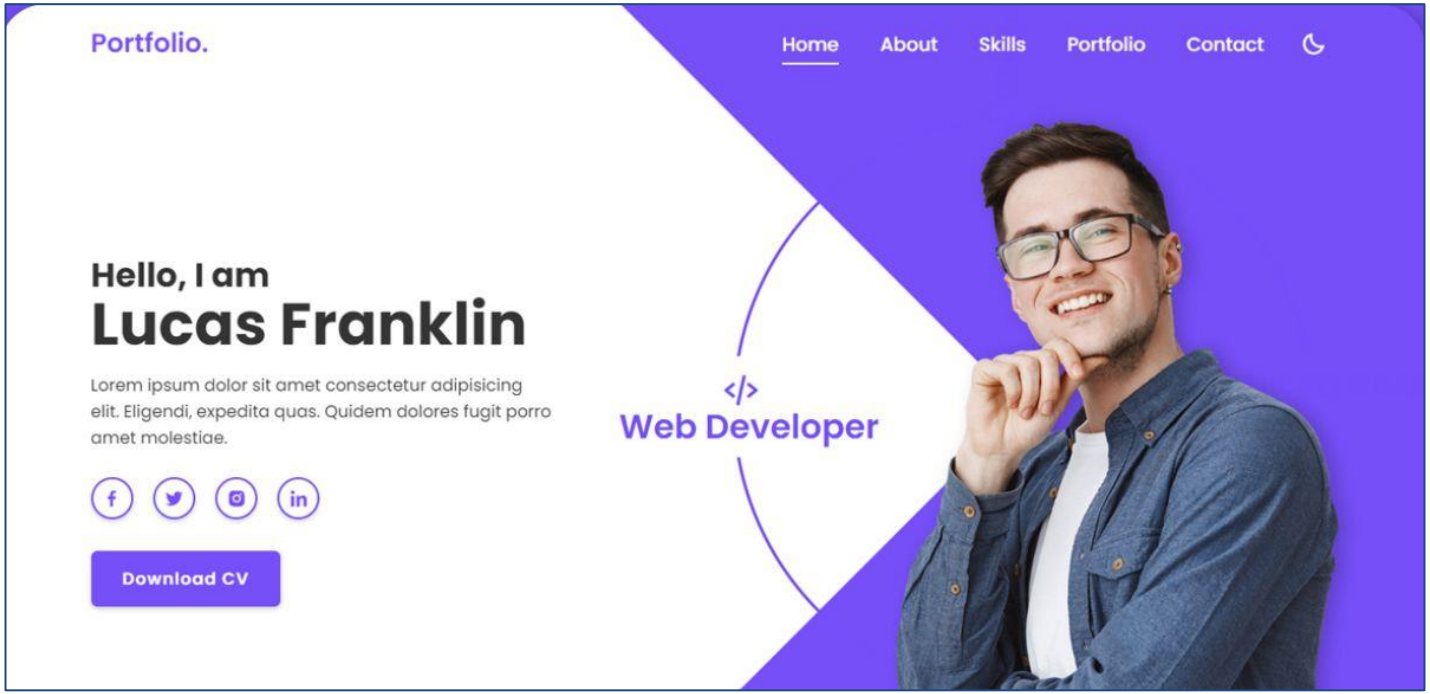
- **GitHub Pages**: Websites para você e seus projetos.
  - Além dos projetos, é possível também construir um perfil para você utilizando diversos templates.



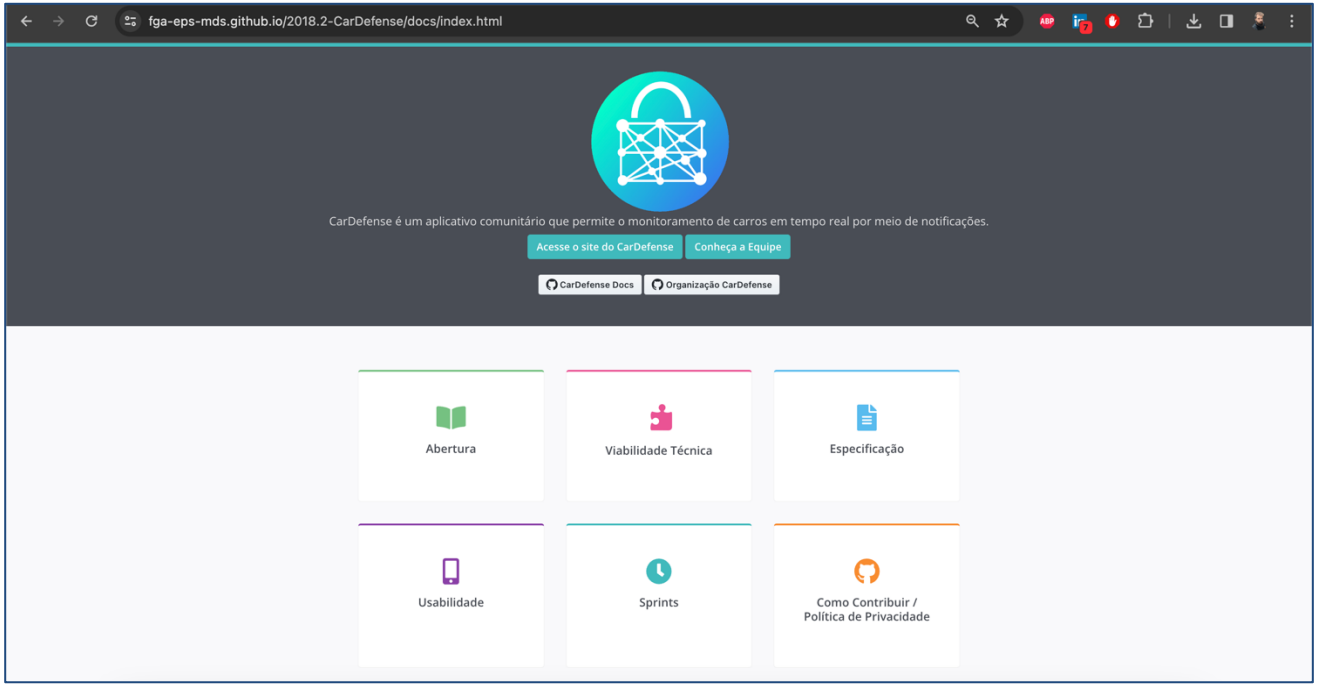
- **GitHub Pages**: Websites para você e seus projetos.
  - A URL do seu **perfil** será:
    - *https://seunomeusuario.github.io/*



- **GitHub Pages**: Websites para você e seus projetos.
  - A URL do seu **perfil** será:
    - *https://seunomeusuario.github.io/*



- **GitHub Pages**: Websites para você e seus projetos.
  - A URL do seu **projeto** será:
    - *https://seunomeusuario.github.io/**seurepositorio***



- **GitHub Pages**: Websites para você e seus projetos.
  - A URL do seu **projeto** será:
    - *https://seunomeusuario.github.io/**seurepositorio***



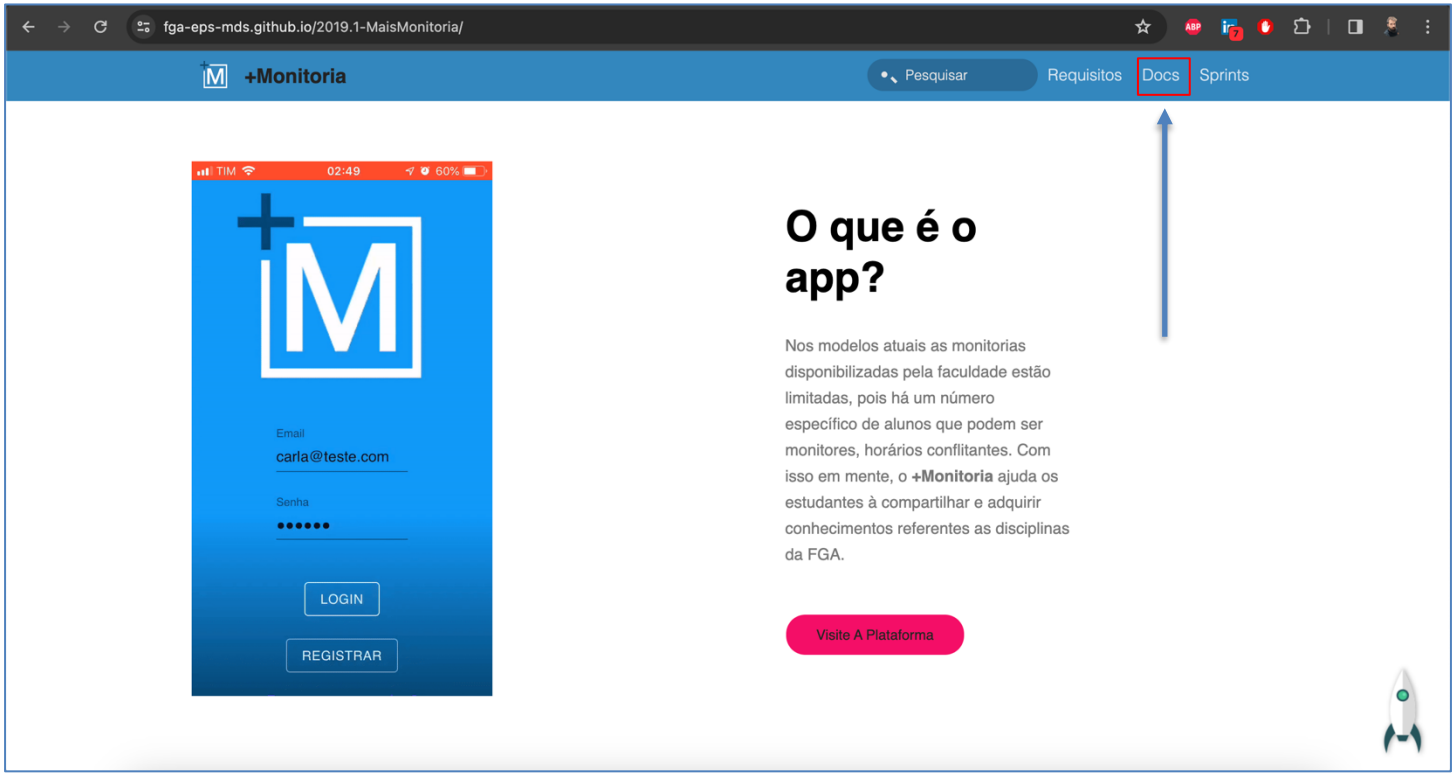
- **GitHub Pages**: Websites para você e seus projetos.
  - https://pythonmg.github.io/



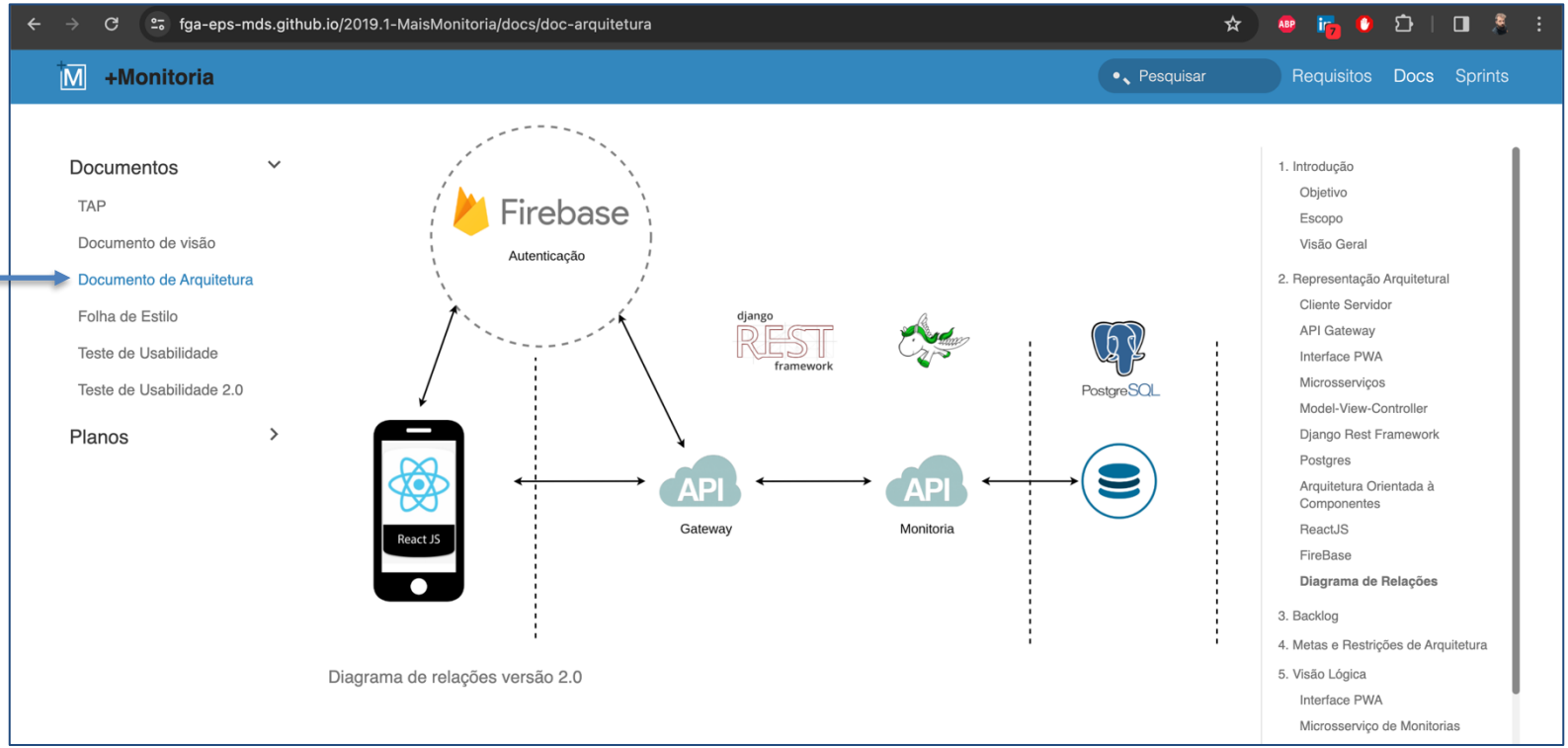
- **GitHub Pages**: Websites para você e seus projetos.
  - Vejamos um exemplo de projeto:



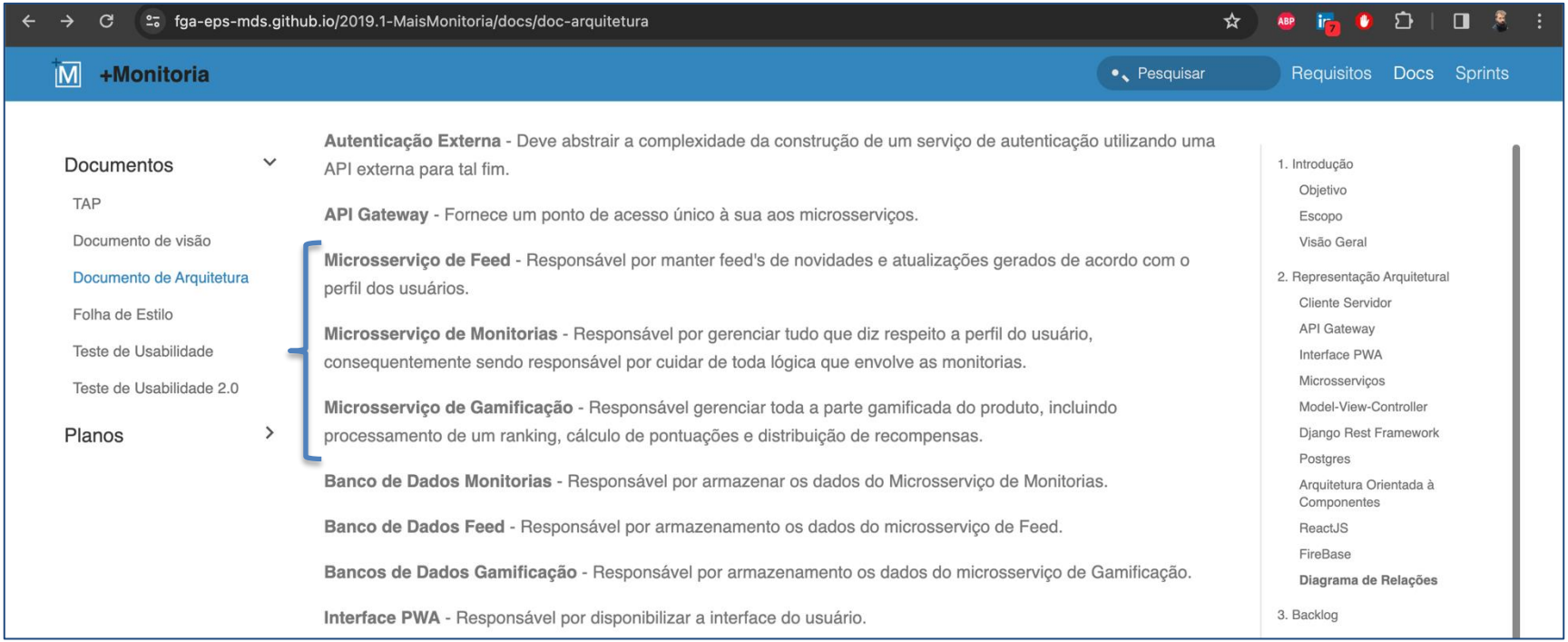
- **GitHub Pages**: Websites para você e seus projetos.
  - Vejamos um exemplo de projeto:



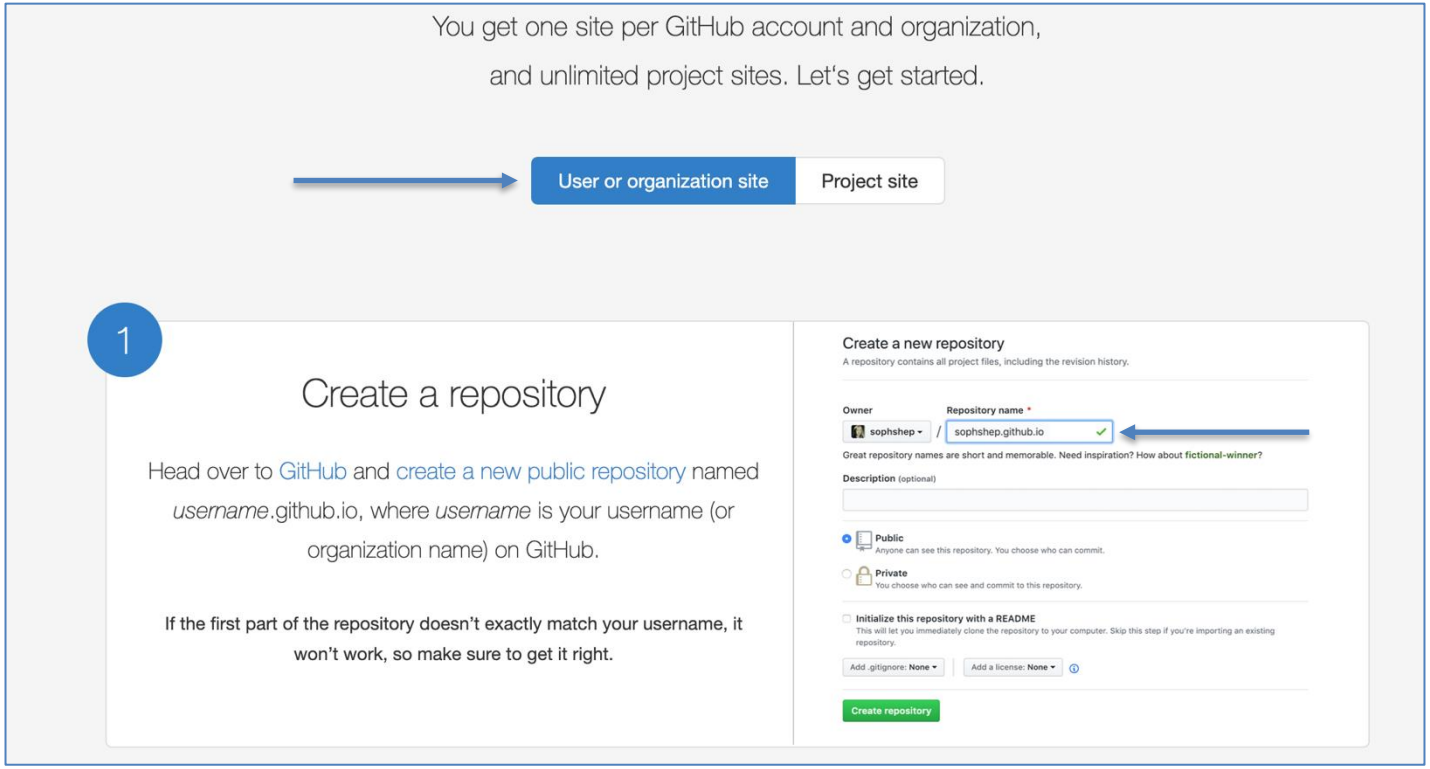
- **GitHub Pages**: Websites para você e seus projetos.
  - Vejamos um exemplo de projeto:



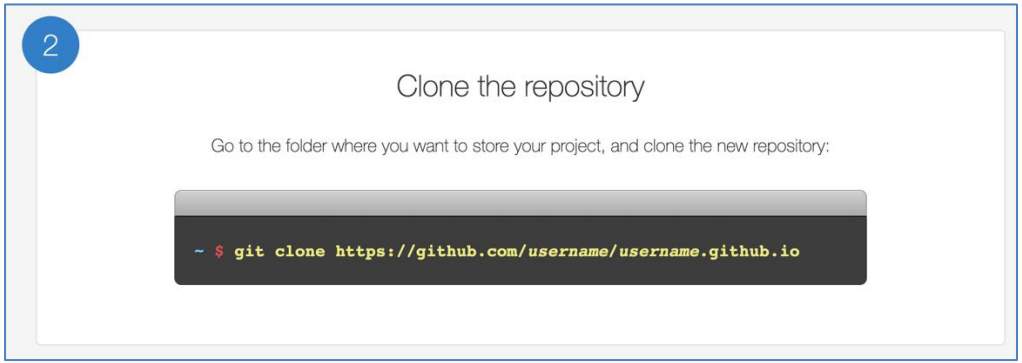
- **GitHub Pages**: Websites para você e seus projetos.
  - Vejamos um exemplo de projeto:



- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **perfil**:



- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **perfil**:



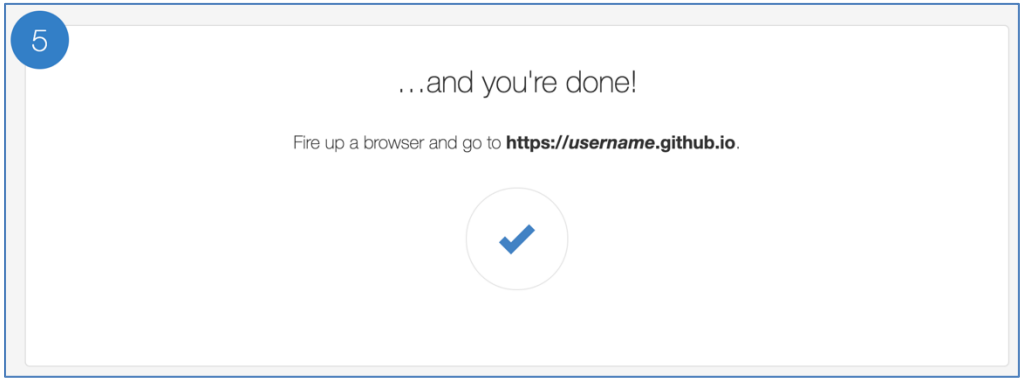


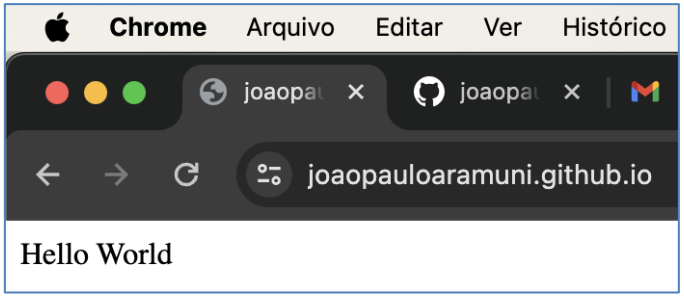
- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **perfil**:

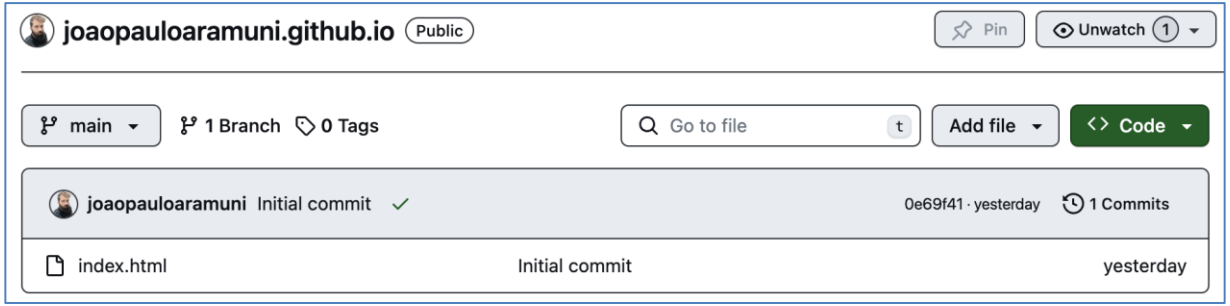


  - Caso tenha problemas nesta etapa com autenticação no Git, consulte os links:
    - https://stackoverflow.com/questions/35942754/how-can-i-save-username-and-password-in-git
    - https://git-scm.com/docs/git-credential-store

- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **perfil**:







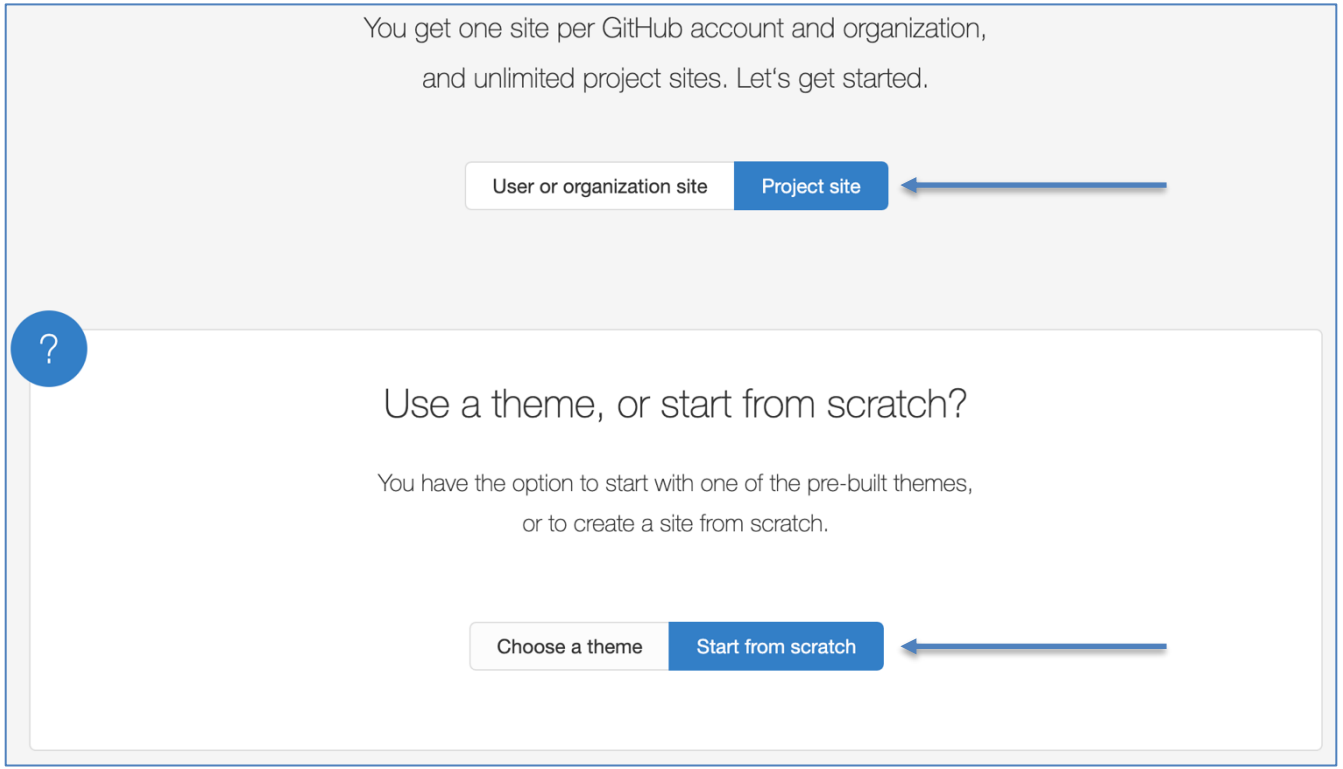
- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **perfil**:

Além de HTML, CSS e JavaScript, o GitHub Pages também aceita Markdown.

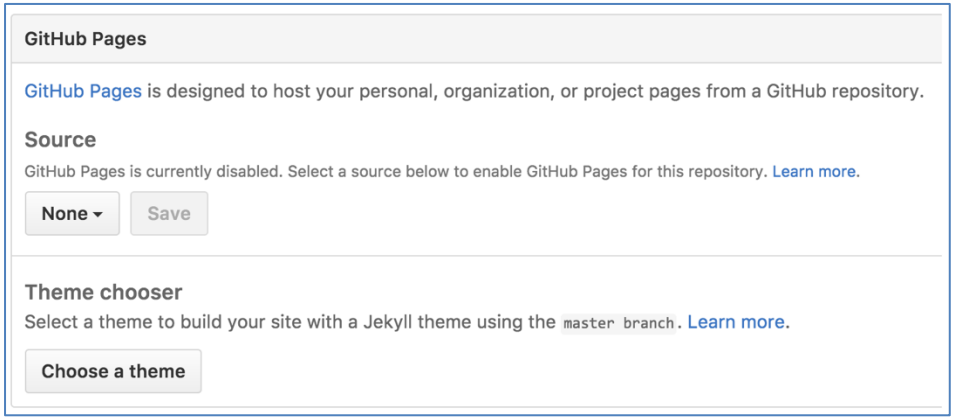
No meu caso, criei um arquivo **index.md** (Markdown) ao invés de **index.html**.



- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **projeto**:

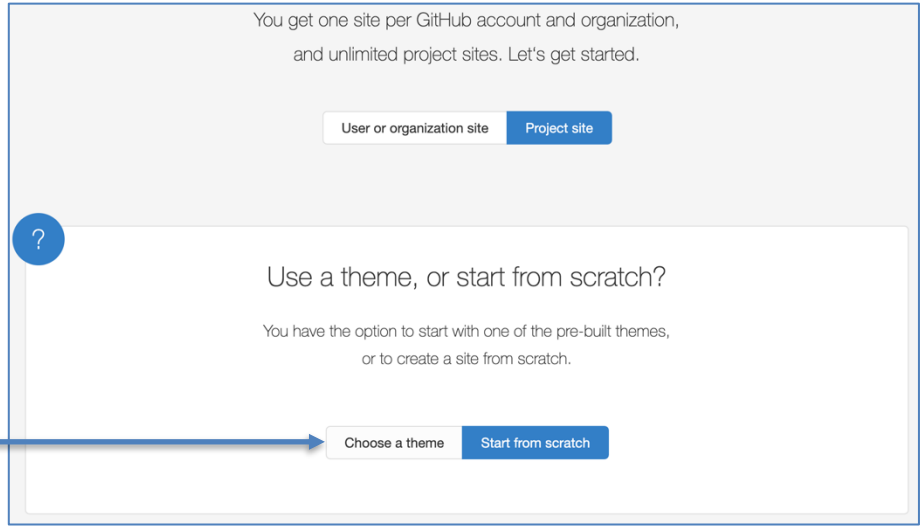


- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **projeto**:



O botão **Choose a theme** foi removido do GitHub

Este link está com o passo a passo defasado

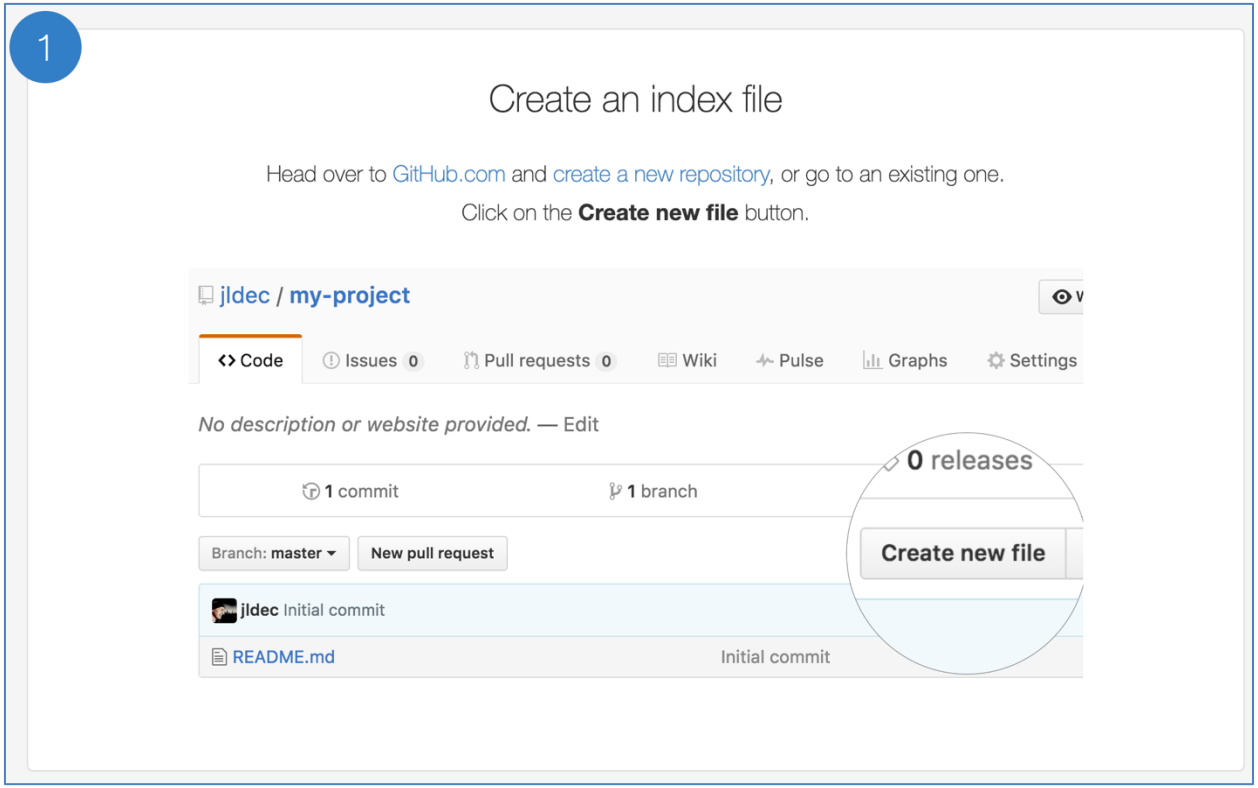


- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **projeto**:
    - Para adicionar um tema ao projeto, leia a documentação:
    - https://docs.github.com/pt/pages/setting-up-a-github-pages-site-with-jekyll/adding-a-theme-to-your-github-pages-site-using-jekyll

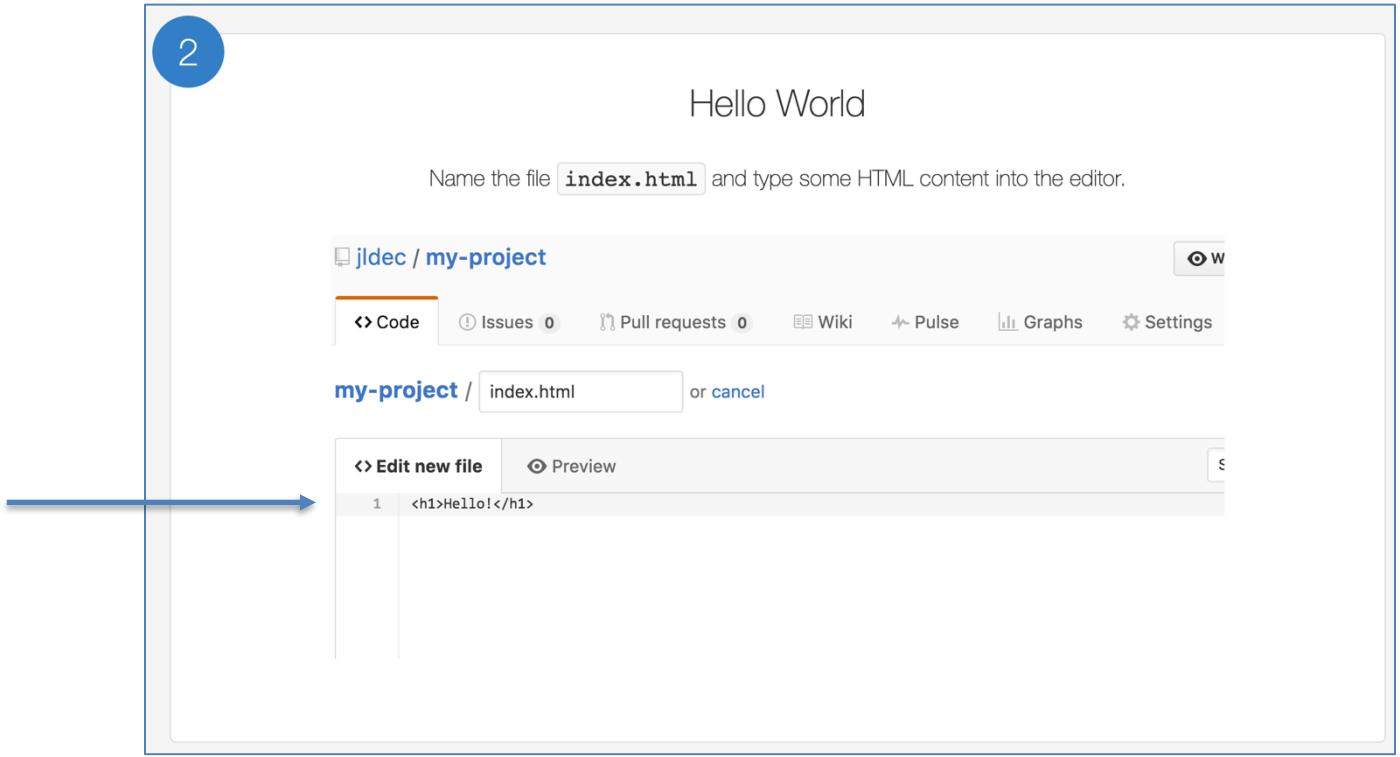


- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **projeto**:

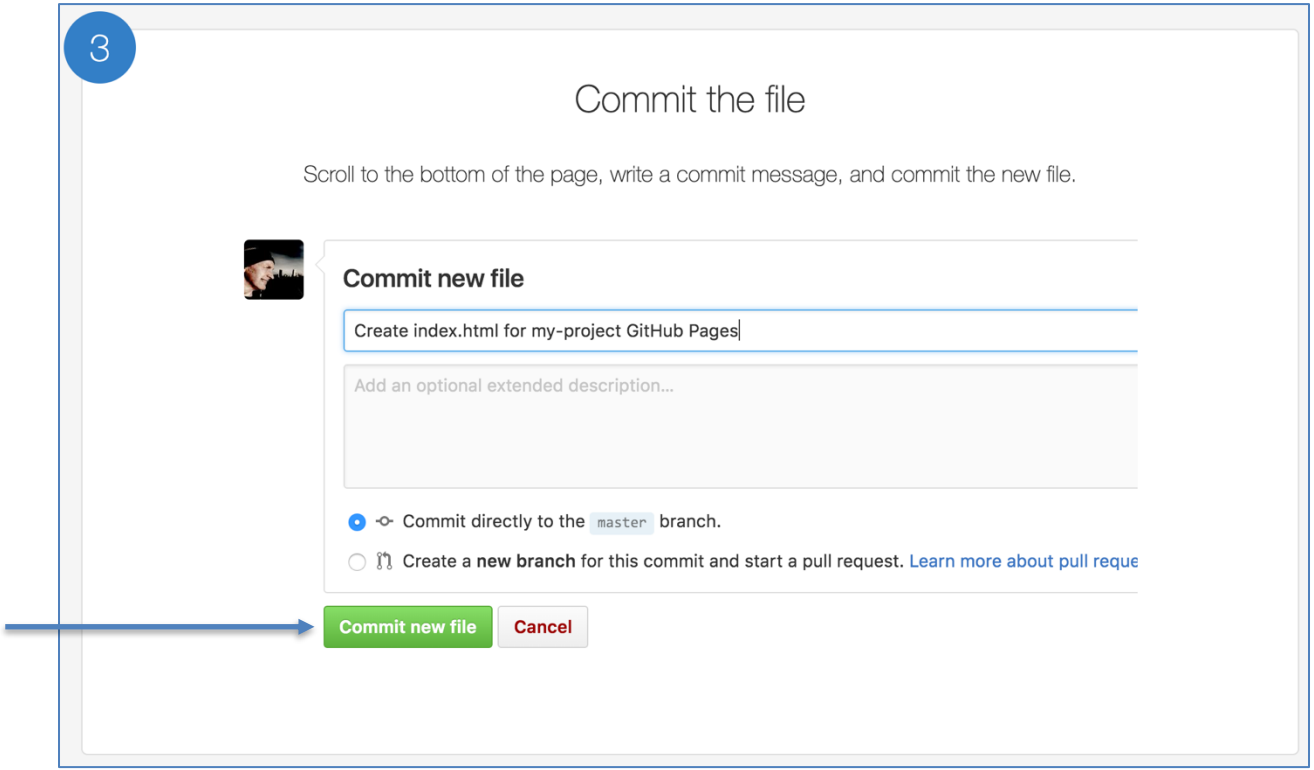
Adicionando o **index.html** manualmente



- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **projeto**:

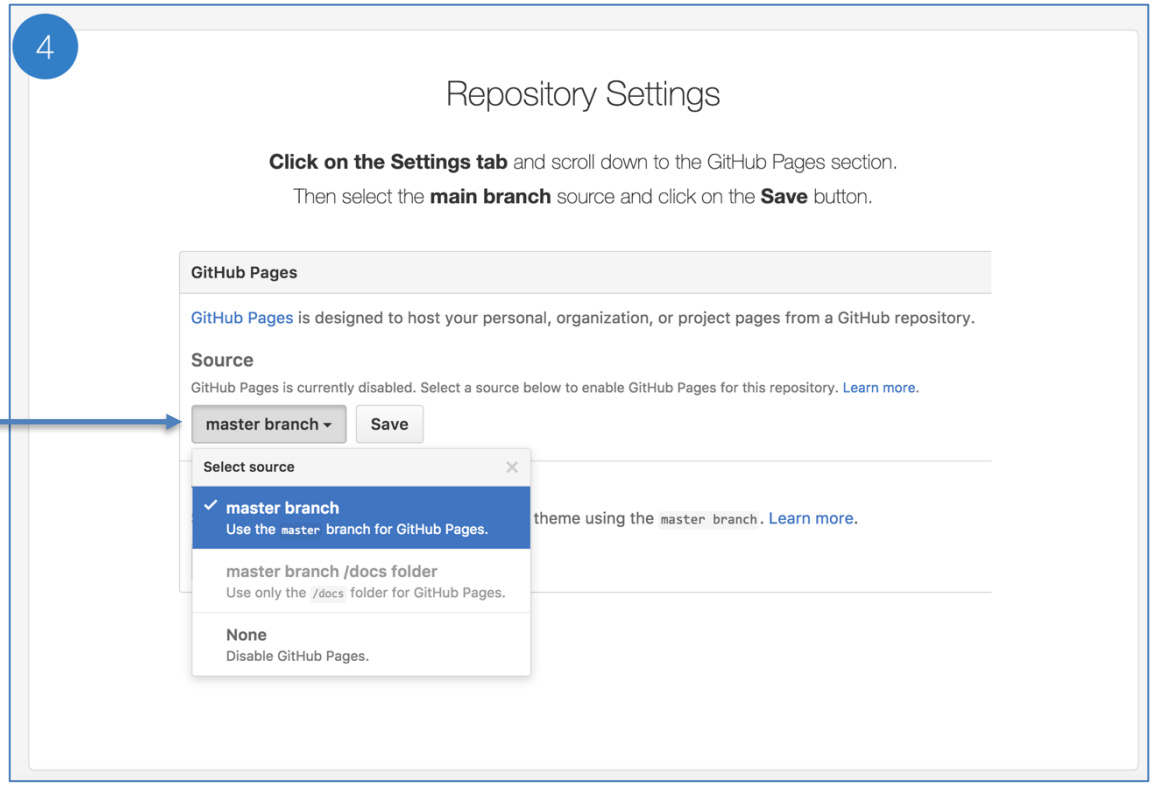


- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **projeto**:

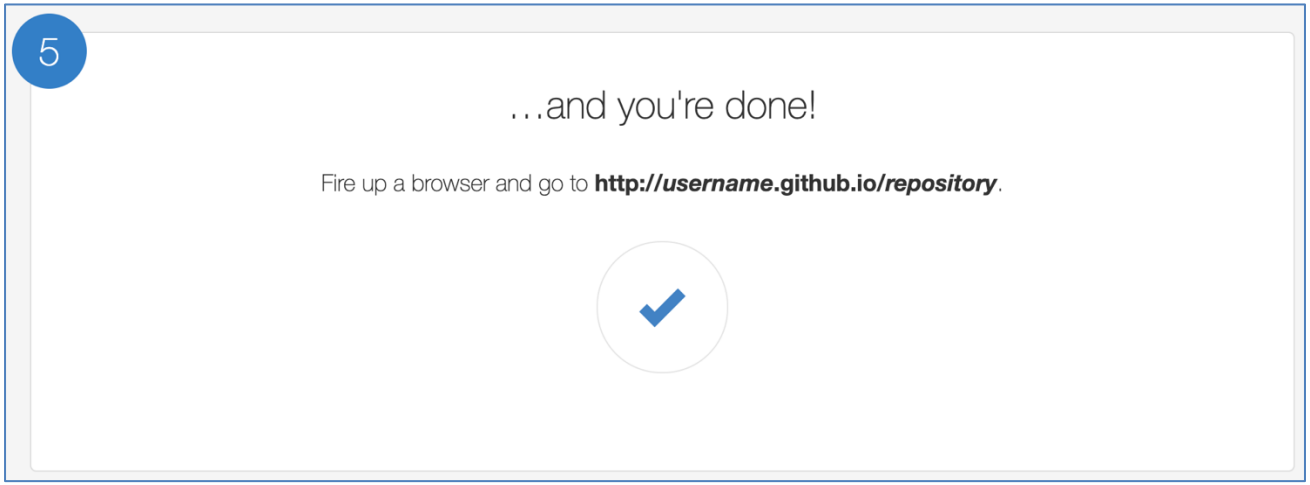


- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **projeto**:

master = main



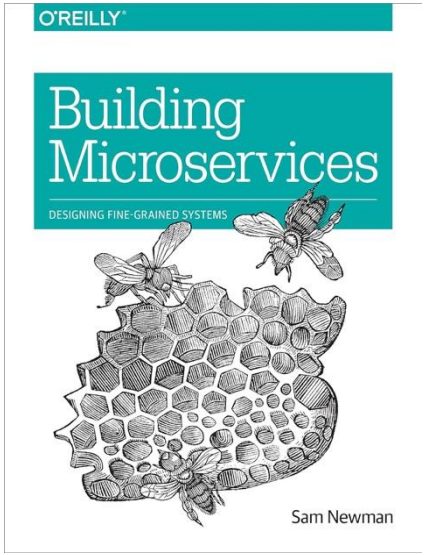
- **GitHub Pages**: Websites para você e seus projetos.
  - Como fazer - para o seu **projeto**:

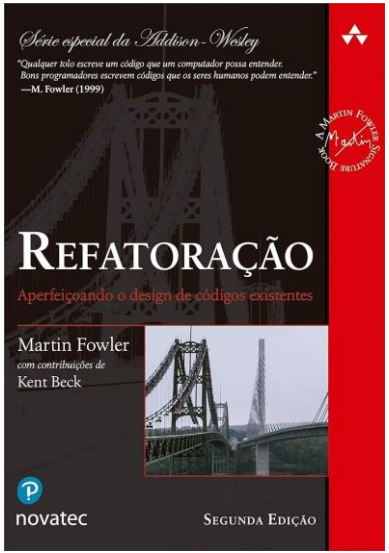


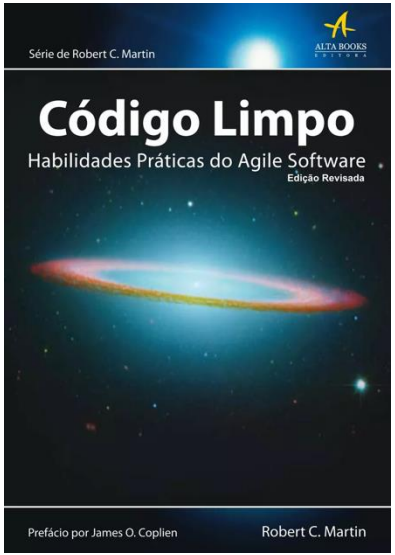
  - Feito isso, basta passar os arquivos da documentação da API e do diagrama de arquitetura para dentro do repositório e referenciá-los no **index.html**.
  - Lembrando que o GitHub Pages aceita: HTML, CSS, JavaScript e Markdown.

Referências básicas

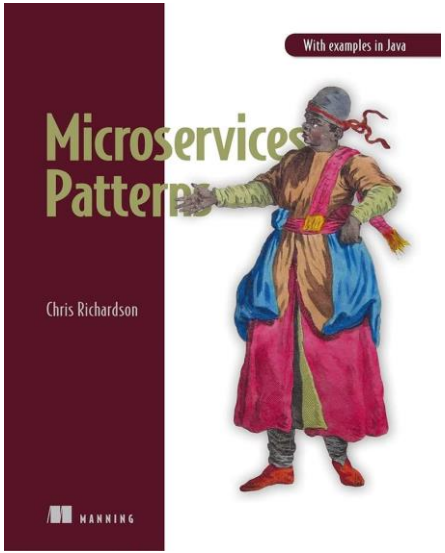


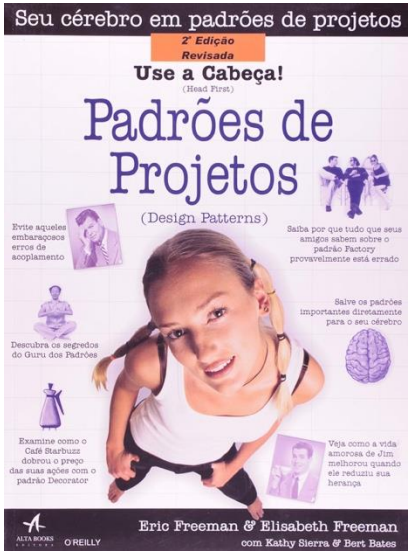


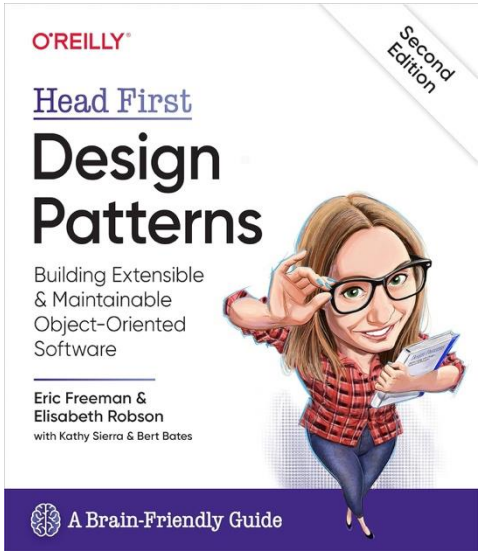


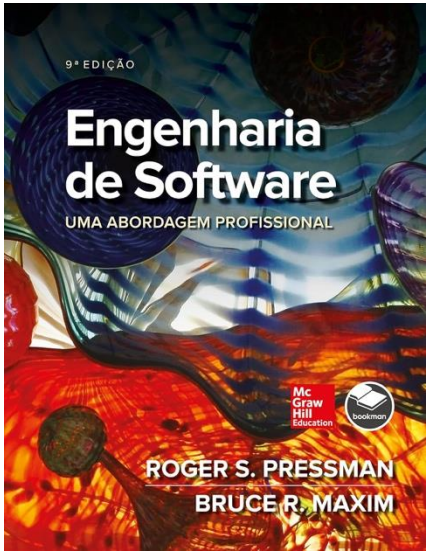


Referências complementares

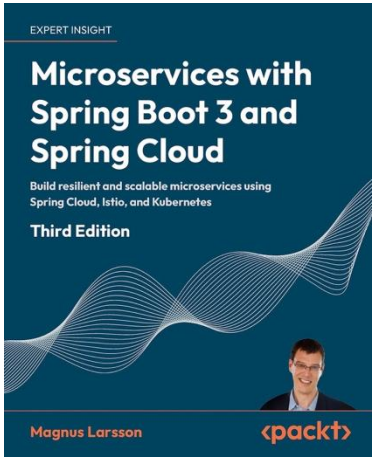




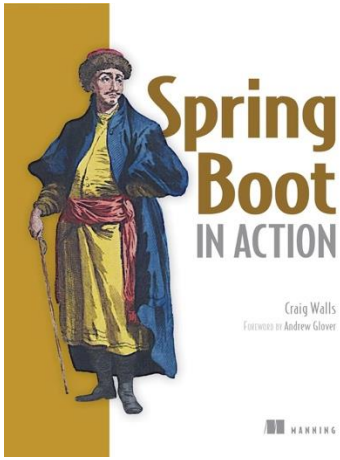


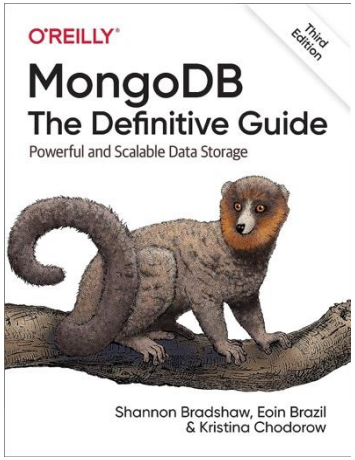


Outras referências

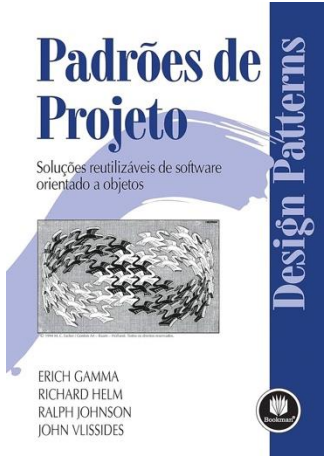












Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com





GitHub LinkedIn Lattes

...